

RAWHIDE SYSTEM PROGRAMMER'S MANUAL

Revision 1.4

Digital Company Confidential



Author: Rawhide Team (Editor: Sam Duncan)

Last Revision: 05/16/96 10:36:21 AM

DIGITAL CONFIDENTIAL
Digital Equipment Corporation
Maynard, Massachusetts

This information shall not be disclosed to persons other than DIGITAL employees or generally distributed within DIGITAL. Distribution is restricted to persons authorized and designated by originating organization. This document shall not be transmitted electronically or copied unless authorized by the originating organization. In addition, this document shall not be left unattended. When not in use, this document shall be stored in a locked storage area. These restrictions are to be enforced until this document is reclassified by the originating organization.

AVS Product Engineering
Digital Equipment Corporation
129 Parker Street.
Maynard
Ma., 01754

May, 96

The drawings and specifications in this document are the property of Digital Equipment Corporation and shall not be reproduced or copied or used in whole or part as the basis for the manufacture or sale of items without written permission.

The information in this document may be changed without notice and is not a commitment by Digital Equipment Corporation. Digital Equipment Corporation is not responsible for any errors in this document.

This specification does not describe any program or product that is currently available from Digital Equipment Corporation, nor is Digital Equipment Corporation committed to implement this specification in any program or product. Digital Equipment Corporation makes no commitment that this document accurately describes any product it may make.

Copyright 1993, 1994, 1995 by Digital Equipment Corporation

All Rights Reserved

Printed in USA

The following are trademarks of Digital Equipment Corporation:

DEC	PDP	VAXstation
DECchip	ULTRIX	VMS
DECnet	UNIBUS	VT
DECUS	VAX	
MicroVAX	VAXBI	
MicroVMS	VAXcluster	

Revision History:

- 1.4 - 9 April, 1996; Updated with all known changes for final release of the IO subsystem. This includes the addition of a Top of Memory, per request from our UNIX partners. Also included is a summary of the uncached processor address aliasing between Interrupt Acknowledge and some dense address spaces. Added Linear address space for EV56 byte/word as on Kodiak. Deleted GCD from most places in the document. Fixed monster window spec in chapter 3.
- 1.0 - 23-August-1995; All reported errors should have been fixed. Both interface chips are in layout and no changes expected. This version generated using a MSWord Macro - some slight formatting changes may be visible. Comments encouraged. Updates expected to Chapter 5 in the area of error logout frame.

Many thanks to all who helped to write this document. In particular, the following people contributed substantial sections or otherwise participated in the development of the manual. The following are key contributors: Walt Arbo, Dick Beaven, Bill Bruce, Lauren Carlson, Steve Coe, Jon Crowell, Bill Cummins, Sam Duncan, Darrel

Revision History and Acknowledgments

Donaldson, Don Dutile, Paul Guglielmi, Dennis Hayes, Craig Keefer, Andy Koning, Dmetro Kormeluk, John Lynch, Dave Mayo, Tom McLaughlin, Steven Shirron, Art Singer, Wilson Snyder, Maurice Steinman, Hitish Vyas, Jerry Wernicki. Sections also lifted from ALCOR, Sable, Lynx, PCIA, PCI, and other standards and specification (thanks to all).

Table of contents

1. INTRODUCTION	1-1
1.1 SUMMARY	1-1
1.2 DEFINITION OF TERMS	1-1
1.3 PRODUCT OVERVIEW	1-2
1.4 PRODUCT CHARACTERISTICS (SYSTEM BUILDING BLOCKS).....	1-4
1.5 PRODUCT CHARACTERISTICS (SYSTEMS).....	1-10
1.6 SUPPORTED OPTIONS	1-14
1.7 IO PERFORMANCE ESTIMATES.....	1-17
1.8 PEER-TO-PEER DMA DEADLOCK AVOIDANCE	1-22
2. RAWHIDE ADDRESSING.....	2-23
2.1 ADDRESS MAPPING INTRODUCTION	2-23
2.2 MCBUS ADDRESS MAP	2-24
2.3 CPU VIEW OF IO SPACE	2-32
2.4 (E)ISA IO ADDRESS SPACE	2-40
2.5 SIZING THE MCBUS THROUGH SOFTWARE	2-42
2.6 EV56 & EV6 ADDRESSING, COMPATIBILITY	2-43
3. PCI TO PHYSICAL MEMORY ADDRESSING.....	3-45
3.1 INTRODUCTION.....	3-45
3.2 DMA TRANSACTIONS.....	3-45
4. CONTROL AND STATUS REGISTERS.....	4-59
4.1 REGISTER TYPES.....	4-59
4.2 PCI DEVICE CONFIGURATION REGISTERS	4-63
4.3 EV5 CONFIGURATION REGISTERS	4-64
4.4 GENERAL REGISTERS - DESCRIPTION	4-68
4.5 INTERRUPT REGISTERS.....	4-78
4.6 PERFORMANCE MONITOR REGISTERS.....	4-85
4.7 DIAGNOSTIC REGISTERS - DESCRIPTION.....	4-85
4.8 ERROR SYMPTOM REGISTERS	4-88
4.9 PCI ADDRESS RELATED REGISTERS: DESCRIPTIONS	4-95
4.10 SCATTER/GATHER ADDRESS TRANSLATION REGISTERS	4-102
4.11 MEMORY CONFIGURATION REGISTERS.....	4-103
4.12 EV5 CONFIGURATION REGISTERS - DESCRIPTION.....	4-103
4.13 STANDARD IO CSRs (EISA/PCI BUS BRIDGE ESC CHIP ETC.)	4-104
5. HARDWARE INTERRUPTS, AND EXCEPTIONS	5-115
5.1 OVERVIEW	5-115
5.2 SYSTEM INTERRUPTS	5-115
5.3 ERROR REPORTING PHILOSOPHY	5-121
5.4 RAWHIDE SYSTEM BUS TO PCI BUS BRIDGE ERRORS	5-121
5.5 EV5 ERROR HANDLING	5-125
5.6 PCI ERROR HANDLING	5-125
5.7 RAWHIDE (MC BUS) ERROR HANDLING	5-128
5.8 ERROR INSERTION	5-131
5.9 DIGITAL UNIX AND OPEN VMS ERRORS	5-132
5.10 MICROSOFT WINDOWSNT ERROR REPORTING	5-142
5.11 ERROR LOGGING INFORMATION	5-153

6. POWER-UP AND INITIALIZATION	6-161
6.1 FIRMWARE OVERVIEW	6-161
6.2 POWER-UP SEQUENCE OVERVIEW.....	6-161
6.3 INITIALIZATION AND TESTING PERFORMED BY SERIAL ROM.....	6-161
6.4 INITIALIZATION AND TESTING PERFORMED BY EXTENDED ROM	6-162
6.5 INITIALIZATION PERFORMED BY CONSOLE.....	6-162
6.6 CSR AND IPR INITIALIZATION	6-164
6.7 I/O SPACE ADDRESS MAP	6-164
6.8 SOFTWARE CONSIDERATIONS.....	6-164

1. Introduction

1.1 Summary

This chapter provides an overview of the Rawhide system components and the key features. The components include; EV5 CPU modules, memory modules, IO bridges and buses, system power and packaging.

1.2 Definition of terms

ALCOR	: high end Alpha workstation designed by DEC's workstation group.
Demi-Sable	: low cost mid-range Alpha server designed in DEC's server group.
Dodge System	: quad processor rackmount Rawhide system with 8 PCI slots, (3 are combo EISA/PCI) and up to 8GB of memory.
ASIC	: Application Specific Integrated Circuit
CAP chip	: chip that implements the control and address portion of the MC-bus to PCI-bus bridge.
Durango System	: VOD (video on demand) version of the Rawhide system. Rackmountable dual processor with 16 PCI slots and up to 4GB of memory.
EDO DRAM	: "Extended Data Out" DRAM, new asynchronous DRAM that allows a faster page mode operation.
EISA bus	: Standard PC bus for optional peripheral adapters.
EV-5	: Second generation of CMOS5 Alpha chips. EV45 was the first generation of CMOS5 Alpha chips. (EV45 was a shrunk version of EV4. EV4 was the CMOS4 version of Alpha chip.)
EV-56	: Third generation of CMOS5 Alpha chips.
GID	: Global ID—identifies the MCbus building block. On Rawhide, this is always #7.
Horse Module	: Circuit board that contains the CAP chips and the MDP chips for implementing the complete MC-bus to PCI-bus bridge.
MC-bus	: the bus that interconnects CPU, memory, and I/O on the same motherboard; also called the local bus.
MDP chip	: chip that implements the data path portion of the MC-bus to PCI-bus bridge.
memory holes	: locations in the Rawhide memory map where physical memory is discontiguous do to the mixing of different size memory card pairs.
memory channel	: follow on to reflective memory, developed in DEC, utilizes a write only protocol for implementing high performance clusters
MID	: MCbus node ID—the physical node/slot number of a module in the MCbus.
PCI bus	: the new generation standard I/O bus for PCs. Now being used in DEC's Alpha servers.
PLL	: phase lock loop.

RM	: “reflective memory”, taken from the Encore approach a write only protocol for connecting systems. Also known as Memory Channel.
Sable	: mid-range Alpha server designed in DEC's server group
Saddle Module	: PCI backplane that connects to the Horse module. This module also includes the EISA bus bridge, EISA bus, X-bus, and all the standard I/O logic.
Stampede Hub	: A newly designed interconnect box that allows up to 8 systems to be connected in a memory channel cluster. The system must have a PCI memory channel adapter installed and then connects to the hub with a 10 ft cable.
Synchronous DRAM	: new architecture for DRAM that operates synchronously, requires a clock and has significant internal control that eliminates the need for complex external memory control logic.

1.3 Product Overview

Rawhide is a set of core logic designs used to build EV5-based SMP systems. The design center allows 1-4 CPU's on the motherboard with 1-4 memory slots (each slot is a pair of modules), and 1-4 bus bridges to PCI. Configuration limitations do not allow the maximum number of each device type to coexist on a system building block at the same time. The maximum number of logical slots on a single Rawhide motherboard is limited to between 8 and 16, depending on specific motherboard configuration. The memory subsystem supports two memory module types, a high performance SYNCHRONOUS DRAM (SDRAM) memory module, and high performance EXTENDED DATA OUT DRAM (EDO DRAM) memory. A memory channel PCI bus adapter and memory channel Stampede Hub provide a way to loosely couple a cluster of Rawhide building blocks.

The Rawhide core logic will be used in EV5, EV56 and EV6 processor based systems. This standard logic set will provide the following features:

- Extensible PCI bridge logic allowing 2 or more 64 bit wide PCI buses to be spawned by a single system bus.
- Only one EISA bus is supported in all SMP configurations¹. This limitation avoids the design complexity of deadlock avoidance when more than one EISA bus exists in a system².
- High bandwidth memory and system bus capability supporting both SDRAMs and standard EDO DRAMs for constructing premium high performance systems.

¹ ISA/(E)ISA are product requirements because a few devices not available on the PCI (and may never be). For example, Token ring, serial/parallel ports, FAX modem, etc.

² The EISA bus has no mechanism to back it off of the PCI once it has acquired it. It must therefore be guaranteed to gain access to all necessary resources before it can be granted the PCI. With two such buses in the same system, either all I/O buses must be arbitrated as though they were a single I/O bus or multiple EISA buses should be avoided. In Rawhide we chose to limit to a single EISA bus since all major options will be available on PCI.

The block diagram for the quad processor mother board used in the Dodge system is show below :

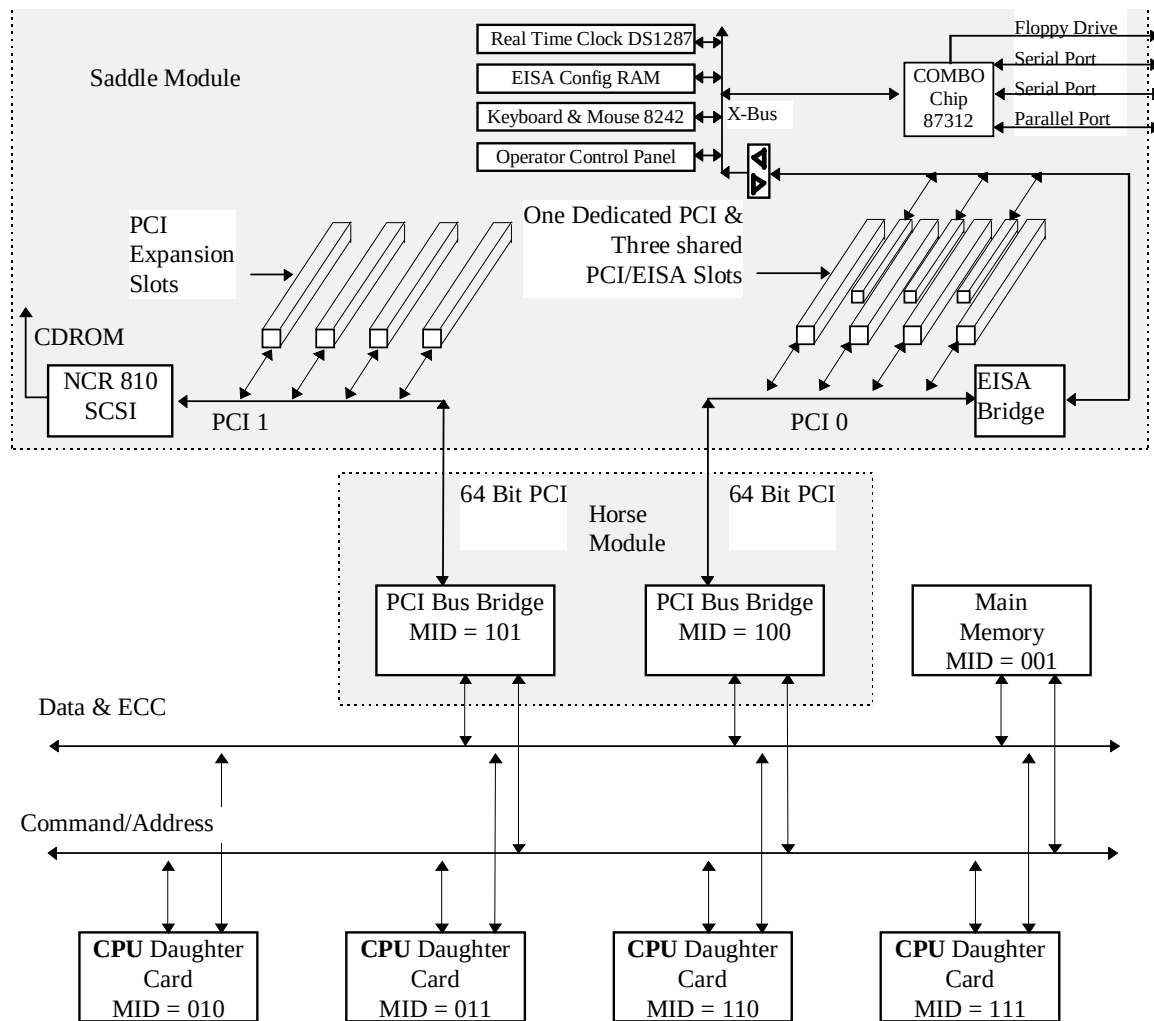


Figure 1-1 Full Rawhide System Configuration Block Diagram³

All configurations will be supported by OpenVMS, Digital UNIX, and WindowsNT. A standard re-configurable console will be provided that supports all configurations. It is a goal that all possible EV5 CPU variations (speeds and cache sizes) will be supported by the initial OS releases. This will be done by passing all hardware specific information to the operating systems at boot time.

It is a goal of the project to design all components such that they can be easily packaged for sale through channels (Channel ready). This needs to be considered by all design teams; each team will consider and describe how this will effect each design.

³ MID is MC Bus Module ID number. It is a hard wired geographical address.

1.4 Product Characteristics (System building blocks)

1.4.1 Rawhide system bus

All Rawhide systems are designed around a standardized local system bus (MC_Bus). There is a single standard Rawhide system bus on each Rawhide motherboard.

1.4.1.1 Rawhide System Bus (MC_BUS) characteristics:

- Bus width 128 bits + ECC
- ECC is generated when data is stored and checked by the consumer of the data when it is consumed (e.g.; EV5 or IO bridges)
- Supports multiprocessing, 1-4 CPUs, 1-4 IO bridges, 1-4 Memory module pairs.
- Maximum of 10 nodes (example: 4 CPU, 4 MEM, 2 PCI).
- Runs synchronous to the CPU and memory
- Bus cycle time is a multiple of the CPU clock with a minimum cycle time of 15ns per data beat for a 3nS CPU clock cycle time

cpu_osc	400	666.33	600.0	588.235	550.00	533.333	MHZ
system_osc	66.593	66.593	59.962	58.7858	54.9635	53.2973	MHZ
EV_clock_speed (nS)	2.5	3.0	3.33	3.4	3.64	3.75	nS
EV_clock_speed (MHz)	400	333	300	294	275	266.	
MC_bus_clock_speed	15	15	16.6	17	18.2	18.75	nS
PCI Clock Speed	30nS 33MHz	30nS 33MHz	30nS 33MHz	30nS 33MHz	30nS 33MHz	30nS 33MHz	

Figure 1-2 CPU Frequency vs. System Bus Frequency

- 1 GB/s asymptotic bandwidth(Usable bandwidth: 948 MB/s SYNC-MEM read, 853 MB/s EDO-MEM read)
- Distributed clocking using low cost PLL circuits (~\$100-200/system)
- Central arbitration via logic on the motherboard.
- Bus designed to function using low cost non-controlled impedance circuit boards (8 layer and up)
- The Rawhide system bus (MC_BUS) is fully specified in the Rawhide Bus Spec, Rev 1.0, by Glenn Herdeg. (Copies can be obtained from POBOXA::HERDEG)

1.4.2 CPU Modules

Clock speeds and cache sizes and actual performance of implementations that become products are the subject of another document. This describes the range of possible designs on this generation of Rawhide.

1.4.2.1 EV5 Uncached CPU module

The CPU module uses an EV5 CPU with no backup cache. It uses the EV5 internal 96 KB on-chip, write-back cache to yield good performance when used with the high bandwidth SYNC-MEMORY option.

- Single EV5 CPU with no external cache, system bus interface logic, and PLL clock, yielding a low cost EV5 CPU with reasonable performance.
- Module and design supports all future variants of EV5 and EV56. The following cycle clocks are possible with this design 16.6 nS -18.75 nS.

1.4.2.2 EV5 Cached CPU module

The CPU module uses an EV5 CPU with an on module backup cache and duplicate tag. The cached CPU module is form compatible with the Uncached EV5 CPU modules.

- Single EV5 CPU chip with supporting a 1MB, 2 MB, and 4MB Synchronous cache, duplicate tag store, system bus interface logic, and PLL clock, yielding the highest EV5 CPU performance when used with SYNC-MEMORY.
- Module and design supports all future variants of EV5 and EV56. The minimum SYSCLOCK multiple is 5 and a minimum MC_BUS cycle time of 15 nS.

1.4.3 Memory modules

The Rawhide memory modules provide system main memory via the Rawhide system bus (MC_BUS). There are two technologies of memory module being designed for the system. These are :

- 1) The Synchronous memory module (SYNC_MEMORY), providing the highest bandwidth (948MB/Sec Read - 853MB/Sec Write) and lowest latency (120 nS to first data) at a premium cost.
- 2) The EDO memory module (EDO-MEMORY), providing lower performance than the SYNC_MEMORY (10% slower), with costs approximately xx% higher than standard DRAM chips.

With the Rawhide system memory modules **must** be used in pairs as each module covers half of the 128 bit Rawhide system bus. The system customers will see part numbers for pairs of memory modules and size listed as the combined size. Any combination of memory modules can be used in a Rawhide system. The only requirement is they must be added in matched pairs. SDRAM and EDO memory cards can be mixed in a system but the diagnostic implications are not currently fully understood.

When large memory card pairs are used with smaller memory card pairs, the cards must be inserted in the mother with the largest capacity card installed in the lowest order slot. The mixing of large card pairs with smaller card pairs results in memory holes. (Memory holes are discussed in a later section.)

1.4.3.1 SYNC-MEMORY module

The synchronous memory module uses synchronous DRAM parts for the highest performance subsystem. It is critical in systems where the highest memory bandwidth is needed, such as with quad processor system scaling. The SYNC-MEMORY has the following specifications:

- Bandwidth: 948 MB/s READ, 853 MB/s WRITE
- Latency: 120 nS READ (8 ticks from address at pins of EV chip to first data back at pins of EV chip and 165 nS to last data at the pins of the EV), n nS WRITE (n ticks)
- Supported module pair sizes (each module size is half the pair size listed); 32 MB, 64 MB, 128 MB and 256 MB using 16 Mbit chips. (The implementation of the 32 MBytes option is being questioned at this time. It is feasible but may not be necessary.) When 64 Mbit chips are available the memory capacity will increase by a

factor of 4 allowing memory module pairs to provide 512 MB and 1 GB of memory. When the 64 Mbit memory chips reach the cost break even point with the 16 Mbit memory chips then smaller memory cards can be made from the 64 Mbit chips.

1.4.3.2 EDO-MEMORY module

The EDO memory module uses extended data out DRAM parts for a high performance memory subsystem. It allows for a memory cost that tracks industry DRAM prices while providing good system characteristics. It will be used where cost is important over absolute maximum system performance. The EDO-MEMORY module uses custom ASICs to control the DRAMS and interfaces to the Rawhide system bus. It has the following specifications:

- Bandwidth: 853 MB/s READ, 853 MB/s WRITE
- Latency: 135 nS READ (9 ticks from address at pins of EV chip to first data back at pins of EV chip and 180 nS to last data at the pins of the EV), n nS WRITE (n ticks)
- Supported module pair sizes (each module size is half the pair size listed); 64 MB, 256 MB, and 512 MB using 16 Mbit chips. When 64 Mbit chips are available the memory capacity will increase by a factor of 4 allowing memory module pairs to provide 1 GB of memory.

1.4.3.3 Memory Holes

One of the Rawhide memory subsystem requirements is to be able to mix different memory card sizes on the same motherboard. In past systems this mixing of memory card sizes was managed in hardware by the memory control ASIC. Since there is no memory ASIC in a system with SDRAM memory, this traditional approach could not be used. Instead of managing different size memory cards in hardware the Rawhide system takes advantage of the fact that all of our operating systems have built in support for mapping clusters of memory pages. Console firmware only describes existing memory and does not create cluster descriptors for memory holes. A memory cluster is created for each region of contiguous memory. If bad memory is detected in any one of these regions, that memory is marked bad.

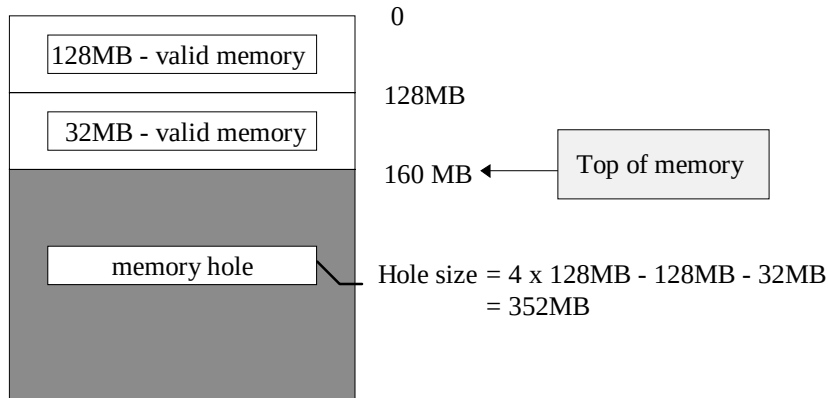
There are some requirements imposed on the system for this approach to work :

- Memory daughter cards must be installed in matched pairs, i.e. the pair must be the same size.
- Hardware will detect the size of memory installed in the lowest order memory slot.
- Hardware will then set its memory decode logic to treat all daughter cards as being the same size as the memory installed in the lowest order memory slot.

In order to understand the implications of this approach four cases are described below. The first three cases cover configurations where the operating systems won't see memory holes and the fourth case shows configurations that will see memory holes.

CASE 1 :

A large memory card pair is installed with a smaller memory card pair with the larger memory card pair installed in the lowest order memory slot. For example, assume a 128 MByte memory daughter card pair is installed in the lowest order memory slot, and a 32 MByte memory daughter card pair is installed in the next memory slot. The memory map for this configuration is show below (lower order addresses start at the top of the memory map) :

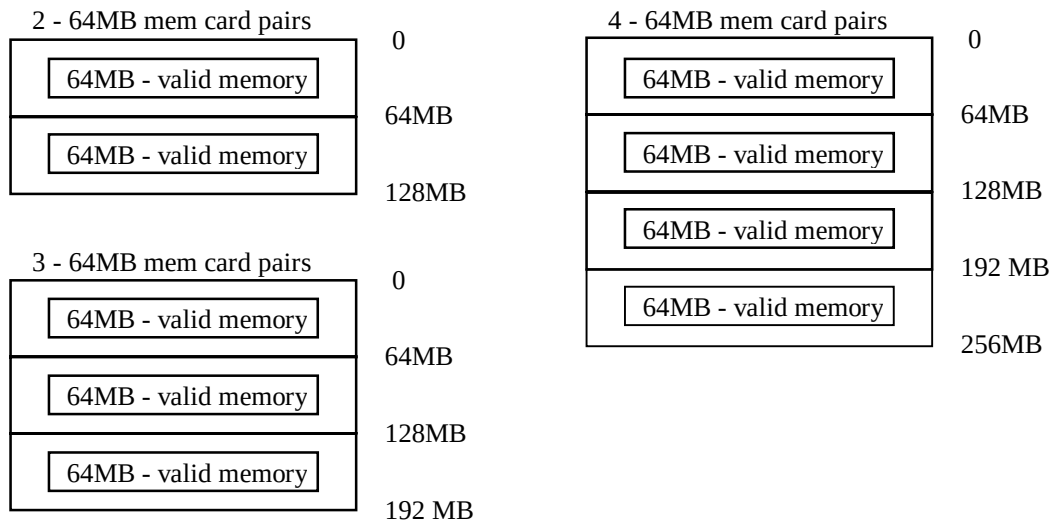


For this configuration of memory the hole is above the top of memory, so from an operating system perspective there is no memory hole just 160 MByte of contiguous memory. For two memory card pairs, as long as the largest memory daughter card pair is installed in the lowest order memory slot, any combination of two daughter card pairs will result in a contiguous memory map, that is no memory holes.

Case 2 :

All of the memory daughter cards installed in a mother board are the same size. In this situation the memory has no memory holes. All of the memory will be contiguous. Shown below are the various memory configurations for 64 MByte memory cards :

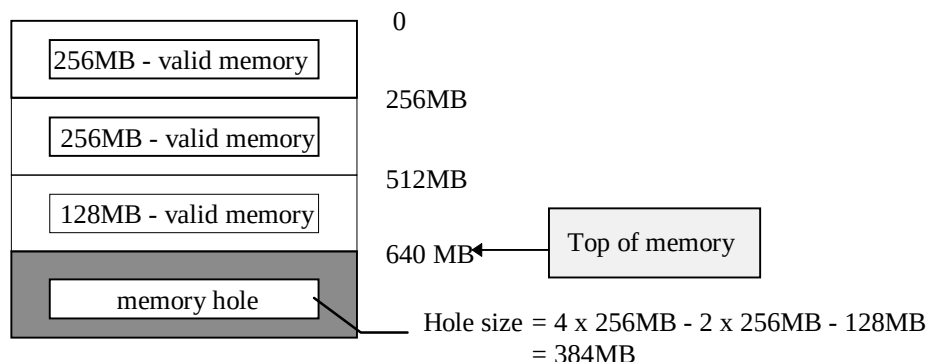
64MB memory card configurations



Although 64 MByte memory are shown in this example, the 128 MByte and 256 MByte memory cards behave in exactly the same manner. For configurations where all memory cards are the same size, any number of memory daughter card pairs will result in memory being contiguous, as long as the memory slots are populated in order.

Case 3 :

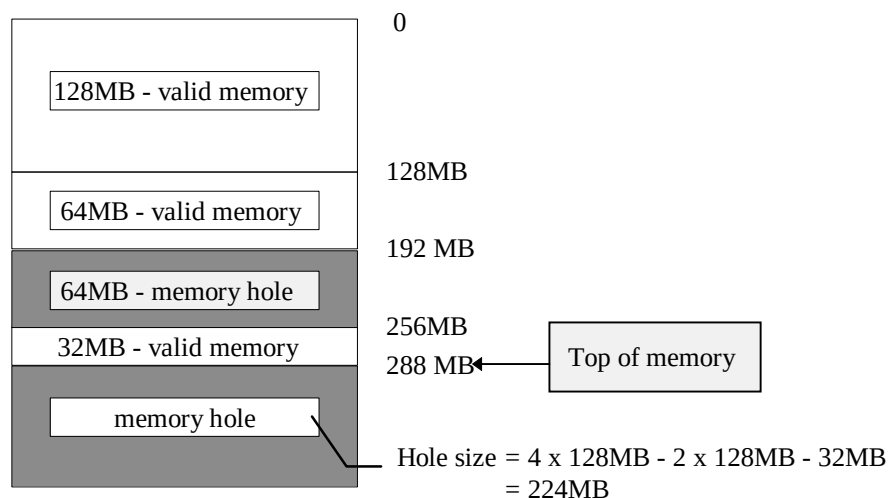
If 3 or 4 memory card pairs are installed in a mother board with only one smaller memory card pair and all of the other larger memory card pairs are the same size, then the memory will appear contiguous. This is a combination of the first case and the second case. The larger memory card pairs must be installed in the lower order memory slots and the small memory card pair must be installed in the memory slots just above the larger memory cards. For example assume there are two 256 MByte memory card pairs and one 128 MByte memory card pair. The memory map would appear as follows :



This case is similar to the first case. The memory hole appears above the top of memory. The operating system will see 640 MByte of contiguous memory.

Case 4 :

The final case is where memory holes are seen by the operating systems. If 3 or 4 memory daughter card pairs are installed in a motherboard and at least two of the pairs are smaller capacity memories than the others then memory holes will exist. As an example, if one 128 MByte memory daughter card pair is installed in a motherboard with a 64 MByte memory daughter card pair and a 32 MByte memory daughter card pair, the following memory map would result :



In this example there will be one 64 MByte memory hole just above the 192 MByte memory address region. The 96 MByte memory hole, just above the 288 MByte memory address region, will appear above the top of memory in this memory configuration and therefore is not seen by the operating system. The maximum number of memory holes that will be seen by the operating system in a given memory configuration is $N-2$. Where N is the number of memory daughter card pairs installed in the motherboard. In this example there are 3 memory daughter card pairs installed in the motherboard so there is 1 memory hole visible to the operating system. The maximum number of

memory holes that a single motherboard can have is 2, since a single motherboard only support four memory daughter card pairs.

1.4.4 MC_BUS to PCI Bus Bridge Logic

The PCI bus bridge spawns a 64 bit wide PCI bus from the Rawhide MC_BUS. It is implemented in a set of three ASICs that provide the following characteristics:

- High performance connection for PCI IO devices in Rawhide systems.
- System supports multiple independent PCI buses (1-4) with the use of additional PCI bus bridge chip sets.
- Each bridge chip set (ASICs) implements a single 64 bit wide PCI bus
- Same basic functionality as the ALCOR PCI bus bridge with the addition of the following;
 - 1) Support for multiple PCI buses off a single system bus,
 - 2) Support for peer-to-peer DMA with software managed deadlock avoidance.
 - 3) Direct mapped Dual Address Cycle "Monster Window" (1 TByte: 2^{40} bytes)
- The PCI bus bridge uses MC_BUS interrupt transactions to pass interrupts to processors in the system.

1.4.4.1 MC_BUS to PCI bus bridge (Horse) and PCI Backplane (Saddle) modules

The bridge module, known as the Horse module, plugs in to the Rawhide MC_BUS and provides the physical as well as logical bridge between the MC_BUS on the motherboard and the PCI backplane. The Horse module implements the PCI Bus bridge logic for two independent 64 bit PCI buses. The PCI backplane, known as the Saddle module, plugs onto the Horse module and contains the PCI bus connectors, EISA bridge logic, EISA bus connectors, X-bus bridge, and standard I/O logic. The first PCI bus supports a PCI-to-EISA bridge chip set that spawns the EISA bus. The standard I/O logic is mapped either on to the PCI bus or the EISA. The interrupts from the IO devices and bus slots are fielded on the Horse module and passed to the correct processor(s) for service. There are no operating system visible considerations on the MC to PCI bus bridge. The two PCI buses are completely independent. The presence or lack of presence of a second PCI bridge does not have any specific operating system implications. There are no module specific CSRs. Once firmware sets the bridge up the operation of the bridge is virtually transparent to the operating system.

PCI Bus slots (Dodge Configuration)	Total: 8 slots, (Bus 0: 4 , Bus 1: 4) (5 slots are dedicated, 3 slots combination PCI/EISA)
EISA bus slots (Dodge Configuration)	3 combination PCI/EISA slots (bridged from Bus 0)
Standard IO Functionality	1- SCSI adapter, 8 bit, single ended (NCR810) 2- Combo chip (X-bus, 87312) - 2 serial, 1 parallel, 1 floppy, plus keyboard and mouse controller 3- TOY Clock (bq1287); real time clock, 114 bytes NVRAM, periodic interrupt 4- interrupt controller (up to 22 independant interrupt lines per PCI bus segment) 5- FEPRM for console (Size: 1MB) 6- I ² C micro-controller for interfacing with the Rawhide system monitoring sensors and the Sable OCP for control functions; HALT BUTTON, RESTART BUTTON, HALT ENABLE, DCOK

1.4.5 Power Supplies

1.4.5.1 RAWHIDE Wide Tower Supply (Proposed for use in Rawhide Modular Packaging)

- The RAWHIDE WT Supply (modified WT Supply) is being considered for the Dodge and Durango configurations of the Rawhide system. Dodge is a rack mountable quad processor Rawhide system. Durango is a rack mountable dual processor Rawhide system.

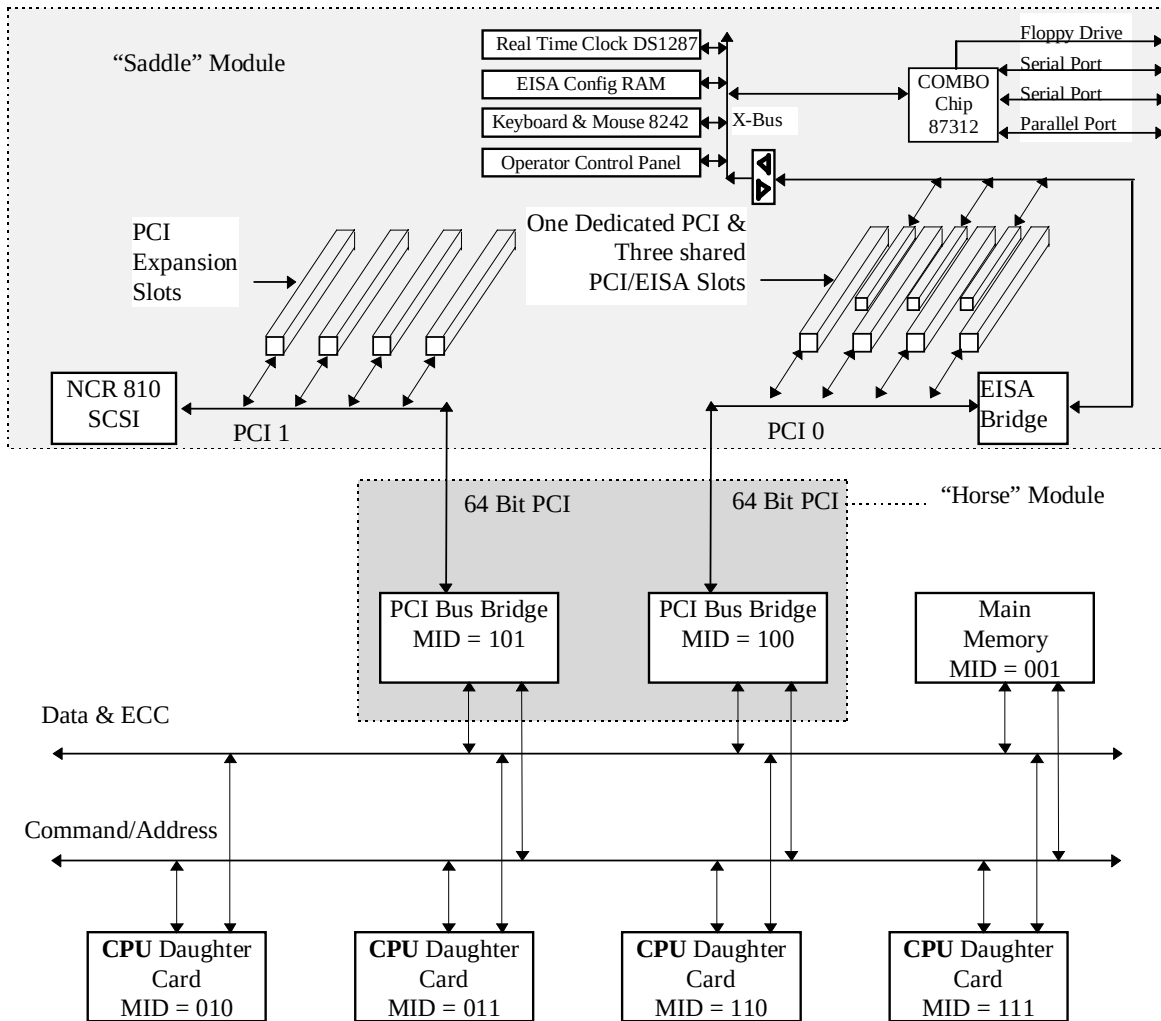
1.5 Product Characteristics (Systems)

1.5.1 Rawhide Modular Rack-mount System Enclosure (a.k.a. Dodge)

The Rawhide Modular Rack-mount system enclosure is the general compute server configuration, also known as the Integrated I/O and Processor (Dodge) configuration. The rack cabinet is a standard 19" cabinet. Configuration:

CPU's	1-4 Alpha (cached or Uncached)
I/O Buses	2 PCI 64-bit buses and 1 EISA bus with the following slot counts: 5 dedicated PCI slots, 0 dedicated EISA slots 3 PCI/EISA combo slots
Integral I/O	the following do not require the use of an I/O slot: 1 floppy disk 1 CD-ROM 2 serial ports 1 parallel port 1 keyboard and mouse controller 1 PCI-to-EISA bridge/control 1 Demi-Sable OCP support
Memory	4 memory daughter card pairs, supporting up to 1 GB using 16 Mbit RAMs, up to 4 GB using 64 Mbit RAMs. Memory daughter card types: SYNC DRAM: 32 MB, 64 MB, 128 MB, 256 MB (16 Mbit) 512 MB, 1 GB (64 Mbit) EDO DRAM: 64 MB, 256 MB (16 Mbit) 1 GB (64 Mbit)
Power Supply	up to 3 modified (3.3V) wide tower supplies Power: TBD W tbd amps @ 3.3V tbd amp @ 5.0 V tbd amp @ -5.0V tbd amp @ 12.0V tbd amp @ -12.0V
Enclosure	Rack-mount: Height: 480mm (12 inches) Width: xmm (TBD inches) Depth: xmm (TBD inches)
Device bays	1 floppy drive 1 CD-ROM 0 3 ½" hard disk drives (mount hard drives separately in rack)
FY96 Cost (\$goal/\$curr)	\$x/\$x base system cost, including: 1 CPU 32 MB memory 1 CD-ROM 1 floppy disk drive 1 power supply

Introduction

Figure 1-3 Dodge System⁴ Configuration Block Diagram

Software can size the system by reading device type and revision over the I²C bus or by reading the Device Type registers⁵. Firmware is expected to size, initialize, and configure the PCI bus bridge, and PCI devices. Chapter 5 describes the initialization and configuration sequence followed by firmware.

⁴ Workstation version of Rawhide is expected to deliver a Dodge crate with EV56 CPU Daughter cards.

⁵ Only MC-PCI bus bridges have Device ID registers.

1.5.2 Rawhide Modular Rack-mount Expanded IO Enclosure (a.k.a. Durango)

The Rawhide Modular Rack-mount expanded I/O configuration, also known as the PCI I/O (PIO) configuration, is targeted at the video-on-demand market. This configuration will also be used for high-demand multiple I/O device systems. Configuration:

CPU's	1-2, Alpha (cached or Uncached)
I/O Buses	4 PCI 64-bit buses and 1 EISA bus with the following slot counts: 16 dedicated PCI slots (no EISA slots at this time)
Integral I/O	the following do not require the use of an I/O slot: 1 floppy disk 1 CD-ROM 2 serial port 1 parallel port 1 keyboard and mouse controller 1 PCI-to-EISA bridge/control 1 Demi-Sable OCP support
Memory	2 memory daughter card pairs, supporting up to 512 MB using 16 Mbit RAMs, or up to 4 GB using 64 Mbit RAMs (when available). Memory daughter card types: SYNC DRAM: 64 MB, 128 MB, 256 MB (16 Mbit) 512 MB, 1 GB (64 Mbit) EDO DRAM: 64 MB, 256 MB (16 Mbit) 1 GB (64 Mbit)
Power Supply	up to 3 modified (3.3V) wide tower supplies Power: TBD W tbd amps @ 3.3V tbd amp @ 5.0 V tbd amp @ -5.0V tbd amp @ 12.0V tbd amp @ -12.0V
Enclosure	Rack-mount: Height: 480mm (12 inches) Width: xxmm (TBD inches) Depth: xxmm (TBD inches)
Device bays	1 floppy drive 1 CD-ROM 0 3 ½" hard disk drives (mount hard disk drives separately in rack)
FY96 Cost (\$goal/\$curr)	\$x/\$x base system cost, including: 1 CPU 32 MB memory 1 CD-ROM 1 floppy disk drive 1 power supply

Introduction

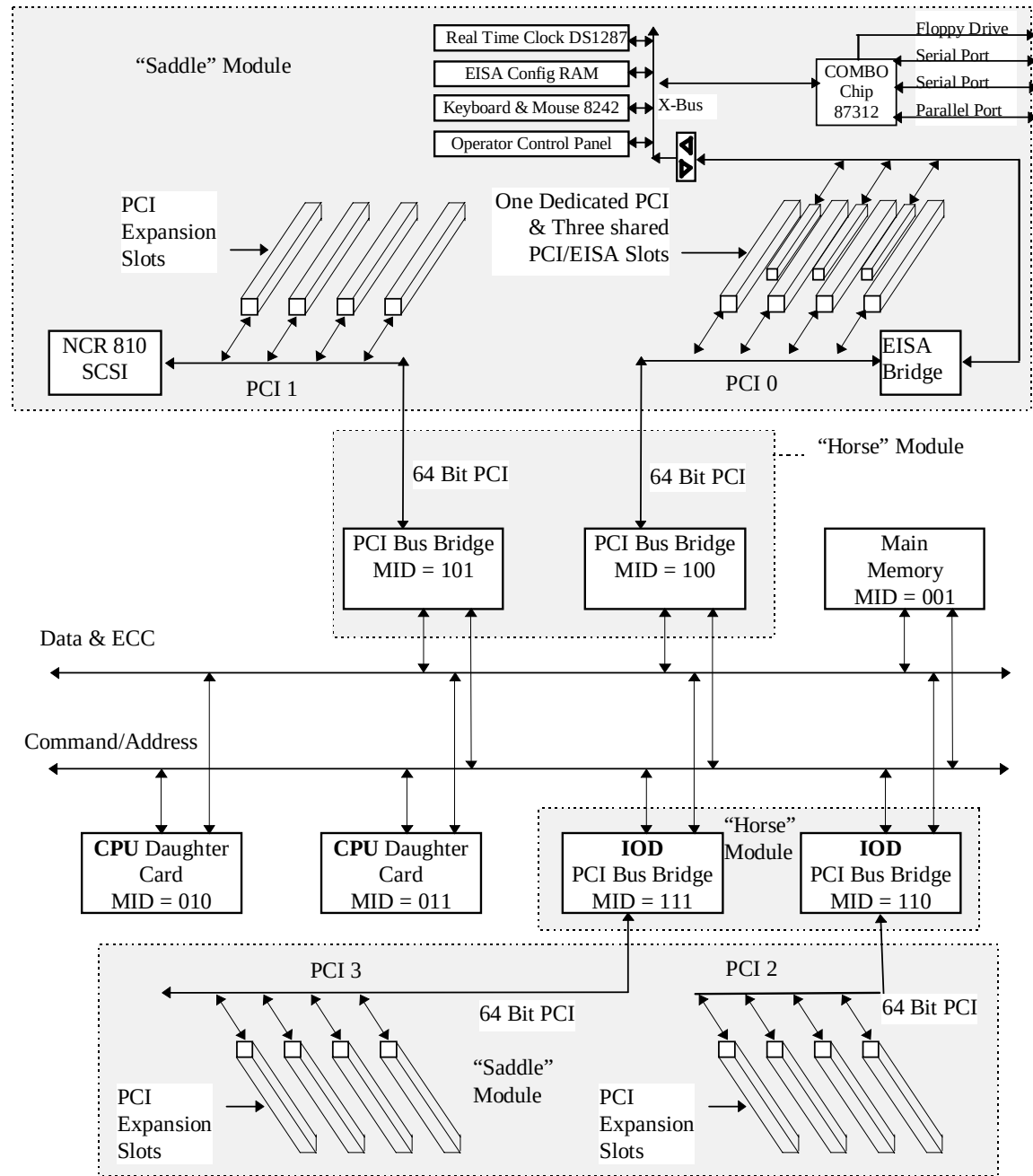


Figure 1-4 Durango System Configuration Block Diagram

1.6 Supported Options

Rawhide Required Options at FRS

See the RAWHIDE_QUAL notes conference for the most up to date listing of supported options.

- Uncached CPU Physical characteristics:

Module size	13 inches wide, 6 inches high (note: Module size will be a subset that fits all planned system enclosures including; sable and modular rack enclosures)
DEC part number	B3001-AA
Power consumption	x Watts, x Amps @3.3V, x Amps @5V
Schedule:	Power on:2/95 // Group 2 (tbd) // FRS : Q2'FY96
FY96 Cost (goal\$/cur\$)	\$ x / \$ x

- Cached CPU Physical characteristics:

Module size	x inches wide, x inches high (note: Module size will be a subset that fits all planned system enclosures including; sable and modular rack enclosures)
DEC part number	B3002-AA
Power consumption	x Watts, x Amps @3.3V, x Amps @5V
Schedule:	Power on: // Group 2: // FRS:
FY96 Cost (goal\$/Cur\$)	\$ x / \$ x

- SDRAM-Memory module physical characteristics:

Module size	x inches wide, x inches high (note: Module size will be a subset that fits all planned system enclosures including; sable and modular rack enclosures)
DEC part number	MS3020-AA, BA, CA, DA
Power consumption (module pair)	x Watts, x Amps @3.3V, x Amps @5V [first] x Watts, x Amps @3.3V, x Amps @5V [4 pairs]
Schedule:	Power on:2/95 // Group 2 (tbd) // FRS : Q2'FY96
FY96 Xfr cost: (goal\$/ current \$)	\$ x / \$ x (64 MB - MS3030-BA) \$ x / \$ x (128 MB - MS3020-CA) \$ x / \$ x (256 MB - MS3030-DA)

- EDO-Memory module/logic physical characteristics:

Module size	x inches wide, x inches high (note: Module size will be a subset that fits all planned system enclosures including; sable and modular rack enclosures)
DEC part number	MS3030-BA, DA
Power consumption (module pair)	x Watts, x Amps @3.3V, x Amps @5V [first] x Watts, x Amps @3.3V, x Amps @5V [4 pairs]
Schedule:	Power on: // Group 2: // FRS:
FY96 Xfr cost: (goal\$/ current \$)	\$ x / \$ x (64 MB - MS3030-BA) \$ x / \$ x (256 MB - MS3030-DA)

- MC_BUS to PCI Bridge (Horse) Physical characteristics:

Performance	DMA read bandwidth: 200 MB/s (xfr size: 512MB) DMA write bandwidth: 250 MB/s (xfr size: 512MB) LW read bandwidth: 6 MB/s LW write bandwidth: 33 MB/s Read target latency: 540 nS
DEC part number	B3040-AA, AB
Power consumption	x Watts, x Amps @3.3V, x Amps @5V [first] x Watts, x Amps @3.3V, x Amps @5V [4 pairs]
ASIC technology	Hudson CSALT-5Lite (or LSI Logic 500K)
Schedule:	ASIC Power on: 5/95// Group 2: // FRS: Q2'FY96
FY96 Xfr cost: (goal\$/ current \$\$)	\$ / \$ (logic set for 1 bus bridge)

- PCI Backplane (Saddle) Physical characteristics:

Performance	(same as B3040)
DEC part number	B3050-AA, AB
Power consumption	x Watts, x Amps @3.3V, x Amps @5V [first] x Watts, x Amps @3.3V, x Amps @5V [4 pairs]
Schedule:	Power on: 5/95// Group 2: // FRS: Q2'FY96
FY96 Xfr cost: (goal\$/ current \$\$)	\$ x / \$ x

- Second MC_BUS to PCI Bridge (Gelding: Durango) Physical characteristics:

Performance	(same as B3040)
DEC part number	B3041-AA
Power consumption	x Watts, x Amps @3.3V, x Amps @5V [first] x Watts, x Amps @3.3V, x Amps @5V [4 pairs]
ASIC technology	Hudson CSALT-5Lite (or LSI Logic 500K)
Schedule:	ASIC Power on: 5/95// Group 2: // FRS: Q2'FY96
FY96 Xfr cost: (goal\$/ current \$\$)	\$ x / \$ x (logic set for 1 bus bridge)

- Second PCI Backplane (Side Saddle: Durango) Physical characteristics:

Performance	(same as B3050)
DEC part number	B3051-AA
Power consumption	x Watts, x Amps @3.3V, x Amps @5V [first] x Watts, x Amps @3.3V, x Amps @5V [4 pairs]
Schedule:	Power on: 5/95// Group 2: // FRS: Q2'FY96
FY96 Xfr cost: (goal\$/ current \$\$)	\$ x / \$ x

- Dodge Motherboard Backplane (IIP) Physical characteristics:

Performance	?????
DEC part number	54-23803-01
Power consumption	x Watts, x Amps @3.3V, x Amps @5V [first] x Watts, x Amps @3.3V, x Amps @5V [4 pairs]
Schedule:	Power on: 2/95// Group 2: // FRS: Q2'FY96
FY96 Xfr cost: (goal\$/ current \$\$)	\$ x / \$ x

- Durango Motherboard Backplane (PIO) Physical characteristics:

Performance	?????
DEC part number	54-23805-01
Power consumption	x Watts, x Amps @3.3V, x Amps @5V [first] x Watts, x Amps @3.3V, x Amps @5V [4 pairs]
Schedule:	Power on: ?/95// Group ?: // FRS: Q?'FY96
FY96 Xfr cost: (goal\$/ current \$\$)	\$ x / \$ x

1.7 IO Performance Estimates

The following tables present the results of queuing model simulations and hand calculations (done by Paul Guglielmi, Susan Carr, and Zarka Cventanovic) using the MC bus specification and MC-PCI bus bridge design information supplied by the IOD design team. They represent our working assumptions about how IO performance on Rawhide.

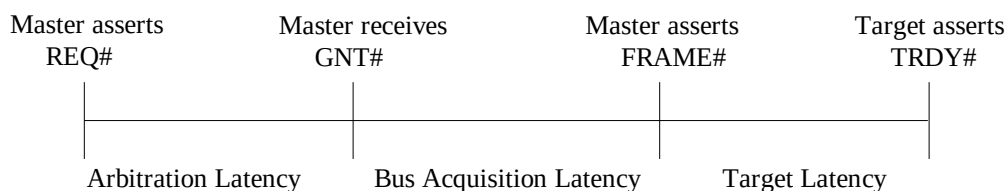


Figure 1-5 Components of Access Latency⁶

	Arbitration Latency		PCI Bus Acquisition Latency		Delayed or Pended Read Latency	Target Latency	
	Idle Bus	Busy Bus	Idle Bus	Busy Bus		Best Case ⁷	Worst Case
DMA Local Memory Read Direct Mapped	60nS	30nS + Master Latency Timer value *4 (worst)	30nS	30nS + Master Latency Timer value (worst)	NA	540nS (18PCI cycles)	540nS + MC Arbitration Latency
DMA Local Memory Read TLB hit	"	"	"	"	NA	600nS (20 PCI cycles)	600nS + MC Arbitration Latency
DMA Local Memory Read TLB miss	"	"	"	"	NA	900nS (30 PCI cycles)	(?25 PCI cycles?) + MC Arbitration Latency *2
Peer-Peer DMA Read on Local PCI	"	"	"	"	target device dependent	60nS ⁸	target device dependent
Peer-Peer DMA Read across the MC bus	"	"	"	"	???	60nS ⁸	60nS ⁸
All DMA Writes	"	"	"	"	0	60nS	60nS
PIO (CPU) read	"	"	"	"	410nS	60nS	Device Dependent
PIO (CPU) write	"	"	"	"	NA	60nS	Device Dependent

⁶ This figure came from the PCI specification version 2.1.

⁷ Best case assumes a 66Mhz system bus.

⁸ When the bridge target latency is longer than 32 PCI cycles the target is required to use the PCI Delayed Read Protocol. When the transaction is successfully retried target latency is as shown. TLB miss and peer-peer DMA are always delayed read operations.

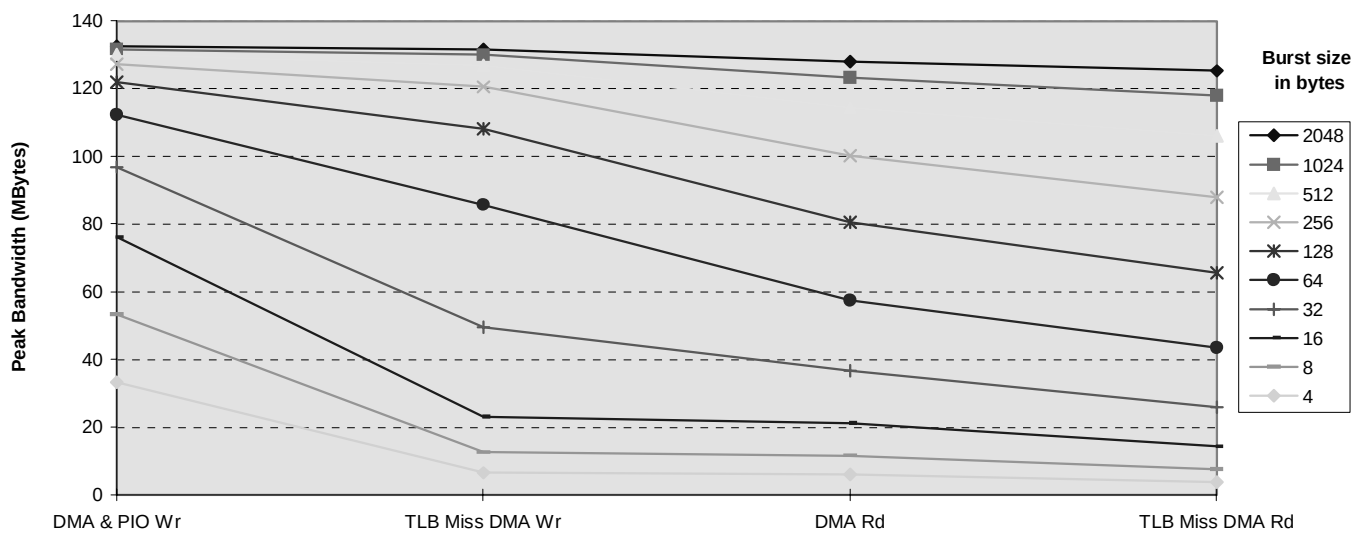


Figure 1-6 Peak Performance on a 32 bit PCI as a function of burst size and transaction type

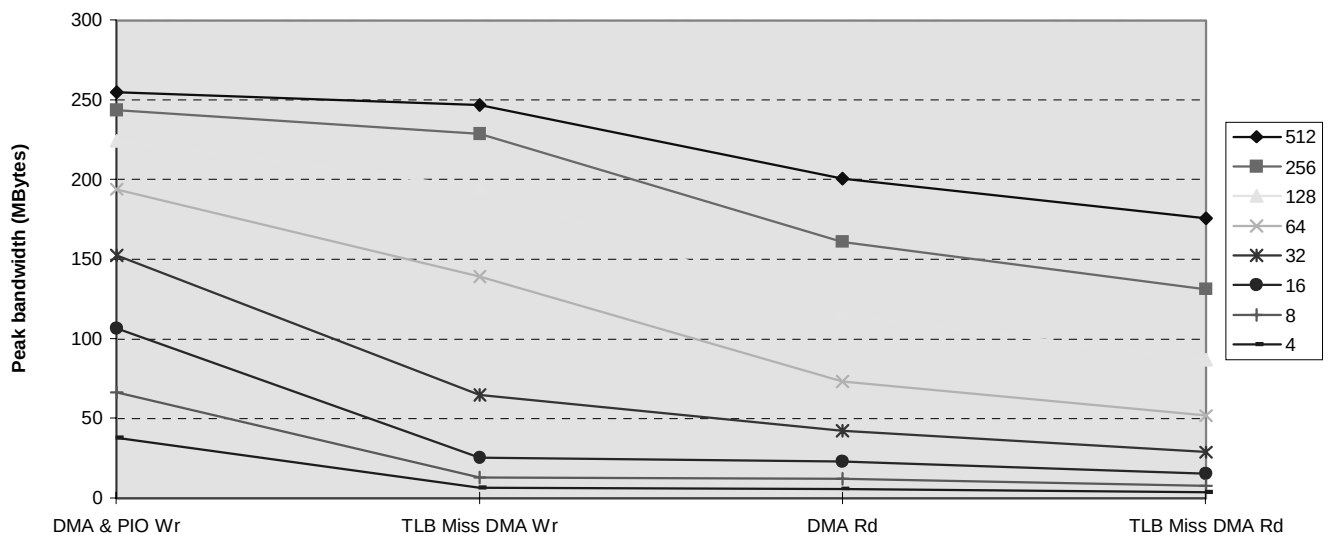


Figure 1-7 Peak Performance on a 64 bit PCI with a system bus running at 15nS (66MHz) as a function of burst size and transaction type⁹

⁹ Read performance is a function of target latency. On systems running the MC bus at a slower speed the target latency will be longer and performance will vary directly as a function of system bus speed.

Frequently asked questions:

- 1) Does Rawhide IO support PCI fast back-to-back?
As a target we do. As a master (initiator) we do not because the address and data come from different ASICs and we need to leave a turnaround cycle between driving data at the end of a transaction and driving address at the beginning of the next transaction.
- 2) Why is Rawhide a medium DEVSEL speed device?
To be a fast DEVSEL device we would only have 8nS to decode the address and assert DEVSEL. We opted for simplicity.

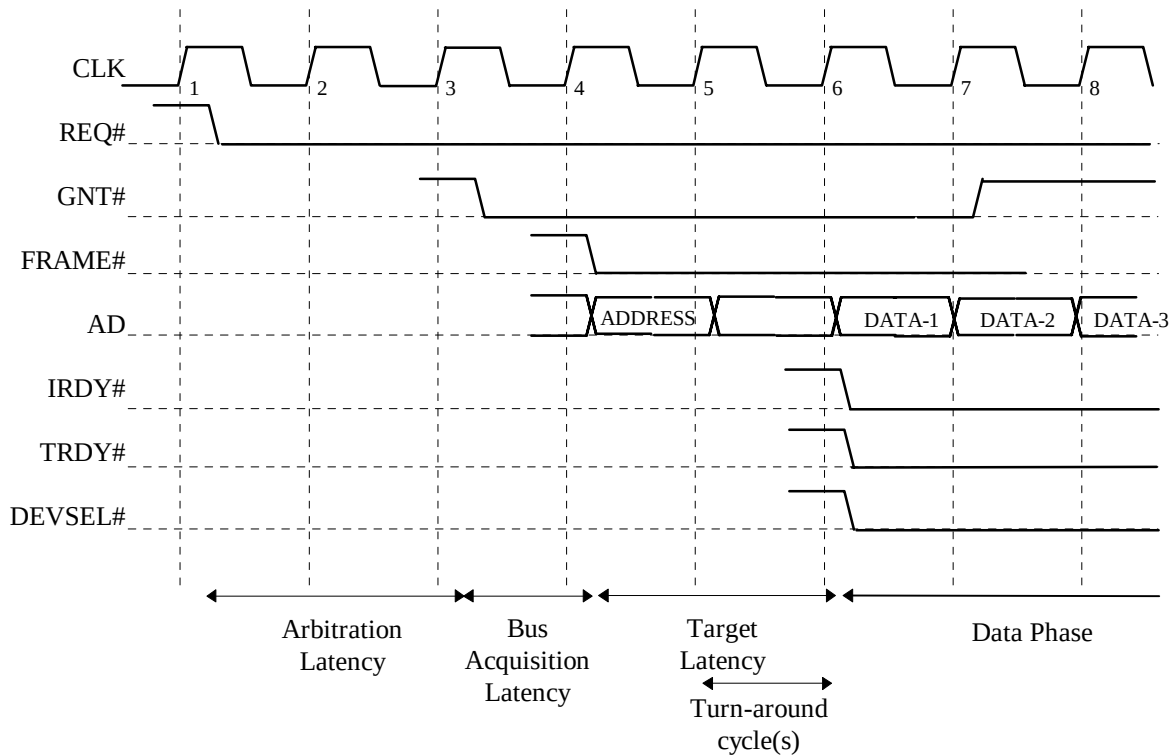


Figure 1-8 Arbitration & read transaction example

Introduction

							#####
Read target latency	540	ns					
DMA read - 64 bit PCI							
Bus turnaround	30	ns					
Burst Size	16	32	64	128	256	512	bytes
Data xfer time	60	120	240	480	960	1920	ns
BW MB/s	25.4	46.4	79.0	121.9	177.8	213.3	MB/s
Queuing model			88.8	131.4	172.6	204.6	MB/s
DMA writes - 64 bit PCI							
Bus turnaround + C/A	60	ns					
Burst Size	16	32	64	128	256	512	bytes
Data xfer time	60	120	240	480	960	1920	ns
PCI limited BW	133.3	177.8	213.3	237.0	251.0	258.6	MB/s
RMW timing diagram	66.7	118.5					MB/s
Queuing model	66.6	118.3					MB/s
Write probe timing diagram	88.9	152.4	213.3	237.0	251.0	258.6	MB/s
Queuing model	88.8	152.4	212.9	236.6	250.5	258.1	MB/s
DMA read - 32 bit PCI							
Bus turnaround	30	ns					
Burst Size	16	32	64	128	256	512	bytes
Data xfer time	120	240	480	960	1920	3840	ns
BW MB/s	23.2	39.5	61.0	88.9	106.7	118.5	MB/s
Queuing model			66.6	88.3	105.0	116.1	MB/s
DMA writes - 32 bit PCI							
Bus turnaround + C/A	60	ns					
Burst Size	16	32	64	128	256	512	bytes
Data xfer time	120	240	480	960	1920	3840	ns
PCI limited BW	88.9	106.7	118.5	125.5	129.3	131.3	MB/s
RMW timing diagram	59.3	97.0					MB/s
Queuing model	59.2	96.9					
Write probe timing diagram	76.2	106.7	118.5	125.5	129.3	131.3	MB/s
Queuing model	76.1	106.5	118.2	125.2	129.1	131.0	MB/s

Table 1-1 Performance Estimates - DMA to memory in the same building block

Introduction

Peer to peer writes - 64 bit PCI							
Target C/A + bus turn around	90	ns					
Burst Size	16	32	64	128	256	512	bytes
Data xfer time	60	120	240	480	960	1920	ns
Target PCI limited BW	106.7	152.4	193.9	193.9	193.9	193.9	MB/s
Write timing diagram - 0 unpend buf	29.6	53.3	88.9	88.9	88.9	88.9	MB/s
Queuing model							MB/s
Write timing diagram - 1 unpend buf	66.7	112.3	177.8	177.8	177.8	177.8	MB/s
Queuing model							MB/s
Write timing diagram - 2 unpend buf	88.9	152.4	193.9	193.9	193.9	193.9	MB/s
Queuing model							MB/s
Peer to peer read - 64 bit PCI							
Fixed overhead	870	ns					
Burst Size	16	32	64	128	256	512	bytes
Data xfer time	120	240	480	960	1920	3840	ns
BW MB/s	16.2	28.8	47.4	57.7	64.7	68.8	MB/s
Queuing model							MB/s
Peer to peer writes - 32 bit PCI							
Target C/A + bus turn around	90	ns					
Burst Size	16	32	64	128	256	512	bytes
Data xfer time	120	240	480	960	1920	3840	ns
Target PCI limited BW	76.2	97.0	112.3	112.3	112.3	112.3	MB/s
Write timing diagram - 0 unpend buf	26.7	44.4	71.1	71.1	71.1	71.1	MB/s
Queuing model							MB/s
Write timing diagram - 1 unpend buf	56.1	88.9	112.3	112.3	112.3	112.3	MB/s
Queuing model							MB/s
Write timing diagram - 2 unpend buf	76.2	97.0	112.3	112.3	112.3	112.3	MB/s
Queuing model							MB/s
Peer to peer read - 32 bit PCI							
Bus turnaround	870	ns					
Burst Size	16	32	64	128	256	512	bytes
Data xfer time	240	480	960	1920	3840	7680	ns
BW MB/s	14.4	23.7	35.0	43.5	49.6	53.3	MB/s
Queuing model							MB/s

Table 1-2 Performance Estimates - Peer to Peer DMA across the MC bus

PIO read - 32 bit PCI		
Burst Size	32	bytes
BW MB/s	38.1	MB/s
Queuing model		MB/s
PIO write - 32 bit PCI		
Burst Size	32	bytes
BW MB/s	100.0	MB/s
Queuing model		MB/s

Table 1-3 Programmed IO performance estimates

1.8 Peer-to-Peer DMA Deadlock Avoidance

Peer-to-Peer DMA is the transfer of data from one IO device to another without going through memory. For example, in Figure 1-9, a DMA write from a device in PCI Expansion slot **A** to a device in PCI Expansion slot **B** would be a local peer-peer DMA. A write from a device in PCI Expansion slot **A** to a device in PCI Expansion Slot **E** would be peer-peer across the MC bus. The features that makes peer-peer DMA possible on Rawhide are posted writes, delayed reads, and reordering of transactions in the IQ to permit posted writes to bypass pending reads. See appendix (Chapter 8) for details.

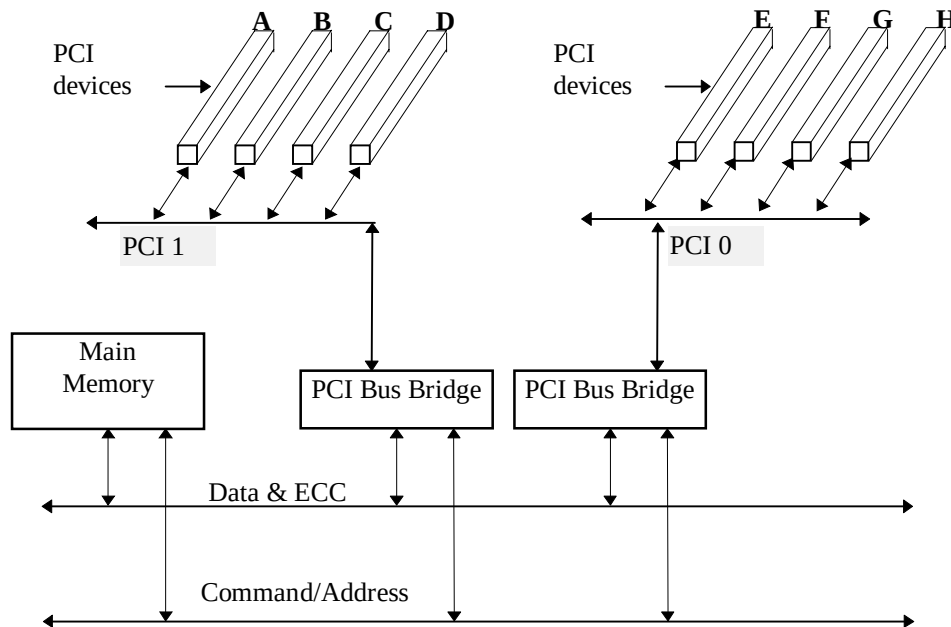


Figure 1-9 Peer-Peer DMA Example

Applications are expected to limit peer-peer DMA target devices to those whose characteristics are similar to main memory. These are devices that will consume write data or produce read data in a deterministic amount of time without requiring use of the PCI to release busy resources on the device.

To avoid deadlocks it is recommended that an initiator of peer-peer DMA not be the target of peer-peer DMA. For example, if Device **A** initiates a DMA read targeting device **E** and at about the same time device **E** initiates a DMA read targeting device **A**, the PCI bus bridge will use the delayed read protocol for both devices. Each bus bridge will initiate an MC bus (pending) read.

The conservative approach is to limit each device to the role of either master or target of peer-peer DMA but not allow a single device to be both peer-peer initiator and peer-peer target at the same time. Rawhide does not support Peer-to-Peer DMA where an EISA device is initiator or target in the transaction. If EISA devices are expected to be used in peer-to-peer DMA then the PCI-EISA bus bridge and all devices on the EISA bus must collectively follow the rule summarized above as if they are one logical device. This means that only one EISA device may participate in peer-to-peer DMA and that device may be either initiator or target but not both.

2. Rawhide Addressing

This chapter describes the mapping of the 40-bit Alpha address into MCBus address space. It explains the translation of processor generated addresses into memory and IO space addresses. The translation of a PCI initiated address into MCBus address space is specified in Chapter 3 but discussed in the context of system address space in this chapter.

Topics covered here include dense, sparse, and linear address space, bus-bridge register addressing, PCI device addressing and EISA requirements. Scatter/gather address translations for DMA operations are discussed in Chapter 3.

2.1 Address Mapping Introduction

As shown and described in Chapter 1, Rawhide consists of modules inside a building block interconnected on the MC bus.

2.1.1 Cacheable and Uncacheable Address Space

The MCBus address space is split into two regions using physical address bit <39>. A transaction with address bit<39> clear targets cacheable address space (main memory) and transactions with address bit<39> set target uncached address space (IO address space).

Cacheable address space is where main memory resides. Rawhide main memory may physically reside and be addressed over a large, discontinuous address space (with bit <39> = 0). Discontinuous clusters of memory may be located on a single MCBus.

Uncached address space is where local MCBus CSRs, IO buses and IO device CSRs reside. Uncached space IO addresses are recognized as MCBus CSR references or are translated into PCI cycles.

2.1.2 Uncached Address Space: PCI bus Bridge

References to uncached address space are to PCI devices and System Configuration registers on the MC bus. PCI has 3 primary address spaces: PCI memory cycles, PCI I/O cycles, and PCI Configuration cycles. These 3 access cycles are mapped into 3 separate, uncached address spaces on the MCBus: PCI memory space, PCI IO space, and PCI Configuration space. PCI Special cycles and PCI INT ACK cycles are implemented through IOD register-space accesses.

Address space on the PCI bridges are fixed in size and location on an MCBus. Each PCI bus uses 8GBytes of MCBus address space. When EV56 Byte/Word transactions are enabled the spaces marked linear are accessible. The 8GBytes is divided into the following sized regions:

- 4 GBytes Sparse PCI memory (128 MBytes byte accessible PCI memory space)
- 2 GBytes Dense or Linear mapped PCI memory space.
- 1 GByte Sparse PCI IO (32 MBytes PCI IO space)
- 512 MByte Sparse PCI Configuration (16MB PCI Configuration space)
- 32 Mbytes Linear PCI IO space
- 16 Mbytes Linear PCI Configuration space
- 512 MByte Pseudo Sparse Bridge Control and Status Registers (16 MBytes IOD CSRs)

2.1.3 Linear Address Space

A future version of the Cached CPU ASIC could support linear IO addressing. The Linear Address windows in the IO ASIC are for compatibility with EV56 byte/word operations and are disabled by default. The current CPU daughter card does not support this because IO interrupt acknowledge address space only decodes bits <39,32:28> such that references to linear windows alias to IO interrupt acknowledge transactions. When enabled, the dense window and linear windows are addressed using the EV56 Byte/Word instructions. When disabled the linear IO and Configuration space are unused and the linear memory window behaves as a dense window.

2.2 MCbus Address Map

As described in the introduction, MCbus address space is divided between memory and IO through bit 39. Table 2-1 shows the top-most address map on an MCbus, as seen by a processor on it.

CPU Address <39:0>	Description
00.0000.0000 - 7F.FFFF.FFFF	(Cacheable) Memory Space
80.0000.0000 - EF.FFEF.FFFF	Reserved Uncached Memory Space
F0.0000.0000 - FF.FFEF.FFFF	IO - Uncached-Memory Space

Table 2-1 RAWHIDE CPU Address Map

2.2.1 CPU to Memory and IO Space Address-Bit Translation

In address-bit fashion, the following describes the CPU Address to PCI bus and bridge IO Space decode/translation in EV5 compatability mode (CAP_CTRL<EV56_BW_EN> = 0; ICSR[17] = 0).

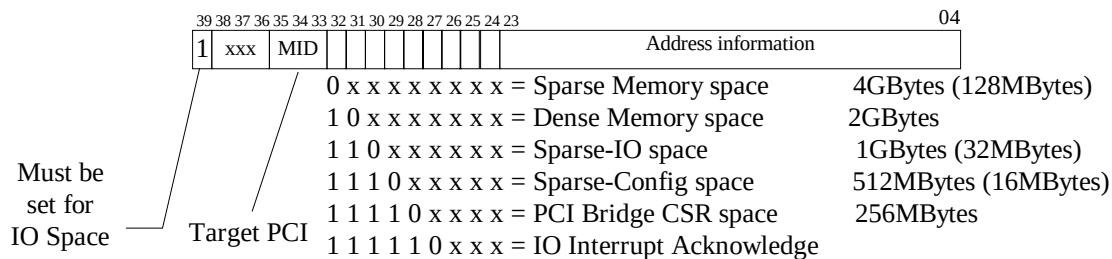


Figure 2-1 IO Space Address Translation with PCI bus bridges as targets, EV5 Compatability mode.

The following describes the CPU Address to CPU IO Space decode/translation. In those PALcode implementations which support CALL_PAL MTPR to IPIR, these locations are accessed only by PALcode and not directly by the Operating System.

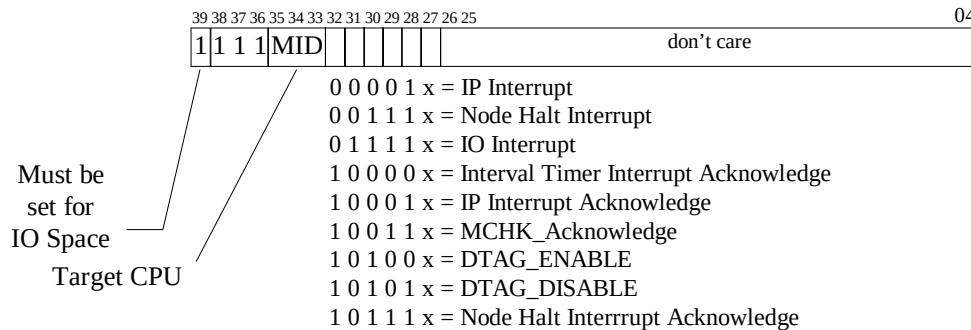


Figure 2-2 IO Space Address Translation with CPU as target¹⁰

The following describes the CPU Address to PCI bus and bridge IO Space decode/translation in EV56 byte/word ECO enabled mode (CAP_CTRL<EV56_BW_EN> = 1; ICSR[17] = 1). Note that this will not work with the current version of CPU daughter cards. (The size field is not visible to software.)

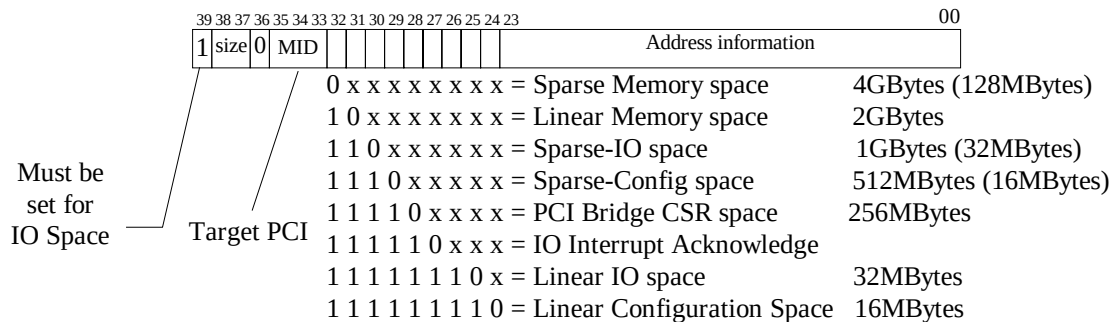


Figure 2-3 IO Space Address Translation with PCI bus bridges as targets, EV56 Byte/Word ECO enabled mode.¹¹

Dodge	MID	Durango
reserved	000	reserved
Memory	001	Memory
CPU0	010	CPU0
CPU1	011	CPU1
PCI0	100	PCI0
PCI1	101	PCI1
CPU2	110	PCI2
CPU3	111	PCI3

Table 2-2 Module ID assignment

¹⁰ Only byte writes are supported to this address space on an EV56 with byte/word to IO space enabled

¹¹ Address bit 36 must be 0 when a cached EV56 processor addresses linear configuration and IO space only. Otherwise it is a don't care bit.

Rawhide Addressing

A<39>	A<38:37>	A<36>	A<35:33>	A<32:29>	A<28:24>	
0	xx	x	xxx	xxxx	xxxxx	Cacheable Memory Address Space
1	xx	x	MID	0xxx	xxxxx	Sparse Memory
1	xx	x	MID	10xx	xxxxx	Dense Memory
1	xx	x	MID	110x	xxxxx	Sparse IO
1	xx	x	MID	1110	xxxxx	Sparse PCI Device Config Space
1	xx	x	MID	1111	0xxxx	MC-PCI Bus Bridge CSRs
1	xx	x	MID	1111	10xxx	Inerrupt Acknowledge CSRs

Table 2-3 Fixed Address for MC-PCI Bus Bridge CSRs when CAP_CTRL<EV56_BW_EN> = 0
(EV5 compatability mode)

READS:			
ADDR<39>	ADDR<38:37>	IN4_VALID<3:0>	Comment
1	SIZE<1:0>=0	QW_MASK<3:0>	No change, read the requested quadwords and return them
1	SIZE<1:0>=1	ADDR<3:2>, ADDR<1:0>=DC	Read the LW addressed by ADDR<39:2>
1	SIZE<1:0>=2	ADDR<3:1> ADDR<0>=DC	Read the word addressed by ADDR<39:1>
1	SIZE<1:0>=3	ADDR<3:0>	Read the byte addressed by ADDR<39:0>

Table 2-4 Used when decoding reads of linear address space

WRITES:			
ADDR<39>	ADDR<38:37>	IN4_VALID<3:0>	Comment
1	SIZE<1:0>=0	LW_MASK<7:0>	No change, write the modified longwords
1	SIZE<1:0>=1	LW_MASK<7:0>	No change, write the modified longwords
1	SIZE<1:0>=2	ADDR<3:1> ADDR<0>=DC	Write the word addressed by ADDR<39:1>
1	SIZE<1:0>=3	ADDR<3:0>	Write the byte addressed by ADDR<39:0>

Table 2-5 Used by hardware when decoding writes of linear address space

Rawhide Addressing

CPU Addr <39:0>		Description	(Max.) Size
00.0000.0000	- 7F.FFFF.FFFF	Cacheable address space	512 GBytes
80.0000.0000	- E9.FBFF.FFFF	reserved	
E9.FC00.0000	- E9.FDFF.FFFF	PCI 0 Linear IO Space	32MBytes
E9.FE00.0000	- E9.FEFF.FFFF	PCI 0 Linear Configuration Space	16MBytes
E9.FF00.0000	- EB.FDFF.FFFF	reserved	
EB.FC00.0000	- EB.FDFF.FFFF	PCI1 Linear IO Space	32MBytes
EB.FE00.0000	- EB.FEFF.FFFF	PCI1 Linear Configuration Space	16MBytes
EB.FF00.0000	- F3.FFFF.FFFF	reserved	
F4.0000.0000	- F4.0FFF.FFFF	(unused)	8 GBytes
F4.1xxx.xxxx		CPU0 IP INTR	
F4.7xxx.xxxx		CPU0 Node HALT	
F4.Fxxx.xxxx		CPU0 IO INTR	
F5.0xxx.xxxx		CPU0 Interval Timer Interrupt ACK ¹²	
F5.1xxx.xxxx		CPU0 IP Interrupt ACK ¹²	
F5.3xxx.xxxx		CPU0 MCHK_Acknowledge ¹²	
F5.4xxx.xxxx		CPU0 DTAG_ENABLE	
F5.5xxx.xxxx		CPU0 DTAG_ENABLE	
F5.6000.0000	- F5.6FFF.FFFF	CPU0 CC_CONFIG	
F5.7xxx.xxxx		CPU0 Node HALT Interrupt ACK ¹²	
F5.8000.0000	- F5.FFFF.FFFF	(unused)	
F6.0000.0000	- F6.0FFF.FFFF	(unused)	
F6.1xxx.xxxx		CPU1 IP INTR	
F6.7xxx.xxxx		CPU1 Node HALT	
F6.Fxxx.xxxx		CPU1 IO INTR	
F7.0xxx.xxxx		CPU1 Interval Timer Interrupt ACK ¹²	
F7.1xxx.xxxx		CPU1 IP Interrupt ACK ¹²	
F7.3xxx.xxxx		CPU1 MCHK_Acknowledge ¹²	
F7.4xxx.xxxx		CPU1 DTAG_ENABLE	
F7.5xxx.xxxx		CPU1 DTAG_ENABLE	
F7.6000.0000	- F7.6FFF.FFFF	CPU1 CC_CONFIG	
F7.7xxx.xxxx		CPU1 Node HALT Interrupt ACK ¹²	
F7.8000.0000	- F7.FFFF.FFFF	(unused)	8 GBytes
F8.0000.0000	- F8.FFFF.FFFF	PCI0 Sparse Memory Space	128 MB, PCI
F9.0000.0000	- F9.7FFF.FFFF	PCI0 Dense/Linear Memory Space	2GB, PCI
F9.8000.0000	- F9.BFFF.FFFF	PCI0 Sparse IO Space	32 MB, PCI
F9.C000.0000	- F9.DFFF.FFFF	PCI0 Sparse Config Space	16 MB PCI
F9.E000.0000	- F9.EFFF.FFFF	PCI0-Bridge Sparse CSR. Space	8 MBytes
F9.F000.0000	- F9.F000.3EC0	unused	
F9.F000.3F00	- F9.F000.3F40	PCI0-Bridge Interrupt ACK	
F9.F000.3F80	- F9.FBFF.FFFF	unused	
F9.FC00.0000	- F9.FDFF.FFFF	reserved PCI0 Linear IO Space alias	32MBytes
F9.FE00.0000	- F9.FEFF.FFFF	reserved PCI0 Linear Configuration Space alias	16MBytes
F9.FF00.0000	- F9.FFFF.FFFF	unused	

¹² On EV56 with IO space byte/word enabled, all writes to this address should be byte writes.

Rawhide Addressing

FA.0000.0000	- FA.FFFF.FFFF	PCI1 Sparse Memory Space	128 MB, PCI
FB.0000.0000	- FB.7FFF.FFFF	PCI1 Dense/Linear Memory Space	2GB, PCI
FB.8000.0000	- FB.BFFF.FFFF	PCI1 Sparse IO Space	32 MB, PCI
FB.C000.0000	- FB.DFFF.FFFF	PCI1 Sparse Config Space	16 MB PCI
FB.E000.0000	- FB.EFFF.FFFF	PCI1-Bridge Sparse CSR. Space	8 MBytes
FB.F000.0000	- FB.F000.3EC0	unused	8 MBytes
FB.F000.3F00	- FB.F000.3F40	PCI1-Bridge Interrupt ACK	
FB.F000.3F80	- FB.FBFF.FFFF	unused	
FB.FC00.0000	- FB.FDFF.FFFF	reserved PCI1 Linear IO Space alias	32MBytes
FB.FE00.0000	- FB.FEFFF.FFFF	reserved PCI1 Linear Configuration Space alias	16MBytes
FB.FF00.0000	- FB.FFFF.FFFF	unused	
FC.0000.0000	- FC.0FFF.FFFF	unused	
FC.1xxx.xxxx		CPU2 IP INTR	
FC.7xxx.xxxx		CPU2 Node HALT	
FC.Fxxx.xxxx		CPU2 IO INTR	
FD.0xxx.xxxx		CPU2 Interval Timer Interrupt ACK ¹²	
FD.1xxx.xxxx		CPU2 IP Interrupt ACK ¹²	
FD.3xxx.xxxx		CPU2 MCHK_Acknowledge ¹²	
FD.4xxx.xxxx		CPU2 DTAG_ENABLE	
FD.5xxx.xxxx		CPU2 DTAG_ENABLE	
FD.6000.0000	- FD.6FFF.FFFF	CPU2 CC_CONFIG	
FD.7xxx.xxxx		CPU2 Node HALT Interrupt ACK ¹²	
FD.8000.0000	- FD.FFFF.FFFF	(unused)	
FE.0000.0000	- FE.0FFF.FFFF	(unused)	
FE.1xxx.xxxx		CPU3 IP INTR	
FE.7xxx.xxxx		CPU3 Node HALT	
FE.Fxxx.xxxx		CPU3 IO INTR	
FF.0xxx.xxxx		CPU3 Interval Timer Interrupt ACK ¹²	
FF.1xxx.xxxx		CPU3 IP Interrupt ACK ¹²	
FF.3xxx.xxxx		CPU3 MCHK_Acknowledge ¹²	
FF.4xxx.xxxx		CPU3 DTAG_ENABLE	
FF.5xxx.xxxx		CPU3 DTAG_ENABLE	
FF.6000.0000	- FF.6FFF.FFFF	CPU3 CC_CONFIG	
FF.7xxx.xxxx		CPU3 Node HALT Interrupt ACK ¹²	
FF.8000.0000	- FF.FFFF.FFFF	(unused)	
FF.FFF0.0000	- FF.FFFF.FFFF	(unused; reserved by EV5)	1MB

Table 2-6 Detailed Mcbus Address Map - Dodge (4 CPUs, 2 PCIs) Configuration

Rawhide Addressing

CPU Addr <39:0>	Description	(Max.) Size
00.0000.0000 - 7F.FFFF.FFFF	Cacheable address space	512 GBytes
80.0000.0000 - E9.FBFF.FFFF	reserved	
E9.FC00.0000 - E9.FDFF.FFFF	PCI 0 Linear IO Space	32MBytes
E9.FE00.0000 - E9.FEFF.FFFF	PCI 0 Linear Configuration Space	16MBytes
E9.FF00.0000 - EB.FDFF.FFFF	reserved	
EB.FC00.0000 - EB.FDFF.FFFF	PCI1 Linear IO Space	32MBytes
EB.FE00.0000 - EB.FEFF.FFFF	PCI1 Linear Configuration Space	16MBytes
EB.FF00.0000 - ED.FDFF.FFFF	reserved	
ED.FC00.0000 - ED.FDFF.FFFF	PCI 2 Linear IO Space	32MBytes
ED.FE00.0000 - ED.FEFF.FFFF	PCI 2 Linear Configuration Space	16MBytes
ED.FF00.0000 - ED.FDFF.FFFF	reserved	
EF.FC00.0000 - EF.FDFF.FFFF	PCI 3 Linear IO Space	32MBytes
EF.FE00.0000 - EF.FEFF.FFFF	PCI 3 Linear Configuration Space	16MBytes
EF.FF00.0000 - F3.FFFF.FFFF	reserved	
F4.0000.0000 - F4.0FFF.FFFF	(unused)	8 GBytes
F4.1xxx.xxxx	CPU0 IP INTR	
F4.7xxx.xxxx	CPU0 Node HALT	
F4.Fxxx.xxxx	CPU0 IO INTR	
F5.0xxx.xxxx	CPU0 Interval Timer Interrupt ACK ¹³	
F5.1xxx.xxxx	CPU0 IP Interrupt ACK ¹²	
F5.3xxx.xxxx	CPU0 MCHK_Acknowledge ¹²	
F5.4xxx.xxxx	CPU0 DTAG_ENABLE	
F5.5xxx.xxxx	CPU0 DTAG_ENABLE	
F5.6000.0000 - F5.6FFF.FFFF	CPU0 CC_CONFIG	
F5.7xxx.xxxx	CPU0 Node HALT Interrupt ACK ¹²	
F5.8000.0000 - F5.FFFF.FFFF	(unused)	
F6.0000.0000 - F6.0FFF.FFFF	(unused)	
F6.1xxx.xxxx	CPU1 IP INTR	
F6.7xxx.xxxx	CPU1 Node HALT	
F6.Fxxx.xxxx	CPU1 IO INTR	
F7.0xxx.xxxx	CPU1 Interval Timer Interrupt ACK ¹²	
F7.1xxx.xxxx	CPU1 IP Interrupt ACK ¹²	
F7.3xxx.xxxx	CPU1 MCHK_Acknowledge ¹²	
F7.4xxx.xxxx	CPU1 DTAG_ENABLE	
F7.5xxx.xxxx	CPU1 DTAG_ENABLE	
F7.6000.0000 - F7.6FFF.FFFF	CPU1 CC_CONFIG	
F7.7xxx.xxxx	CPU1 Node HALT Interrupt ACK ¹²	
F7.8000.0000 - F7.FFFF.FFFF	(unused)	8 GBytes

¹³ On EV56 with IO space byte/word enabled, all writes to this address should be byte writes.

Rawhide Addressing

F8.0000.0000 - F8.FFFF.FFFF	PCI0 Sparse Memory Space	128 MB, PCI
F9.0000.0000 - F9.7FFF.FFFF	PCI0 Dense/Linear Memory Space	2GB, PCI
F9.8000.0000 - F9.BFFF.FFFF	PCI0 Sparse IO Space	32 MB, PCI
F9.C000.0000 - F9.DFFF.FFFF	PCI0 Sparse Config Space	16 MB PCI
F9.E000.0000 - F9.EFFF.FFFF	PCI0-Bridge Sparse CSR. Space	8 MBytes
F9.F000.0000 - F9.F000.3EC0	unused	
F9.F000.3F00 - F9.F000.3F40	PCI0-Bridge Interrupt ACK	
F9.F000.3F80 - F9.FBFF.FFFF	unused	
F9.FC00.0000 - F9.FDFF.FFFF	reserved PCI0 Linear IO Space alias	32MBytes
F9.FE00.0000 - F9.FEFFF.FFFF	reserved PCI0 Linear Configuration Space alias	16MBytes
F9.FF00.0000 - F9.FFFF.FFFF	unused	
FA.0000.0000 - FA.FFFF.FFFF	PCI1 Sparse Memory Space	128 MB, PCI
FB.0000.0000 - FB.7FFF.FFFF	PCI1 Dense/Linear Memory Space	2GB, PCI
FB.8000.0000 - FB.BFFF.FFFF	PCI1 Sparse IO Space	32 MB, PCI
FB.C000.0000 - FB.DFFF.FFFF	PCI1 Sparse Config Space	16 MB PCI
FB.E000.0000 - FB.EFFF.FFFF	PCI1-Bridge Sparse CSR. Space	8 MBytes
FB.F000.0000 - FB.F000.3EC0	unused	8 MBytes
FB.F000.3F00 - FB.F000.3F40	PCI1-Bridge Interrupt ACK	
FB.F000.3F80 - FB.FBFF.FFFF	unused	
FB.FC00.0000 - FB.FDFF.FFFF	reserved PCI1 Linear IO Space alias	32MBytes
FB.FE00.0000 - FB.FEFFF.FFFF	reserved PCI1 Linear Configuration Space alias	16MBytes
FB.FF00.0000 - FB.FFFF.FFFF	unused	
FC.0000.0000 - FC.FFFF.FFFF	PCI2 Sparse Memory Space	128 MB, PCI
FD.0000.0000 - FD.7FFF.FFFF	PCI2 Dense/Linear Memory Space	2GB, PCI
FD.8000.0000 - FD.BFFF.FFFF	PCI2 Sparse IO Space	32 MB, PCI
FD.C000.0000 - FD.DFFF.FFFF	PCI2 Sparse Config Space	16 MB PCI
FD.E000.0000 - FD.EFFF.FFFF	PCI2-Bridge Sparse CSR. Space	8 MBytes
FD.F000.0000 - FD.F000.3EC0	unused	32MBytes 16MBytes
FD.F000.3F00 - FD.F000.3F40	PCI2-Bridge Interrupt ACK	
FD.F000.3F80 - FD.FBFF.FFFF	unused	
FD.FC00.0000 - FD.FDFF.FFFF	reserved PCI2 Linear IO Space alias	
FD.FE00.0000 - FD.FEFFF.FFFF	reserved PCI2 Linear Configuration Space alias	
FD.FF00.0000 - FD.FFFF.FFFF	unused	
FE.0000.0000 - FE.FFFF.FFFF	PCI3 Sparse Memory Space	128 MB, PCI
FF.0000.0000 - FF.7FFF.FFFF	PCI3 Dense/Linear Memory Space	2GB, PCI
FF.8000.0000 - FF.BFFF.FFFF	PCI3 Sparse IO Space	32 MB, PCI
FF.C000.0000 - FF.DFFF.FFFF	PCI3 Sparse Config Space	16 MB PCI
FF.E000.0000 - FF.EFFF.FFFF	PCI3-Bridge Sparse CSR. Space	8 MBytes
FF.F000.0000 - FF.F000.3EC0	unused	8 MBytes
FF.F000.3F00 - FF.F000.3F40	PCI3-Bridge Interrupt ACK	
FF.F000.3F80 - FF.FFFF.FFFF	unused	
FF.FC00.0000 - FF.FDFF.FFFF	reserved Linear PCI3 IO Space alias	32MBytes
FF.FE00.0000 - FF.FEFFF.FFFF	reserved Linear PCI3 Configuration Space alias	16MBytes
FF.FF00.0000 - FF.FFEF.FFFF	unused	1MByte
FF.FFF0.0000 - FF.FFFF.FFFF	unused (reserved by EV5)	

Table 2-7 Detailed MCBus Address Map -PIO (2 CPUs, 4 PCIs) Configuration

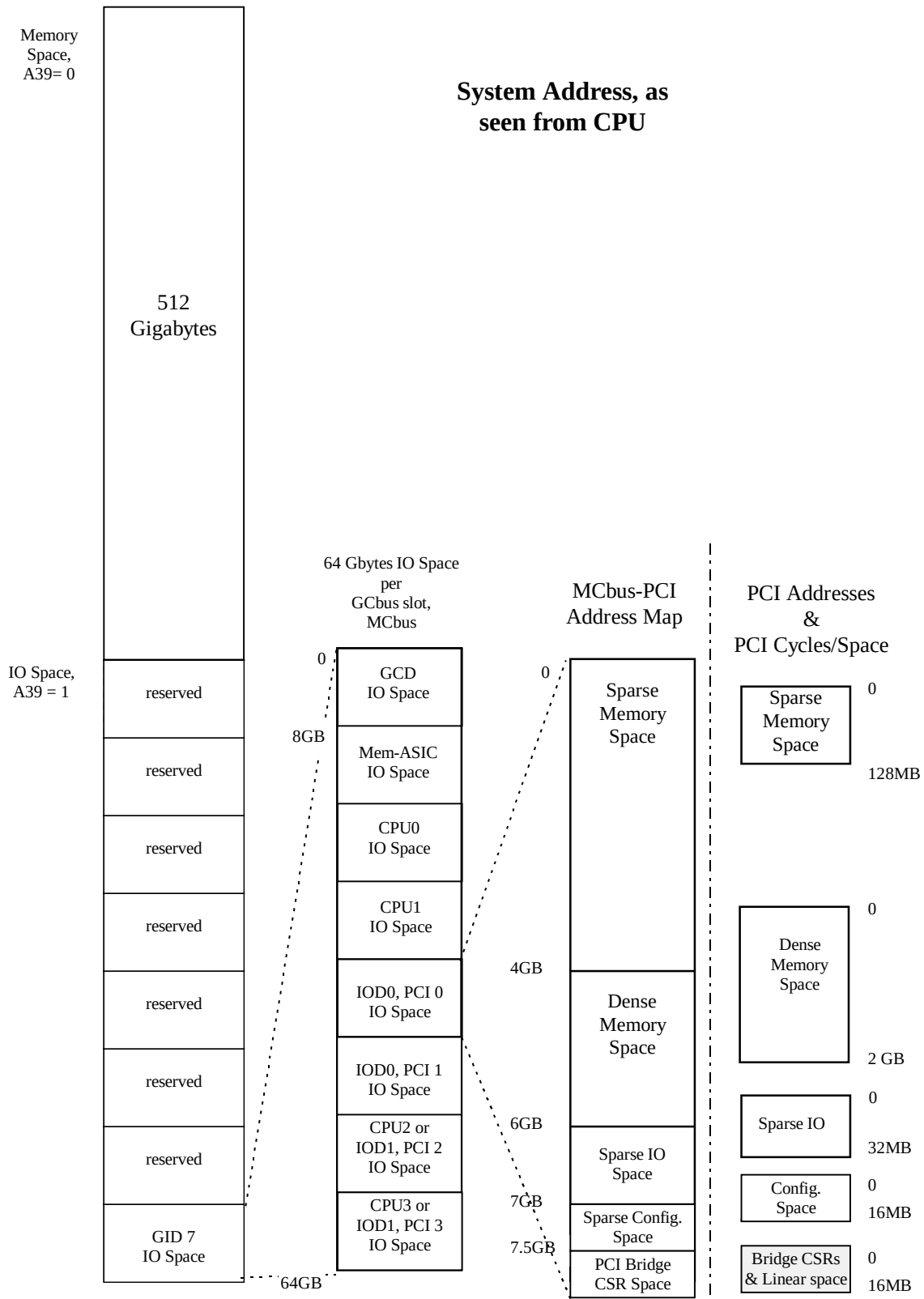


Figure 2-4 Address Space on an IIP

2.3 CPU View of IO Space

Alpha systems support two addressing schemes: Sparse space and Dense space. In Sparse space, byte- and word-size references are implemented by taking the byte-specific (IO) address and shifting it 5-bits to the left, encoding length information into the lower A4:A3 bits; A2:A0 are zeroed.

In Dense space, only longword and quadword accesses are implemented by the hardware interfaces. When an IO reference is made in dense space, multiple Longwords and/or quadwords may be accessed at a time. The requirement in this address space is that there are no side effects to reads, such that more data than requested may be prefetched. The lower physical (CPU) addresses in dense space are not shifted, but remain in their natural (longword or quadword) significant byte alignment.

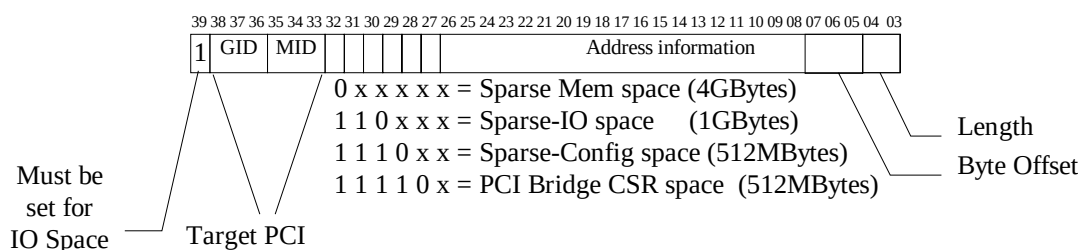


Figure 2-5 Sparse Address Decode

It is not possible (or necessary) on Rawhide to access all of PCI Memory or IO Space through a Sparse Address Window. Each PCI host-bridge provides High Address Extension (HAE) Registers that determine the naturally aligned address on the PCI of a reference to Dense or Sparse PCI Memory Space or Sparse PCI IO Space. It is expected that HAE registers are setup/initialized by POST¹⁴ software and never modified thereafter. Table 2-8 describes the use of the HAE register bits for each PCI Sparse Address Space region.

CPU Address	Sparse Addr. Region	PCI Address A<31:27>	PCI Address A<26:25>	PCI Address A<24>	PCI Address A<23:16>	PCI Address A<15:2>	PCI Address A<1:0>
F(8+y*2) 0000.0000- F(5+y*2) 00FF.FFFF	PCI bus y EISA/ISA compatibility Sparse Mem Space ¹⁵ (16 MBytes)	00000	00	0	CPU A<28:21>	CPU A<20:7>	00
F(8+y*2) 0100.0000- F(8+y*2) 7FFF.FFFF	PCI bus y Sparse Mem. Space (112 MBytes)	HAE_MEM <31:27>	CPU A<31:30>	CPU A<29>	CPU A<28:21>	CPU A<20:7>	00
F(8+y*2) 8000.0000- F(8+y*2) BFFF.FFFF	PCI bus y EISA/ISA compatibility Sparse IO Space (64 KBytes)	00000	00	0	00000000	CPU A<20:7>	A<6:3> 1111 00 else A<6:5>
F(8+y*2) 8000.0000- F(8+y*2) BFFF.FFFF	PCI bus y Sparse IO Space (32MBytes-64KBytes)	HAE_IO <31:27>	HAE_IO <26:25>	CPU A<29>	CPU A<28:21>	CPU A<20:7>	00

Table 2-8 HAE Register Use in Sparse Space Regions

¹⁴POST = Power-On Self-Test => PCI terminology for power-on console/firmware code.
15 y={0,1,2,3}

Rawhide Addressing

Size		Byte Offset	CPU Instruction Allowed	PCI Addr <2:0> see notes	PCI Byte Enable	Data in Register byte lanes
	CPU_Addr <4:3>	CPU_Addr <6:5>				63 31 0
Byte	00	00	LDL, STL	A<7>,00	1110	
		01		A<7>,00	1101	
		10		A<7>,00	1011	
		11		A<7>,00	0111	
Word	01	00	LDL, STL	A<7>,00	1100	
		01		A<7>,00	1001	
		10		A<7>,00	0011	
Tri-Byte	10	00	LDL, STL	A<7>,00	1000	
		01		A<7>,00	0001	
Long-Word	11	00	LDL, STL	A<7>,00	1000	
Quad-Word	11	11	LDQ, STQ	000	0000	
Note: A<7> = MC_address<7> Byte Enable set to 0 indicates that byte lane carries meaningful data. In PCI sparse memory space, PCI Address<1:0> is always zero. Missing entries (e.g. word size with CPU_address<6:5> = 11) have unspecified results						

Figure 2-6 Sparse Memory Space to PCI Address Space Translation

Rawhide Addressing

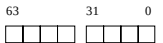
Size		Byte Offset	CPU Instruction Allowed	PCI Addr <2:0> see notes	PCI Byte Enable	Data in Register byte lanes 
	CPU_Addr <4:3>	CPU_Addr <6:5>				
Byte	00	00	LDL, STL	A<7>,00	1110	
		01		A<7>,01	1101	
		10		A<7>,10	1011	
		11		A<7>,11	0111	
Word	01	00	LDL, STL	A<7>,00	1100	
		01		A<7>,01	1001	
		10		A<7>,10	0011	
Tri-Byte	10	00	LDL, STL	A<7>,00	1000	
		01		A<7>,01	0001	
Long-Word	11	00	LDL, STL	A<7>,00	1000	
Quad-Word	11	11	LDQ, STQ	000	0000	
Note: A<7> = MC_address<7> Byte Enable set to 0 indicates that byte lane carries meaningful data. Missing entries (e.g. word size with CPU_address<6:5> = 11) have unspecified results						

Figure 2-7 Sparse IO Space to PCI Address Space Translation

2.3.1 PCI Memory Space

PCI Memory may be accessed through a Dense or a Sparse Address Space. Dense space provides access to a 2 Gigabyte region of PCI memory space. Accesses to Dense space only reference aligned, longword and quadword PCI memory addresses. On RAWHIDE, a Dense space read will generate a burst of two longword reads on the PCI; a Dense space write will generate a burst of up to 8 longword-masked writes on the PCI when a CPU is the source of the write. Bursts are trimmed to eliminate leading and trailing blanks.

When Dense space is the target of peer-peer DMA read the bridge will prefetch one entire cache line for each MC bus transaction. Dense peer-peer DMA writes are treated the same as Dense PIO writes except that they may include up to two double hexawords at a time.

Accesses to PCI Sparse Memory Space can be of any size (byte, word, longword, quadword), naturally-aligned to the size of the transfer. The address shifting algorithm that Alpha uses to generate Sparse-space (byte-level) accesses, the CPU's 4 Gigabyte-sized PCI Sparse Memory Space will only map 128 Megabytes of PCI Memory Space. An HAE Register on an IOD (MCbus-to-PCI bridge) determines which aligned 128 Megabytes on the PCI this window accesses.

Writes to PCI Memory Space, sparse or dense, are not merged by PCI bridge hardware. Writes are issued on the bus in the order the writes are issued across the MCBus (independent of source, CPU write or peer-to-peer DMA write).

2.3.1.1 Dense PCI Memory Space

Any CPU or PCI device (peer-DMA) can access 2 Gigabytes of PCI memory space through the Dense PCI Address Space window. References in Dense space must be longword (32-bits) or quadword (64-bit) aligned and size. A processor initiated Dense space read will generate two longword reads on the PCI (independent of LDL/LDQ from CPU). A processor initiated Dense space write will generate up to 8-longword-masked (burst) write on the PCI. A Dense space read initiated by a device on another PCI bus will generate an aligned double hexaword read on the PCI. A Dense space write initiated by a device on another PCI bus will generate anything from a single byte to aligned two double hexaword (64 byte) write using byte masks to specify valid data. Two double hexaword Peer-to-peer DMA reads may be posted at a time from each MCBus to PCI bus bridge.

IOD can accept a 64 byte burst of write data regardless of the source of the data (CPU or IOD).

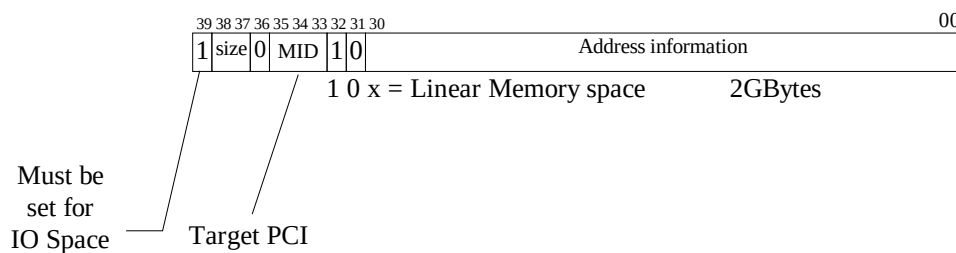
MCbus address bits A<30:4> are the lower (quadword significant) PCI address bits of a Dense PCI Memory space (cycle) reference. The aligned-2 Gigabyte PCI Memory space that is accessed is determined by a High Address Extension (HAE) Register in the IOD. Since Rawhide's PCI bus bridge does not support dual address cycles as a master, there will never be an upper 32-address bits.

2.3.1.2 Sparse PCI Memory Space

Sparse PCI Memory Space is a mechanism for Alpha processors to perform byte (8-bit) and word (16-bit) accesses on/to PCI devices, even though the architecture does not have byte and word load/store instructions. The implementation of Sparse Space on ALPHA is to shift the naturally, byte-aligned address up by 5 bits and to add length field information to the address bits A<4:3>. Due to the address shifting, the 4 Gigabyte Sparse PCI Memory Space on MCBus becomes a 128 Megabyte window of PCI Memory Space. The particular, aligned-128 Megabyte PCI Memory space that is accessed is determined by a High Address Extension (HAE) Register in the IOD.

2.3.1.3 Linear PCI Memory Space

When enabled, the Dense window will support linear (byte/word) addressing.

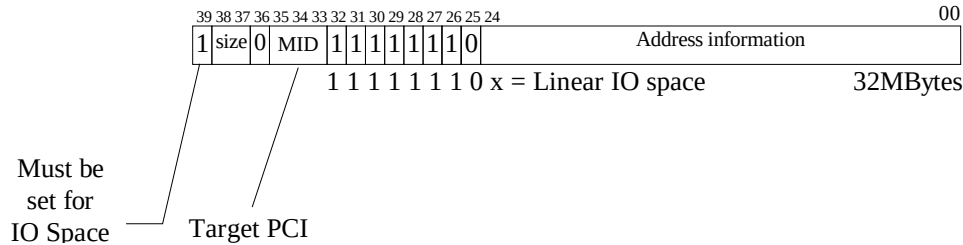


2.3.2 PCI IO Space

PCI IO may only be accessed through Alpha Sparse address space. Similar to PCI Sparse Memory Space, the 1 Gigabyte Sparse PCI IO Space will map 32 Megabytes of PCI IO Space. The particular, aligned, 32 Megabyte PCI IO Space that is accessed is determined by a High Address Extension (HAE) Register in the IOD.

2.3.2.1 Linear PCI IO Space

When enabled in the EV56 & CAP control CSRs, PCI IO space will be accessible through the linear IO space window.



2.3.3 PCI Configuration Space

PCI Configuration is mapped through ALPHA Sparse address mapping to enable byte-, word-, and longword-sized accesses. On RAWHIDE, the IOD (MCbus-to-PCI host bridge) takes a PCI Type-1 Formatted PCI Configuration cycle (shifted up by 5 with the appropriate length bits added in A4:A3 on the MCbus) and directly decodes it to the proper IDSEL assertion and PCI-bus-formatted Type0 Configuration cycle, or forwards a Type1 Configuration cycle to a PCI-to-PCI Bridge. If the PCI bus number in the (CPU/MCbus) Type1 Formatted address is 0, then the PCI configuration cycle is converted into a Type0 formatted PCI Configuration cycle (on the primary PCI bus connected to the IOD). If the PCI bus number in the (CPU) Type1 Formatted address is non-zero, then the IOD passes the configuration cycle onto the bus, expecting an appropriately configured PCI-to-PCI Bridge (PPB) interface to translate it to a (final) Type0 configuration cycle.

Per the PCI Local Bus spec, the significant address bits of a PCI Configuration cycle are 23:0. This translates to a 16MByte address space when mapped onto the PCI, and into a 512MByte address space with respect to the CPU when mapped into an ALPHA Sparse Address window/map.

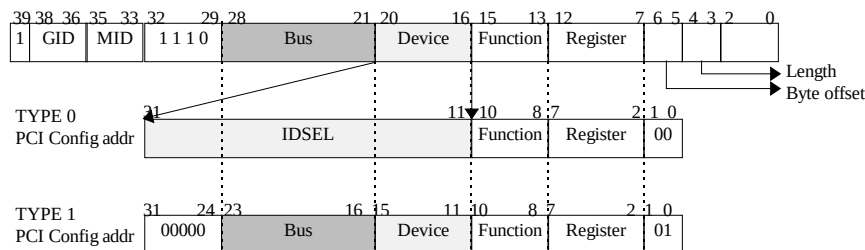


Figure 2-8 Sparse Config Space Address Decode

Device	Address Offset	PCI Address<31:11> - IDSEL	Device
PCI-EISA Bus Bridge or NCR810 SCSI	1.C001.0000	0000.0000.0000.0000.0001 0	1
PCI Option Slot 2	1.C002.0000	0000.0000.0000.0000.0010.0	2
PCI Option Slot 3	1.C003.0000	0000.0000.0000.0000.0100.0	3
PCI Option Slot 4	1.C004.0000	0000.0000.0000.0000.1000.0	4
PCI Option Slot 5	1.C005.0000	0000.0000.0000.0001 0000.0	5

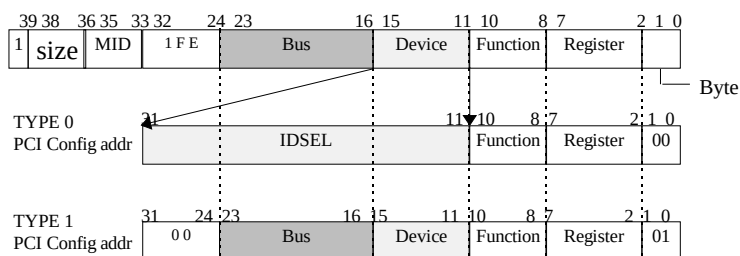
Table 2-9 Offset off GID-MID (F9, FB, FD, FF)

Note: Reference to non-existent addresses result in machine check.

Note: PCI host-bridge CSRs need not be manipulated to perform a proper PCI Configuration space access. All PCI Configuration space accesses are atomic, as are all CSR read or write in Sparse address space. Type 0 vs. Type 1 accesses are determined by the Bus Number field (MCbus A<28:21>) = 0.

2.3.3.1 Linear Configuration Space

The current pass of the IOD does not support linear (byte/word) addressing. If we add support for the EV56 platform it will be decoded as follows:



Device	Address Offset	PCI Address<31:11> - IDSEL	Device
PCI-EISA Bus Bridge or NCR810 SCSI	1.FE00.0800	0000.0000.0000.0000.0001 0	1
PCI Option Slot 2	1.FE00.1000	0000.0000.0000.0000.0010.0	2
PCI Option Slot 3	1.FE00.2000	0000.0000.0000.0000.0100.0	3
PCI Option Slot 4	1.FE00.3000	0000.0000.0000.0000 1000.0	4
PCI Option Slot 5	1.FE00.4000	0000.0000.0000.0001 0000.0	5

2.3.4 IOD-PCI Local CSR (Sparse) Address Space

IOD, PCI-bridge registers accessible from MCBus IO space are mapped to this region of an ALPHA Sparse address space. CSRs in this space are typically setup at console-(system-) initialization, or at OS bootstrap. Error-informational CSRs (that are accessed during normal system run-time) are also accessed through this address space, but infrequently. Refer to Chapter 4 for further IOD register details.

2.3.5 PCI Address Space Conventions

The (PIO) windows from system address space into PCI address space and (DMA) windows from PCI address space into System address space are programmable. By convention, OpenVMS, UNIX, and WNT operating systems will allocate space on each PCI as follows:

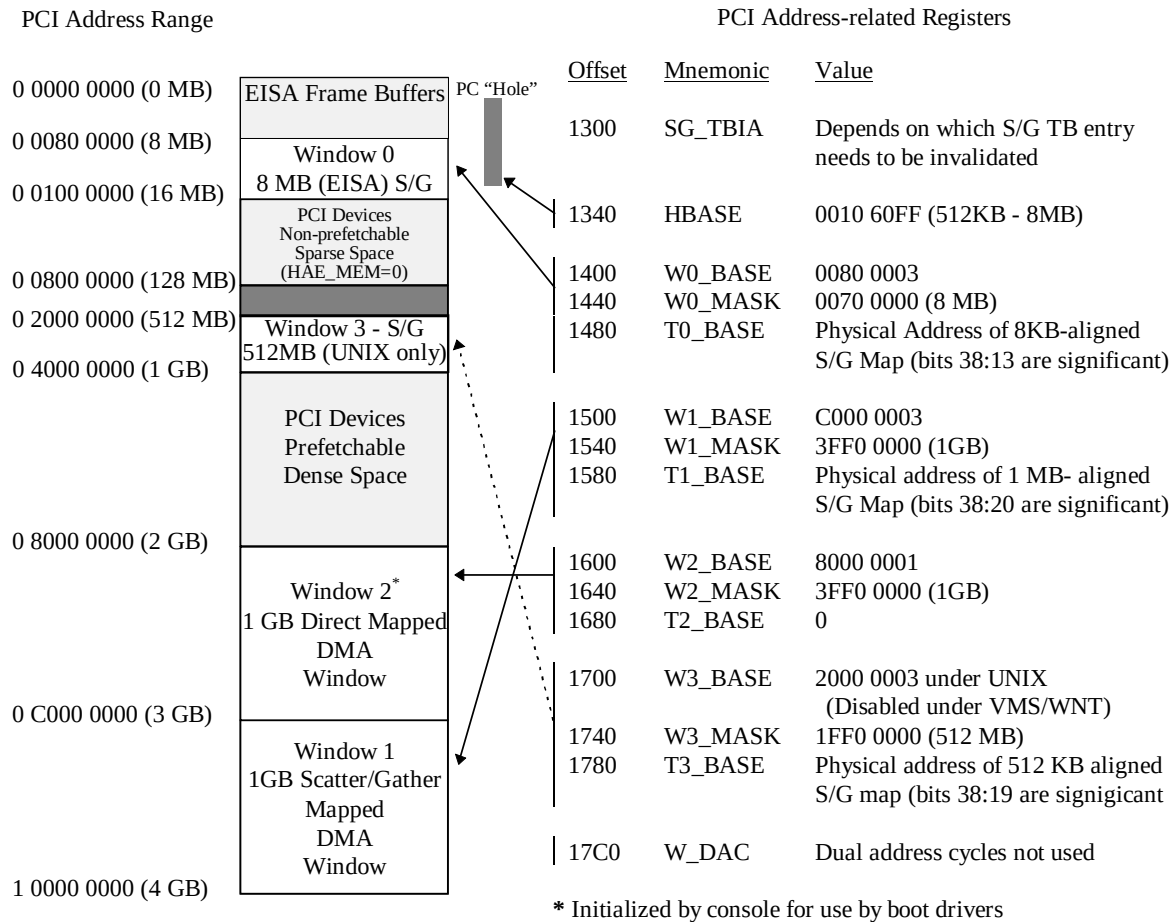


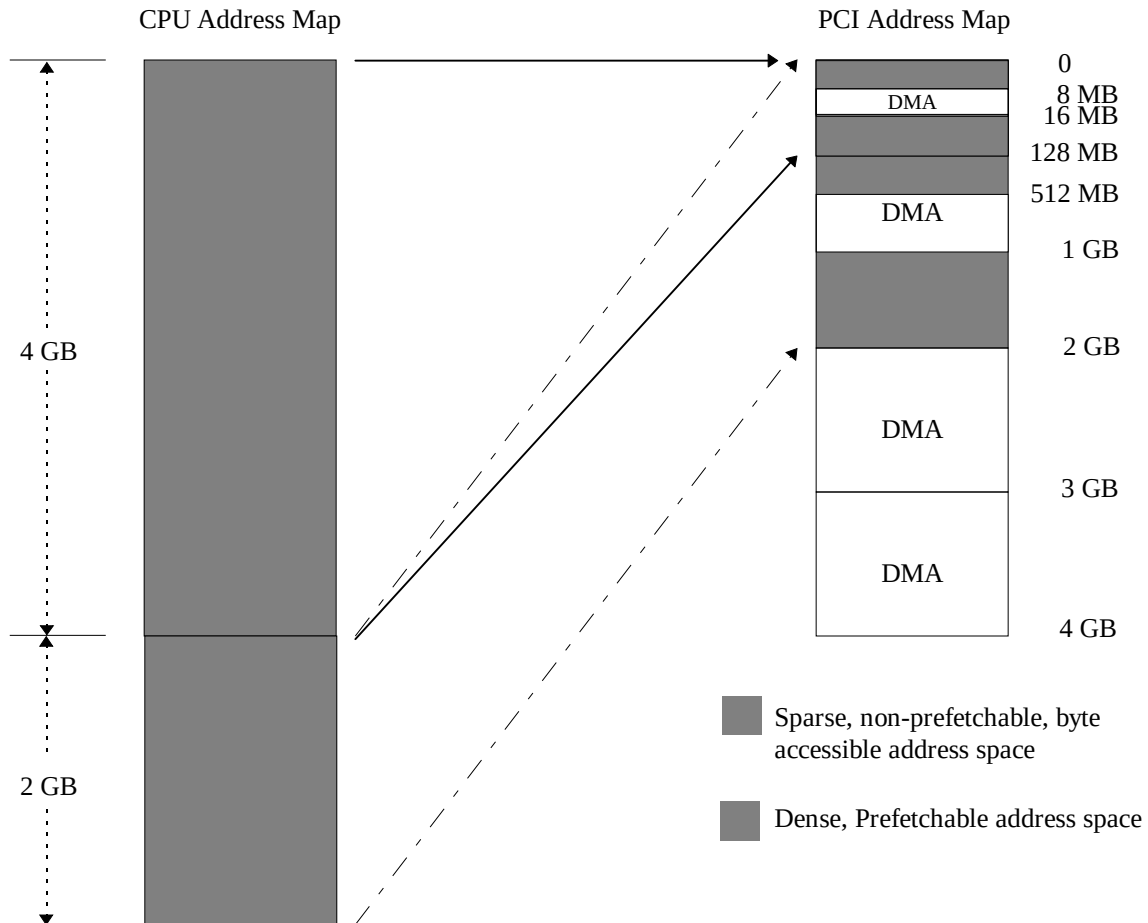
Figure 2-9 PCI Memory Space - DMA Window layout¹⁶

0-1 Mb	EISA/ISA Legacy Devices (e.g., VGA frame buffer)	Sparse
1-4 Mb	EISA/PCI configurable space	Sparse
4-8 Mb	EISA configurable only	Sparse
8-16 Mb	ISA 8 Mb S/G DMA Window (W0)	DMA
16-32 Mb	EISA/PCI 16 Mb S/G DMA Window (W1)	DMA
32-128 Mb	EISA/PCI configurable space	Sparse
128 Mb - 1 Gb	EISA/PCI configurable space	Dense
1 - 4 Gb	Unused	

Figure 2-10 PCI Address Space - for WNT¹⁷

¹⁶ Figure provided by Walt Arbo, OpenVMS group. See Chapter 4 for details on the registers cited in this figure.

¹⁷ Table provided by Eric Rehm, WNT group.

Figure 2-11 CPU-to-PCI Memory Address Map¹⁸

2.3.6 IO Interrupt Vectoring, Determination

(PAL/HAL programmers only)

IO Interrupts (from any IOD to any CPU) is an atomic sequence of four MCBus transactions. The receiving CPU of an IO interrupt is determined by an (interrupt target) register in the IOD on which the IO interrupt is being driven. The atomic operation performs a read-modify-write of a memory location, filling in the vector of the interrupting device. Once this operation is complete, a write to the IO INTERRUPT Space of the CPU-to-be-interrupted is performed. The address of the IO INTERRUPT Space of a CPU is:

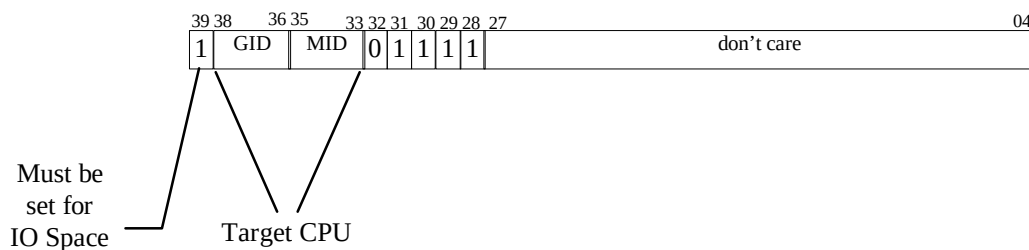


Figure 2-12 CPU IO space address: write causes an IRQ1 interrupt

¹⁸ Figure provided by Walt Arbo, OpenVMS group. This applies to VMS and Digital UNIX only.

2.3.7 IP Interrupt Address Space (Generation)

(Hardware designers only)

IP Interrupts on RAWHIDE are from one CPU to one-and-only-one CPU. IP Interrupts are generated by performing a write to a CPU's IO Space. The address of an IP interrupt sent from one CPU to a receiving CPU is as follows:

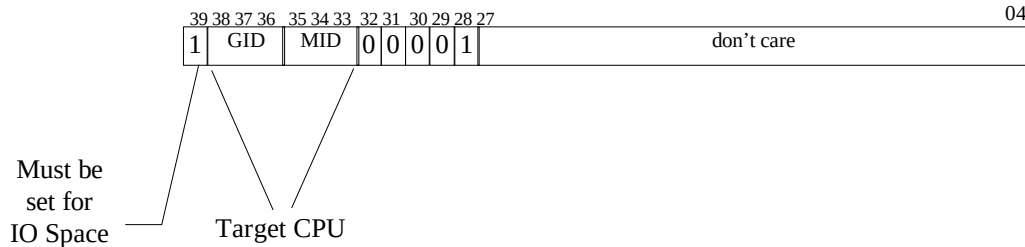


Figure 2-13 Address Decode for IP Interrupt

The data of the IO space write to a CPU's IP INTERRUPT Space is a don't care.

2.3.8 Node Halt Interrupt Address Space (Generation)

(PAL/HAL programmers only)

Node Halt Interrupts can be generated from one CPU to another CPU by the source CPU performing an IO Space write to the receiving/destination CPU's IO Space. The format of the IO Space write is:

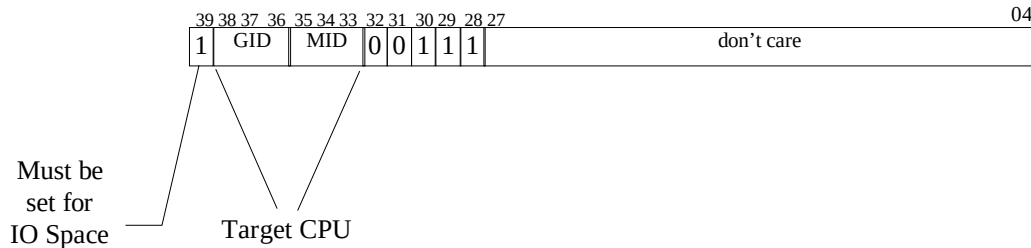


Figure 2-14 Address Decode for Node Halt Interrupt

The data of the IO Space write to a CPU's Node Halt Interrupt Space is a don't care.

2.4 (E)ISA IO Address Space

PCI Sparse I/O space is where EISA device IO CSRs reside. The (E)ISA devices have reserved the lower 64KB of this space. Hence all PCI devices should be relocated above this region. The three Rawhide (E)ISA slots are hardwired through the AEN* allocating 4KB per slot (per the EISA standard). The first slot is reserved for the EISA system board (i.e. X-BUS addressing, etc.). Table 2-10 Rawhide PCI and (E)ISA I/O map shows the PCI and (E)ISA I/O map for the Rawhide system.

CPU address	PCI addr	Range (KB)	Selection
F 9.8000.0000	0000.0000	0 - 4	EISA system board (X-bus, IO ports, etc.)
F 9.8002.0000	0000.1000	4 - 8	EISA slot 1
F 9.8004.0000	0000.2000	8 - 12	EISA slot 2
F 9.8006.0000	0000.3000	12 - 16	EISA slot 3
F 9.8008.0000	0000.4000	16 - 20	reserved
F 9.800A.0000	0000.5000	20 - 24	reserved
F 9.800C.0000	0000.6000	24 - 28	reserved
F 9.800E.0000	0000.7000	28 - 32	reserved
F 9.8010.0000	0000.8000	32 - 36	reserved
F 9.8012.0000 : F 9.801F.0000	0000.9000 : 0000.FFFF	36 - 64	PCI IO area - fixed
F 9.8012.0000 : F 9.801F.0000	0001.0000 : 01FF.FFFF	64KB to 32MB	PCI IO area - relocatable using the HAE_IO register

Table 2-10 Rawhide PCI and (E)ISA I/O map¹⁹

Warning: A quadword access to the PCI sparse I/O space will result in a 2 longword burst on the PCI. However, PCI devices may not support bursting in I/O space.

Historical note: The I/O address decoders on the expansion cards for the PC, XT and AT only looked at address bits <9:5>, ignoring bits <15:10>. The IO address range assigned for usage by expansion IO address decoders, <9:5>, bits <9:8> would be either set to 01 (0100h through 01FFh range), 10 (0200h through 02FFh range), or 11 (0300h through 03FFh range). When any address within the expansion IO address range is driven on the bus an IO expansion card may respond. Because of address aliasing, certain PC-AT IO space address ranges are not usable.

IO Address Range ²⁰	Useable or Unusable
x400h - x4FFh	useable
x500h - x7FFh	Unusable. Appears to be 0100h - 03FFh
x800h - x8FFh	useable
x900h - xBFFh	Unusable. Appears to be 0100h - 03FFh
xC00h - xCFFh	useable
xD00h - xFFFh	Unusable. Appears to be 0100h - 03FFh

Table 2-11 Usable and Unusable IO Address Ranges Above 03FFh

The EISA specification expands the number of IO windows available to the system designer. The EISA bus has a total of 1024 locations of spot specific PC-AT IO address space allocated to ISA expansion boards. Some address space is consumed by EISA IO devices that are integrated onto the PCI backplane. Table 2-12. defines the IO address assignment to support EISA.

¹⁹ In this table, the value of "x" for all (E)ISA slots must be 7 (GID= 7, MID = 4)

²⁰ Note: x = any hex digit

IO Address Range (HEX)	Reserved for	
0000 - 00FF	EISA/ISA System board IO devices (PCI-EISA Bridge registers)	System board (PCI backplane module)
0100 - 03FF	ISA Expansion cards	ISA cards
0400 - 04FF	EISA System board IO (PCI-EISA Bridge registers)	System board (PCI backplane module)
0500 - 07FF	alias of ISA range; do not use	
0800 - 08FF	EISA System board IO (PCI-EISA Bridge registers)	System board (PCI backplane module)
0900 - 0BFF	alias of ISA range; do not use	
0C00 - 0CFF	EISA System board IO (PCI-EISA Bridge registers)	System board (PCI backplane module)
0D00 - 0FFF	alias of ISA range; do not use	
1000 - 10FF	Slot 1 I/O	EISA Slot one
1100 - 13FF	alias of ISA range; do not use	
1400 - 14FF	Slot 1 I/O	EISA Slot one
1500 - 17FF	alias of ISA range; do not use	
1800 - 18FF	Slot 1 I/O	EISA Slot one
1900 - 1BFF	alias of ISA range; do not use	
1C00 - 1CFF	Slot 1 I/O	EISA Slot one
1D00 - 1FFF	alias of ISA range; do not use	
2000 - 20FF	Slot 2 I/O	EISA Slot two
2100 - 23FF	alias of ISA range; do not use	
2400 - 24FF	Slot 2 I/O	EISA Slot two
2500 - 27FF	alias of ISA range; do not use	
2800 - 28FF	Slot 2 I/O	EISA Slot two
2900 - 2BFF	alias of ISA range; do not use	
2C00 - 2CFF	Slot 2 I/O	EISA Slot two
2D00 - 2FFF	alias of ISA range; do not use	
Repeated for	every x000 - xFFF range	

Table 2-12 EISA IO Address Assignments

2.5 Sizing the MCBus Through Software

2.5.1 Configuration Information

Firmware and software may size the MCBus by collecting module presence and size information from an assortment of sources. System software has the option of using data passed through the RPB (Restart Parameter Block) as specified in the Alpha System Reference Manual or probing the hardware as specified in this section..

Memory presence, location, and size information is available by probing the I²C bus (see XYZ System Management specification for more details). During system power-up, firmware probes the I²C bus to obtain memory configuration data which is then converted into a series of memory descriptors and a memory bitmap for consumption by higher level software. These data structures are described in the Alpha SRM. It should be noted that these data structures do not provide explicit memory configuration information. For instance, from these data structures alone, higher level software could not distinguish between four 128MB array pairs and a single 512MB array pair. However, the SRM console will build a FRU table, pointed to be the HWRPB, which will provide both memory size and slot information.

Processor presence is determined in a different manner. During power-up, each CPU powers up in time-staggered fashion. Soon after power-up, serial ROM code on each CPU announces its presence by setting a bit in the Horse's firmware SCRATCH CSR (see Section 4.5.2 for more details, including the algorithm used to determine which one of possibly several SCRATCH CSRs is used by power-up code). System software can later obtain CPU presence information by reading the appropriate SCRATCH CSR's CPU presence bitmask.

MCbus-to-PCI bridge presence is obtained by reading the PCI_REV CSR in system I/O space. All MCBus devices which implement globally readable registers implement this. See section 4.2.1 for a register description.

2.5.2 Hardware Module Revision and Serial Number Information

There is no hardware mechanism for obtaining memory card revision information. The SRM console may provide a command which can be used to manually enter hardware revision and serial number information for memory cards (this is still TBD).

Processor hardware revision information is obtained by reading the WHOAMI CSR on any one of the Horse modules. Not only does WHOAMI provide CPU node ID, but it also provides cache size and hardware revision information. Section 4.2.2 describes the WHOAMI CSR in detail. All Rawhide CPUs implement a 256-byte I²C EPROM, which is used to store system and module serial number data. This EPROM can be accessed by probing the I²C bus. SRM console firmware commands are used to program the EPROM on the Manufacturing floor or during system configuration and/or installation. The format for the EPROM contents is currently TBD. Note that the CPU's EPROM, like all other I²C devices in the system, can only be accessed through the I/O subsystem's I²C controller on the Saddle.

Horse and Saddle module revisions are obtained by reading the low sixteen bits of the PCI_REV CSR on the associated Horse/Saddle pair. Module serial numbers are stored in and retrieved from I²C EPROMs on the Horse and Saddle modules, in the same way they are on Rawhide CPUs

2.6 EV56 & EV6 Addressing, Compatibility

EV56 and EV6 will have the ability to directly address 8-bit (bytes) and 16-bit (words) registers on IO buses without the use of Sparse Space. More directly stated, unlike EV4 and EV5, EV5-6 and EV6 will have 8-bit/byte and 16-bit/word load/store support.

At this time the MCBus-level address map and the CPU daughter cards do not support linear addressing. The IO bus bridge does provide linear address windows that may be used in the future by new CPU daughter card designs.

Rawhide Addressing

A<39>	A<38:37>	A<36>	A<35:33>	A<32:29>	A<28:24>	
0	xx	x	xxx	xxxx		Cacheable Memory Address Space
1	xx	x	MID	0xxx		Sparse Memory
1	size	x	MID	10xx		Dense/Linear Memory
1	xx	x	MID	110x		Sparse IO
1	xx	x	MID	1110		Sparse PCI Device Configuration Space
1	xx	x	MID	1111		MC-PCI Bus Bridge CSRs
1	11 = byte	1	MID	1111	10xxx	Inerrupt Acknowledge CSRs
1	size	0	MID	1111	1110x	Linear PCI IO Space (32MB)
1	size	0	MID	1111	11110	Linear PCI Config Space (16MB)

Table 2-13 Fixed Address for MC-PCI Bus Bridge CSRs when CAP_CTRL<EV56_BW_EN> = 1

3. PCI to Physical Memory Addressing

3.1 Introduction

Incoming PCI addresses (32-bit or 64-bit) are re-mapped into MC bus address space. Rawhide provides four programmable address windows and one fixed window that controls access of PCI peripherals to system memory and global address space. The mapping from PCI address to the physical address can be *direct mapped* (physical mapping with an address offset) or *Scatter/Gather* mapped (virtual mapping). The four programmable address windows are referred to as the PCI DMA target windows. The fixed window is referred to as the Monster Window.

3.2 DMA Transactions

The PCI address and command are captured in the Command and Address Path (CAP) gatearray. The address is compared with five address windows to determine if this PCI transaction should be accepted or ignored by the IOD²¹. Address windows are a requirement of the PCI specification and four of them are software configurable. All PCI commands that target memory or address space on a peer PCI bus will pass through the IOD.

The software configurable windows are almost identical to those implemented on ALCOR. The fifth window is a fixed direct mapped "Monster window" that provides direct access to all of Rawhide system address space. The Monster window supports very high performance Dual Address Cycle (DAC) capable options that need direct access to all of memory.

Each programmable window has three registers associated with it. These are:

- **PCI Base register** (WBASE): This defines the start of the Target Window. This register holds the SG bit that determines if the scatter/gather map will be used for this transaction.
- **PCI Mask register** (WMASK): This defines the size of the Window.
- **Translation Base register** (T_BASE): This holds a Base address used to relocate the PCI address in the CPU's memory space (for direct mapping). It is also used to hold the Scatter/Gather Map Base address for scatter/gather mapping.

In addition, there is an extra register associated with DMA Window 3 only. It is the **DAC BASE** register and is used for PCI 64-bit addressing (i.e., the Dual Address Cycle mode).

Each programmable window provides a mask corresponding to bits <31:20> of an incoming PCI address. The size of each window can be programmed in power of two increments from 1MB to 4GB. This is done by masking bits of the incoming PCI address using the PCI Mask register as shown Table 3-1²².

²¹ DOS compatibility window is identified by registers in the CAP chip. See the addressing chapter for more information.

²² Note that the Mask field pattern was chosen to speed-up timing critical hardware logic

The Target window Base register SG bit determines how the PCI address is translated: if SG is clear, then the address is directly mapped by concatenating the Translation Base Register to it. Otherwise the address is mapped through the scatter/gather table, allowing any 8KB of PCI address to map any 8KB of system address space.

WMASK<31:20>	Size of Window	Value of n
0000.0000.0000	1 Megabyte	20
0000.0000.0001	2 Megabytes	21
0000.0000.0011	4 Megabytes	22
0000.0000.0111	8 Megabytes	23
0000.0000.1111	16 Megabytes	24
0000.0001.1111	32 Megabytes	25
0000.0011.1111	64 Megabytes	26
0000.0111.1111	128 Megabytes	27
0000.1111.1111	256 Megabytes	28
0001.1111.1111	512 Megabyte	29
0011.1111.1111	1 Gigabytes	30
0111.1111.1111	2 Gigabytes	31
1111.1111.1111	4 Gigabytes	32
otherwise	Unpredictable	--

Table 3-1 DMA MASK Register

Only the incoming PCI address bits <31:n> are compared with <31:n> of the PCI Base register as shown in Figure 3-16. If **n** is equal to 32 then no comparison is performed. Windows are not allowed to overlap in PCI address space.

Based upon the value of the PCI Mask register, the unmasked bits of the incoming PCI address are compared with the corresponding bits of each window's PCI Base register. If the Base Registers and the incoming PCI address match, then the PCI address has hit the PCI DMA target window; otherwise it has missed the window. A window enable bit in each Window's PCI DMA Base register allows windows to be independently enabled (WENB = 1) or disabled (WENB = 0).

If a hit occurs in any of the four windows that are enabled, then the IOD will respond to the cycle by asserting the DEVSEL signal. The PCI target windows must be programmed such that their address ranges do not overlap (otherwise the hardware gets confused and results are undefined).

The Window base address must be on a naturally aligned address boundary depending on the size of the window²³. This restriction is not particularly onerous, since the address space of any PCI device can be located anywhere in the PCI's 4GB memory space; and this scheme is compatible with the PCI specification:

- A PCI device specifies the amount of memory space it requires via the Base registers in its configuration space. The Base Address registers are implemented such that the address space consumed by the device is a power of two in size and is naturally aligned on the size of the space consumed.

A PCI device need not use all the address range it consumes (i.e. the size of the PCI address window defined by the Base Address); nor need it respond to unused portions of the address space. The one exception to this is a PCI-bridge which requires two additional registers (the base and limit address registers). These registers accurately specify the address space which the bridge device will respond to²⁴; and are programmed by the POST code. The IOD, as a PCI Host-bridge device, does not have BASE and LIMIT registers²⁵, but does respond to all the addresses defined by the BASE register (i.e. all addresses within a window).

²³ for example, a 4MB window cannot start at address 1MB; it must start at addresses 4MB, 8MB, 12MB, etc.

²⁴ a bridge responds to all addresses in the range: (address<31:n> = base<31:n>)

²⁵ Host-bridges are permitted to diverge from the specification in some cases.

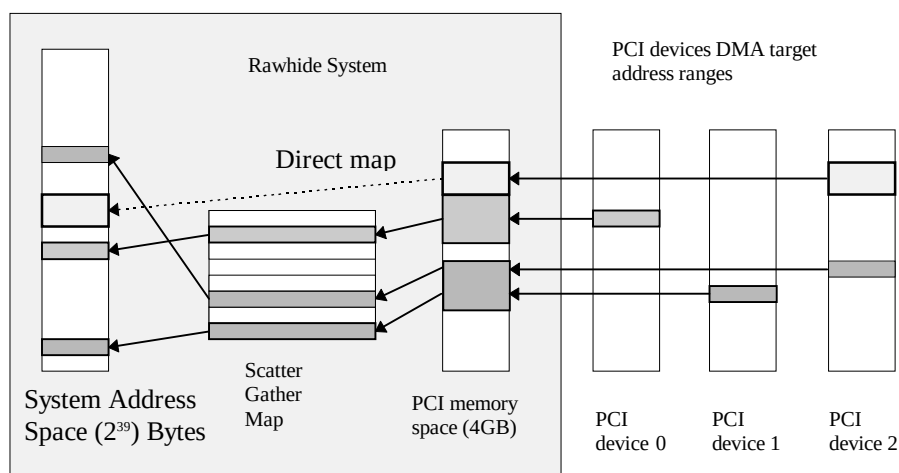


Figure 3-15 PCI DMA Addressing example

Figure 3-15 shows how the DMA address ranges of a number of PCI devices are accepted and translated in the IOD PCI-target window. Notice that the System (MC bus) address is translated differently as a function of the PCI DMA target window. PCI devices are allowed to use multiple DMA target windows (e.g. device 2). The example also shows that the IOD window can be larger than the range of addresses generated by a specific device (see device 0). Device 1 and device 2 use address ranges which are accepted by one IOD window. Each IOD window determines whether direct- or scatter/gather-mapping is used to access physical memory (see S/G-bit later)

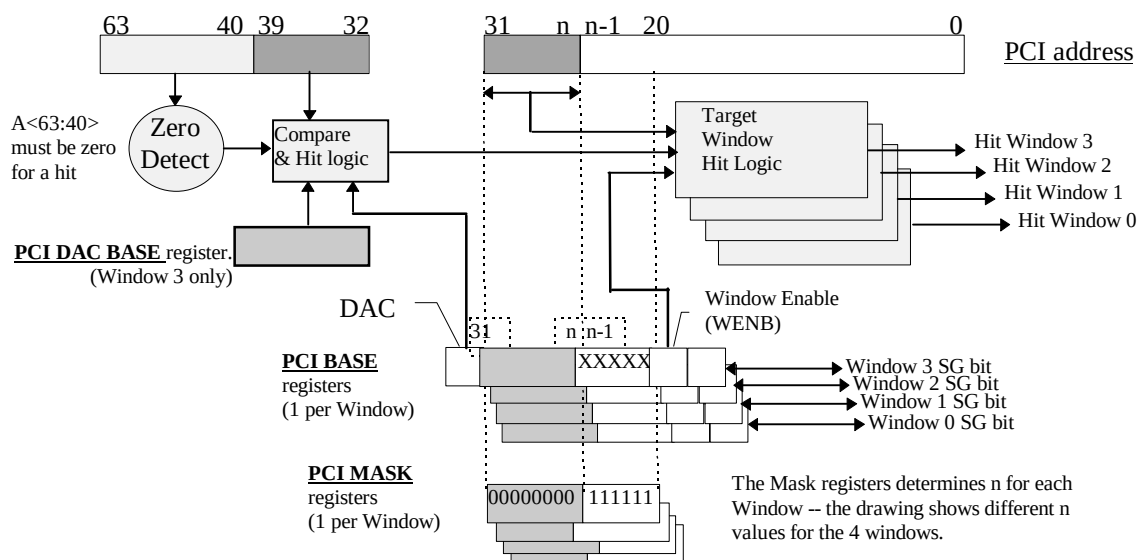


Figure 3-16 PCI Target Window Compare

Figure 3-16 depicts the PCI window logic. The comparison logic associated with PCI address bits <63:32> is only used for the DAC (Dual-Address cycle - only issued if <63:32> are non-zero for a 64-bit PCI address.) mode; and only if enabled by a bit in the PCI BASE register for Window 3. This logic is only applicable to Window 3; the remaining Windows only recognize 32-bit PCI addresses (i.e. SAC, Single Address cycle—all 32-bit addresses. A PCI device must use SAC if <63:32> = 0 of a 64-bit address cycles). For a hit to occur in a DAC address, address bits <63:40> must be zero; bits <39:32> must match the PCI DAC BASE register; and the low-order address bits <31:20> must also hit (see below). This scheme allows a naturally aligned, 1MB-4GB PCI window to be placed anywhere in the first 1TB of a 64-bit PCI address.

When an address match occurs with a PCI Target Window, the IOD ASIC translates the 32-bit PCI address to a memory address <39:0>. The translated address is generated in one of two ways as determined by the SG (Scatter/Gather) bit of the Window's PCI BASE register.

Software is required to do an explicit write to the SG_TBIA register (see Chapter 4) to invalidate the TLB after the in-memory scatter gather map is changed.

3.2.1 Direct-mapped Addressing

If the SG bit is cleared, the DMA address is direct mapped, and the translated address is generated by concatenating bits from the matching window's TRANSLATED BASE Register with bits from the incoming PCI address. The bits involved in the concatenation are defined by the PCI MASK register as shown in table . Note that the unused bits of the Translated Base register as indicated in Table must be cleared (i.e. the hardware performs an OR for the concatenation).

Note that since the TRANSLATED BASE is simply concatenated to the PCI address, then the direct mapping is to a naturally-aligned memory region. For example, a 4MB direct-mapped window will map to any 4MB region in main memory which falls on a 4MB boundary (for instance, it is not possible to map a 4MB region to the main memory region 1MB-5MB).

PCI_MASK <31:20>	Size of Window	Translated Address<39:2> Page address <39:13>; LW address <12:2>
0000.0000.0000	1MB	
0000.0000.0001	2MB	
0000.0000.0011	4MB	
0000.0000.0111	8MB	
0000.0000.1111	16MB	
0000.0001.1111	32MB	
0000.0011.1111	64MB	
0000.0111.1111	128MB	
0000.1111.1111	256MB	
0001.1111.1111	512MB	
0011.1111.1111	1GB	
0111.1111.1111	2GB	
1111.1111.1111	4GB	
		
Note: unused bits of the Translation Base must be zero for correct operation. (Non-zero values have undefined results)		

Table 3-2 Direct-Mapped DMA (MC bus Target) address translation

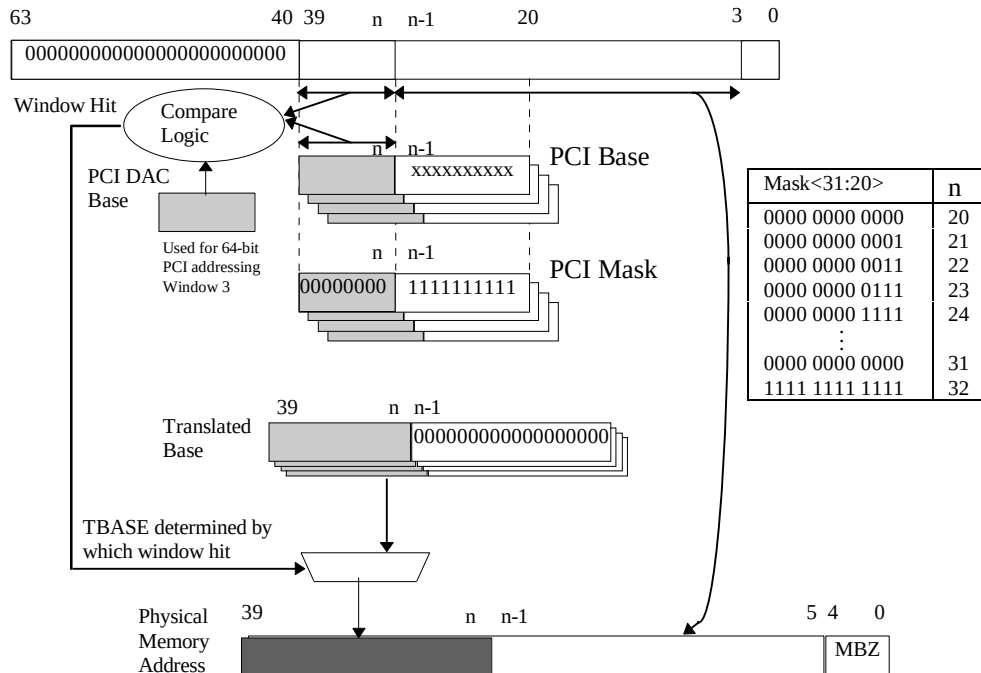


Figure 3-17 Direct Mapped Translation (ALCOR style figure)

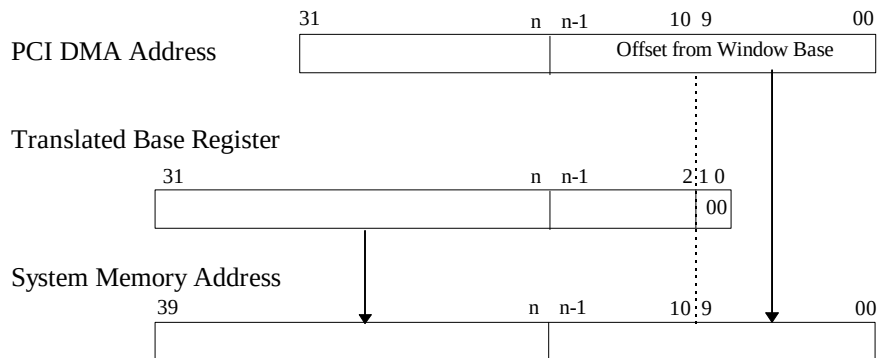


Figure 3-18 Direct Mapped DMA Address Translation (PCIA Style figure)

As an example, the values:

WMASK value: 00Fx xxxx (n=24)

WBASE value: 0100 0001

TBASE value: AB00 0000

Create a 16 Megabyte window in PCI bus address space that ranges from 0100.0000 through 01FF.FFFF. PCI DMA transactions in this range will be forwarded to main memory, translated to the CPU address range 0.AB00.0000 through 0.ABFF.FFFF.²⁶

3.2.2 Scatter Gather Addressing

If the SG bit of the PCI Base register is set, then the translated address is generated by a table lookup. This table is referred to as a Scatter/Gather Map. Figure 3-19 shows the scatter/gather addressing scheme—full details of this

²⁶ This figure and explanation included in response to review comments from those familiar with the PCIA design.

scheme are provided later in this section; but for now a quick description is provided: The incoming PCI address is compared to the PCI Window addresses for a hit. The *Translated Base Register*, associated with the PCI-window which hit, is used to specify the starting address of the Scatter/Gather map table in memory. Bits of the incoming PCI address are used as an offset from this starting address, to access the scatter/gather PTE. This PTE in conjunction with the remaining, least-significant PCI address bits, forms the required memory address.

Each Scatter Gather (SG) Map entry maps an 8KB page of PCI address space into an 8KB page of the processor's address space. This offers a number of advantages to software:

- Performance: ISA devices map to the lower 16MB of memory. NT currently copies data from here to user space. The scatter/gather map avoids this copy.
- User IO buffers cannot be counted on to be physically contiguous nor contained within a page. Without scatter/gather, software needs to manage the "scattered" nature of the user buffer by copies.

In the PC world, the term scatter/gather is not an address translation scheme but instead is used to signify a DMA transfer list. An element in this transfer list contains the DMA address and the number of data items to transfer. The DMA device fetches each item of the list until the list is empty. Many of the PCI devices (e.g. EISA bridge) support this form of scatter/gather.

Each SG entry is a quadword and has a valid bit in bit position 0. Address bit 13 is at bit position 1 of the map entry. Since the IOD chip set only implements a 40 bit address, then bits <63:28> of the SG map entry must be programmed to 0. Bits <27:1> of the SG map entry are used to generate the physical page address. This is appended to the bits <12:6>²⁷ of the incoming PCI address to generate the memory address.

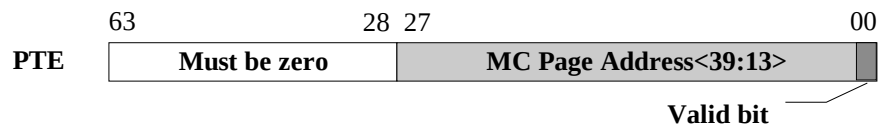


Figure 3-19 Scatter/Gather PTE format).

The size of the Scatter/Gather Map table is determined by the size of the PCI Target Window as defined by the PCI Mask register as shown in table . The number of entries is the Window size divided by the page size (8KB). The size of the table is simply the number of entries multiplied by 8B.

The Scatter/Gather map table address is obtained from the Translated Base register and the PCI address as shown in Figure 3-21-Scatter/Gather Map Translation.

²⁷ This is 12:6 because we do not support wrapped writes/reads.

PCI_MASK <31:20>	Size of Window	Number of PTEs	SG Map Table Size	Translated MC Address<38:3> ²⁸ used to fetch the PTE
0000.0000.0000	1MB	128	1 KB	38 10 19 13
0000.0000.0001	2MB	256	2 KB	38 11 20 13
0000.0000.0011	4MB	512	4 KB	38 12 21 13
0000.0000.0111	8MB	1K	8 KB	38 13 22 13
0000.0000.1111	16MB	2K	16 KB	38 14 23 13
0000.0001.1111	32MB	4K	32 KB	38 15 24 13
0000.0011.1111	64MB	8K	64 KB	38 16 25 13
0000.0111.1111	128MB	16K	128 KB	38 17 26 13
0000.1111.1111	256MB	32K	256 KB	38 18 27 13
0001.1111.1111	512MB	64K	512 KB	38 19 28 13
0011.1111.1111	1GB	128K	1 MB	38 20 29 13
0111.1111.1111	2GB	256K	2 MB	38 21 30 13
1111.1111.1111	4GB	512K	4 MB	38 22 31 13
				T_BASE PCI address
Note: unused bits of the Translation Base must be zero for correct operation. (Non-zero values have undefined results)				

Table 3-3 Scatter/gather mapped PCI Target address translation.

Scatter/Gather TLB

An eight entry Translation-Look-aside Buffer (TLB) is provided in the IOD for Scatter/gather Map entries. The TLB is a fully associative cache and holds the eight most recent Scatter/Gather map look-ups. Each of the eight TLB entries holds a PCI address for the tag, and four consecutive 8KB CPU page addresses as the TLB data, see Figure 3-20.

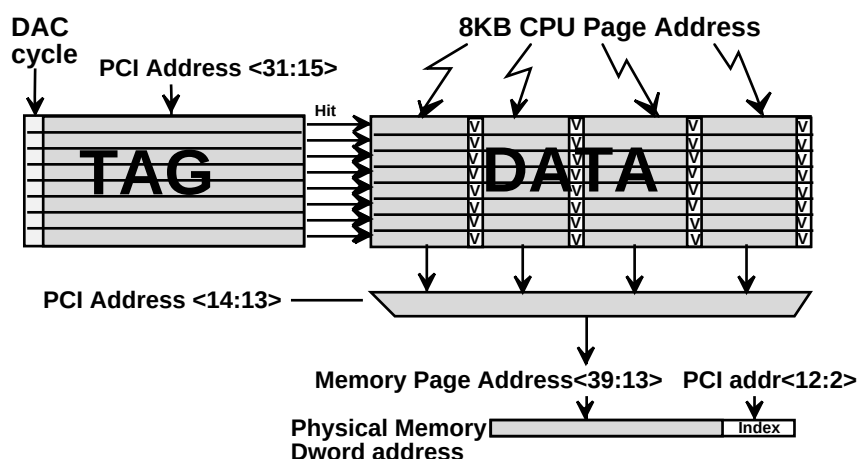


Figure 3-20 Scatter/gather Associative TLB

²⁸ A<39> must be zero because the S/G map is always in memory space.

Each time an incoming PCI address hits in a PCI Target Window which has scatter/gather enabled, bits <31:15> of the PCI address are compared with the 32KB PCI page address in the TLB tag. If a match is found, the required CPU page address is one of the four items provided by the data of the matching TLB entry. PCI address <14:13> select the correct 8KB CPU page from the four fetched.

With a TLB hit, the Scatter/gather Map table look-up in memory is avoided, resulting in enhanced performance. If no match is found in the TLB, the Scatter/gather Map lookup is performed and four PTE entries are fetched and written over an existing entry in the TLB. The TLB entry to be replaced is determined by a round robin algorithm. Coherency of the TLB is maintained by software writes to the SG_TBIA invalidate CSR.

The TAG portion contains a DAC flag to indicate that the PCI Tag address <31:15> corresponds to a 64-bit DAC address. Only one bit is required instead of the high-order PCI address bits <39:32> since only one window is assigned to a DAC cycle, and the Window-hit logic has already performed a comparison of the high-order bits against the PCI DAC BASE register.

Figure 3-21 shows the entire translation from PCI address to physical address on a window that implements scatter/gather. Both paths are indicated: the right-hand-side shows the path for a TLB hit, while the left hand side shows the path for a TLB miss. The Scatter/Gather TLB is shown in a slightly simplified, but functionally equivalent, form to avoid further cluttering the drawing.

The process for a Scatter/gather TLB hit is as follows:

- The Window compare logic determines if the PCI address has hit in one of the four windows, and the PCI BASE<SG> bit determines if the scatter/gather path should be taken. If Window 3 has DAC-mode enabled, and the PCI cycle is a DAC cycle, then a further comparison is made between the high-order PCI bits and the PCI DAC BASE register.
- PCI address <31:13> is sent to the TLB associative Tag together with the DAC-HIT indication. If the address and DAC bits match in the TLB then the corresponding CPU 8KB page-address is read out of the TLB. If this entry is valid then a TLB hit has occurred and this page-address is concatenated with PCI address <12:2> to form the physical memory address. If the data entry is invalid, or if the TAG compare failed, then a TLB miss occurs (see next section).

The process for a Scatter/gather TLB miss is as follows:

- The relevant bits of the PCI address (as determined by the Window Mask register) are concatenated with the relevant TRANSLATED BASE register bits to form the address used to access the Scatter/Gather Map entry from a table located in main memory.
- Bits <27:1> of the map entry (i.e. the PTE) are used to generate the physical page address, which is appended to the page offset to generate the physical memory address. The TLB is also updated at this point (round-robin algorithm) with the four PTE entries which correspond to the 32KB PCI page address which first missed the TLB. The Tag portion of the TLB is loaded with this PCI page address, and the DAC bit is set if this PCI cycle is a DAC cycle.
- If the requested PTE is marked invalid (bit 0 clear) then a TLB invalid entry exception is taken (see Hardware Exceptions and interrupts chapter).

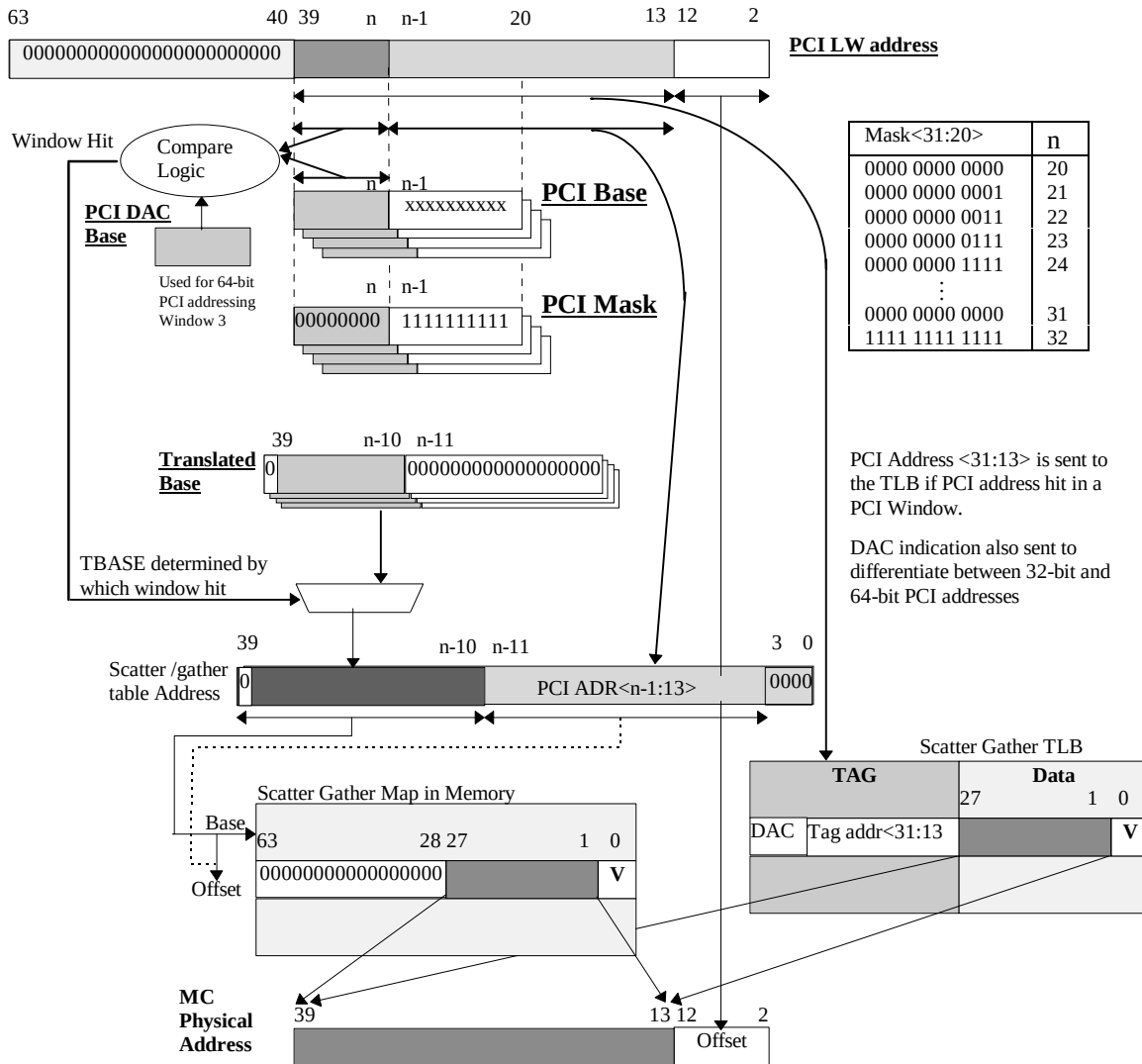


Figure 3-21-Scatter/Gather Map Translation.

Also note that an explicit write to the translation buffer invalidate register is required every time the in-memory scatter-gather map is updated to maintain coherency. This is different than Sable because the Rawhide (MC) bus is pended and keeping track of fill data as a bystander is too complex.

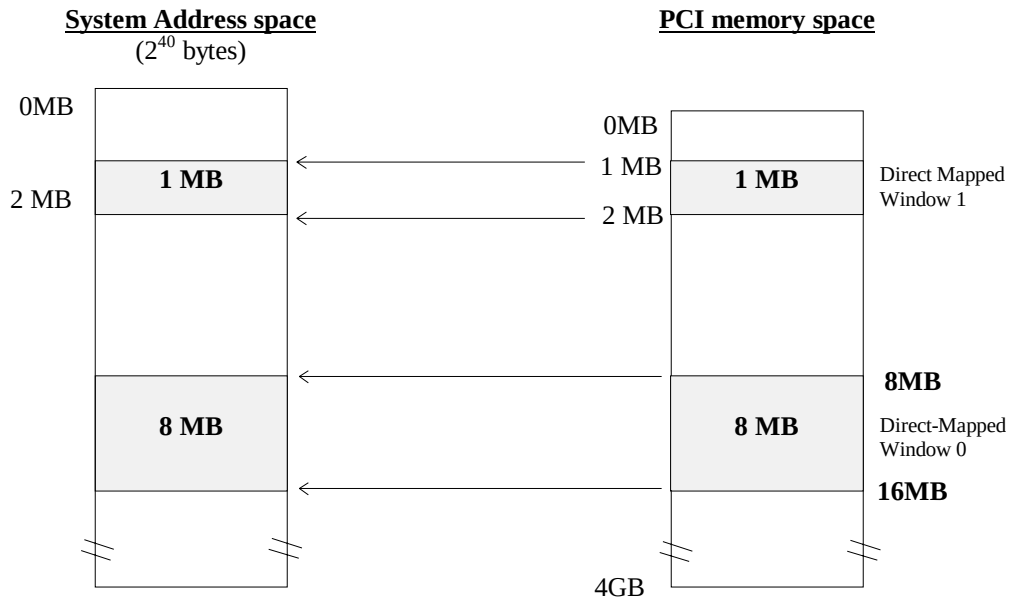
3.2.2.1 PCI-Window suggested use

Table 3-4 and Figure 3-22 show how the SRM Console Firmware will use the DMA windows. Table 3-5 and Figure 3-23 show the PCI-window assignment for UNIX and VMS (configured by firmware).

PCI window 0 was chosen for the 8MB-16MB “EISA” region since this window incorporates the PCI Compatibility logic. PCI window 3 was avoided since this window incorporates the DAC-cycle logic. Of the remaining two windows, PCI window 1 was chosen arbitrarily for the 1MB direct-mapped region, and PCI window 2 is not assigned.

PCI window	Assignment	Offset	Size	Comments
0	Direct	8 MB	8 MB	Console Only. May be bypassed for access to Sparse mapped CSRs as specified by using the PCI Hole Register
1	Direct	1 MB	1 MB	Console Only.
2				unused by the console
3				unused by the console

Table 3-4 PCI window configuration for the SRM console

Figure 3-22 PCI Window Configuration under the SRM console²⁹²⁹ This section updated based on what was observed in the debug lab.

PCI window	Assignment	Offset	Size	Comments
0	Scatter/Gather	8 MB	8 MB	May be bypassed (for access to Sparse mapped CSRs) as specified by using the PCI Hole Register.
1	Direct	2 GB	1 GB	Mapped to 0-1GB of main memory.
2	Scatter/Gather	3 GB	1 GB	General Purpose Scatter/Gather DMA
3	Scatter/Gather	512 MB	512 MB	Not used by Console or VMS. UNIX uses this for peer-peer DMA targets on a different PCI bus

Table 3-5 PCI window configuration for UNIX & VMS System Software

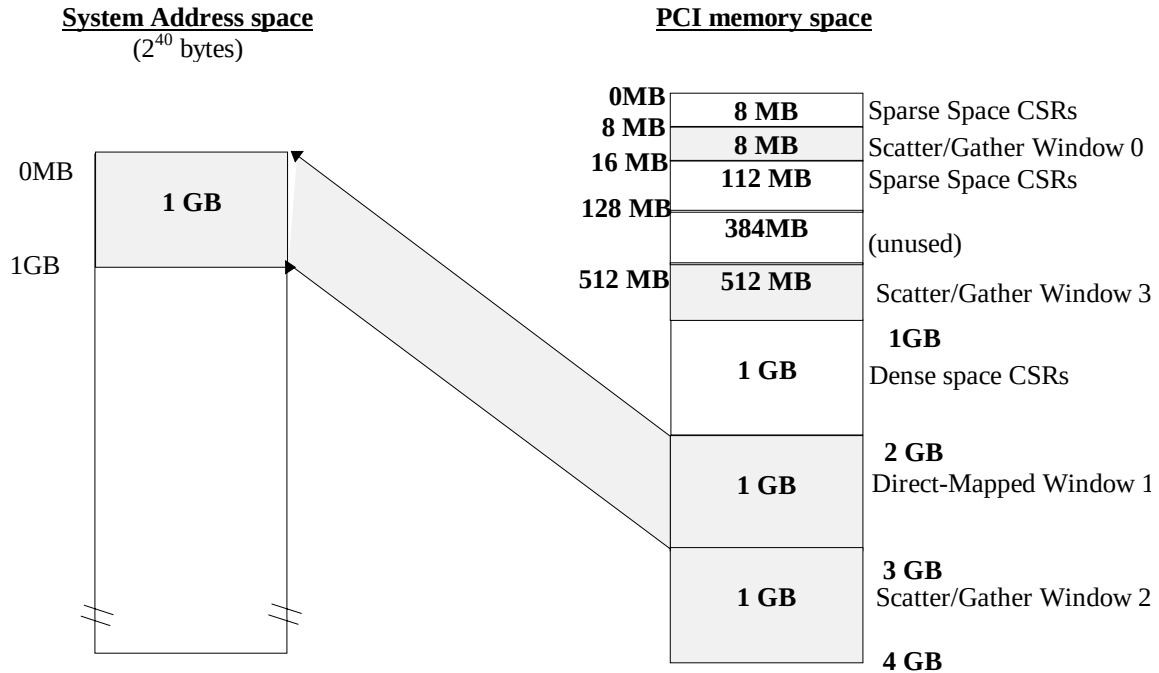
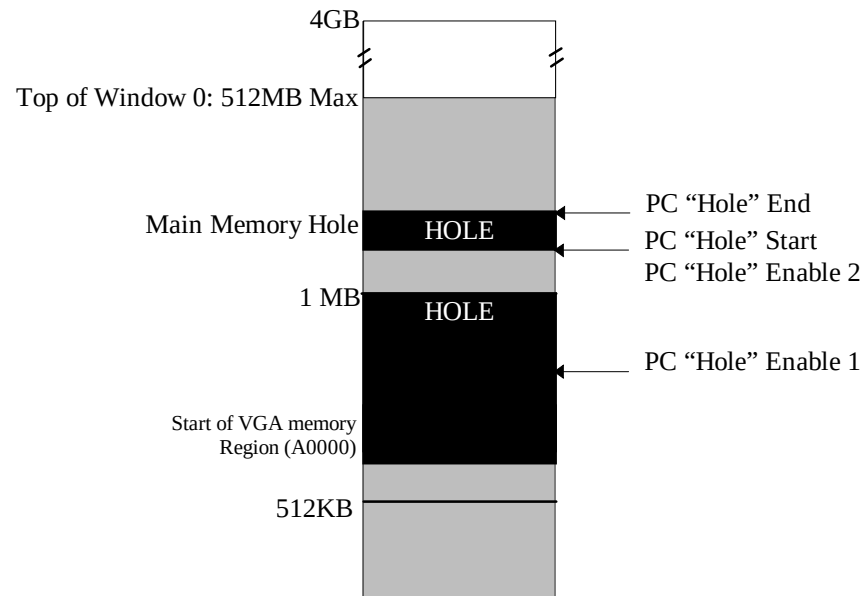


Figure 3-23 PCI Window Configuration UNIX & VMS System Software

3.2.2.2 PC Compatibility Addressing and Holes

The PC architecture allows certain (E)ISA devices to respond to hardwired memory addresses: an example is a VGA graphics devices which has its frame buffer located in memory address region A0000-BFFFF. Such devices, pepper memory space with “holes”, which are collectively known as PC compatibility holes.

To support legacy PC IO devices, these holes are supported in the same way as on Sable: through a PC “Hole” register.



Note: These holes are supported in DMA window 0 only.

Figure 3-24 PC "Hole" area

3.2.3 Monster Window Addressing

The Monster window provides direct access to all of Rawhide system address space to Dual Address Cycle (DAC) capable PCI options. When address bit 40 is set bits 63:41 are all 0, the target address is translated to system address with the top address bits MC_ADDRESS<63:40>=000000.

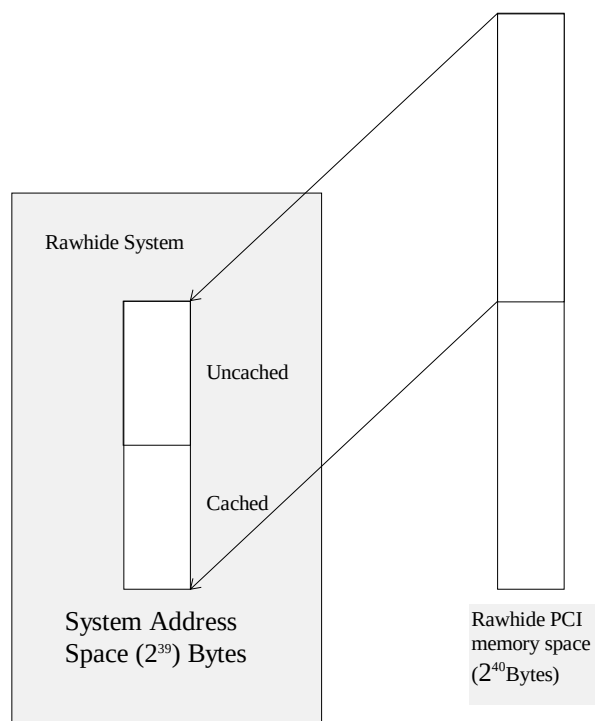


Figure 3-25 Monster Window

4. Control and Status Registers

4.1 Register Types

All Rawhide specific registers are on naturally aligned 64-byte addresses. All are mapped such that 4 bytes must be written or read at a time.

Category	Primary User
PCI Control Registers	software
Scatter/Gather Address Translation Registers	hardware/software
Error Reporting Registers	software, firmware/diagnostics
Hardware Configuration Registers	BIOS/firmware/diagnostics
Diagnostic registers	hardware debug, diagnostics

Table 4-1 Register Types

Most CSRs can be read/written by the software. Some registers are reserved for hardware debug and should not be touched by software. Registers reserved for hardware should only be manipulated in a well controlled environment (e.g. power-up).

4.1.1 Register Addressing

IO Device registers reside on devices attached to the PCI or subordinate buses. These registers are described in detail in device specifications and are only briefly mentioned herein. Private registers are locally accessible only on a CPU Daughter Card (CUD). Most of the registers on a CUD are internal to the EV5.

IO Device configuration registers are in Configuration space, as specified in the PCI standard. All devices with features that are expected to be set by BIOS or configuration software will have configuration space and configuration registers.

Registers on the MC-PCI bus bridge are located at a fixed address offset as specified in the addressing chapter. Access to configuration space is through a memory mapped address window.

Offset Address	Selected Region
0000.0000	General configuration, control, diagnostic, and error logging registers
0000.1000	PCI Error Status
0000.4000	MDPA Error Status & Diagnostic Control
0000.8000	MDPB Error Status & Diagnostic Control
0000.1300	PCI Scatter Gather Registers
0000.1800	Scatter/Gather Address Translation (TLB) Registers
FC00.0000 - FDFE.FFFF	When enabled: Linear PCI IO Space (32MB)
FE00.0000 - FFFF.FFFF	When enabled: Linear PCI Configuration Space (16MB)
1000.3F00 - 1000.3F40	Interrupt Acknowledge
000E.0000-000F.FFFF	Flash ROM space - Offset address into PCI Dense Memory Address Space (only on one PCI and is identical to Sable)

Table 4-2 Hardware Specific MC-PCI Bus Bridge Register Address Map

The address space is a hardware specific variant of sparse space encoding. For the CSRs, CPU Address bits <28:6> are used as the address and CPU address <5:0> must be zero. All the MC-PCI bridge registers are accessed with a LW granularity.³⁰

4.1.2 General Registers

These are Rawhide specific registers. This chapter is organized in roughly the same order as the CSR chapter of the ALCOR system programmers manual.

Register Name	Mnemonic	Offset	Access
PCI Revision	PCI_REV	000	RO
Who Am I	WHOAMI	040	RO
PCI Latency Timer	PCI_LAT	080	RW
MC-PCI Bridge Command	CAP_CTRL	100	RW
Host Address Extension Sparse memory	HAE_MEM	400	RW
Host Address Extension Sparse I/O space	HAE_IO	440	RW
Special Cycle/Interrupt Register	IACK_SC	480	RW
Host Address Extension Dense Memory	HAE_DENSE	4C0	RW

Table 4-3 General MC-PCI Bridge CSRs

Register Name	Mnemonic	Offset	Access
Interrupt Control	INT_CTL	500	RW
Interrupt Request	INT_REQ	540	RO
Interrupt Target Devices	INT_TARG	580	RW
Interrupt Target Address	INT_ADR	5C0	RW
Interrupt Target Address Extension	INT_ADR_EXT	600	RW
Interrupt Mask 0	INT_MASK0	640	RW
Interrupt Mask 1	INT_MASK1	680	RW
Interrupt Acknowledge 0	INT_ACK0	10003F00	WO
Interrupt Acknowledge 1	INT_ACK1	10003F40	WO

Table 4-4 Interrupt CSRs

Register Name	Mnemonic	Offset	Access
Performance Monitor Register	PERF_MON	300	RO
Performance Monitor Control Register	PERF_CONT	340	RW

Table 4-5 Performance Monitor Registers

³⁰ A simplified version of sparse-space is used for hardware convenience. It is identical to "Pseudo Sparse" address space used to access CSRs on ALCOR.

Control and Status Registers

Register Name	Mnemonic	Offset	Access
MC-PCI Bridge Diagnostic Control	CAP_DIAG	700	RW
Diagnostic General Purpose	SCRATCH	740	RW
Diagnostic General Purpose Alias	SCRATCH	780	RW
Top Of Memory	TOP_OF_MEM	7C0	RW
MC Error Information Register 0	MC_ERR0	800	RO
MC Error Information Register 1	MC_ERR1	840	RO
CAP Error Register	CAP_ERR	880	RW1C
PCI Error Status 1	PCI_ERR1	1040	RO
MDPA Status Register	MDPA_STAT	4000	RO
MDPA Syndrome Register	MDPA_SYN	4040	RO
Diagnostic Check MDPA	MDPA_DIAG	4080	RW
MDPB Status Register	MDPB_STAT	8000	RO
MDPB Syndrome Register	MDPB_SYN	8040	RO
Diagnostic Check MDPB	MDPB_DIAG	8080	RW

Table 4-6 Error and Diagnostic Registers

4.1.3 PCI Address-related Registers

Register Name	Mnemonic	Offset	Access
Scatter/Gather Translation Buffer Invalidate	SG_TBIA	1300	RW
PC "Hole" Compatibility Register	HBASE	1340	RW
Window Base0	W0_BASE	1400	RW
Window Mask0	W0_MASK	1440	RW
Translated Base0	T0_BASE	1480	RW
Window Base1	W1_BASE	1500	RW
Window Mask1	W1_MASK	1540	RW
Translated Base1	T1_BASE	1580	RW
Window Base2	W2_BASE	1600	RW
Window Mask2	W2_MASK	1640	RW
Translated Base2	T2_BASE	1680	RW
Window Base3	W3_BASE	1700	RW
Window Mask3	W3_MASK	1740	RW
Translated Base3	T3_BASE	1780	RW
Window DAC Base	W_DAC	17C0	RW

Table 4-7 PCI Address Scatter/Gather Registers

4.1.4 Scatter/Gather Address Translation Registers

Name	Mnemonic	Offset	Access
Translation Buffer Tag0	TB_TAG0	1800	RW
Translation Buffer Tag1	TB_TAG1	1840	
Translation Buffer Tag2	TB_TAG2	1880	
Translation Buffer Tag3	TB_TAG3	18C0	
Translation Buffer Tag4	TB_TAG4	1900	
Translation Buffer Tag5	TB_TAG5	1940	
Translation Buffer Tag6	TB_TAG6	1980	
Translation Buffer Tag7	TB_TAG7	19C0	
Translation Buffer 0 Page 0	TB0_Page0	2000	RW
Translation Buffer 0 Page 1	TB0_Page1	2040	
Translation Buffer 0 Page 2	TB0_Page2	2080	
Translation Buffer 0 Page 3	TB0_Page3	20C0	
Translation Buffer 1 Page 0	TB1_Page0	2100	RW
Translation Buffer 1 Page 1	TB1_Page1	2140	
Translation Buffer 1 Page 2	TB1_Page2	2180	
Translation Buffer 1 Page 3	TB1_Page3	21C0	
Translation Buffer 2 Page 0	TB2_Page0	2200	RW
Translation Buffer 2 Page 1	TB2_Page1	2240	
Translation Buffer 2 Page 2	TB2_Page2	2280	
Translation Buffer 2 Page 3	TB2_Page3	22C0	
Translation Buffer 3 Page 0	TB3_Page0	2300	RW
Translation Buffer 3 Page 1	TB3_Page1	2340	
Translation Buffer 3 Page 2	TB3_Page2	2380	
Translation Buffer 3 Page 3	TB3_Page3	23C0	
Translation Buffer 4 Page 0	TB4_Page0	2400	RW
Translation Buffer 4 Page 1	TB4_Page1	2440	
Translation Buffer 4 Page 2	TB4_Page2	2480	
Translation Buffer 4 Page 3	TB4_Page3	24C0	
Translation Buffer 5 Page 0	TB5_Page0	2500	RW
Translation Buffer 5 Page 1	TB5_Page1	2540	
Translation Buffer 5 Page 2	TB5_Page2	2580	
Translation Buffer 5 Page 3	TB5_Page3	25C0	
Translation Buffer 6 Page 0	TB6_Page0	2600	RW
Translation Buffer 6 Page 1	TB6_Page1	2640	
Translation Buffer 6 Page 2	TB6_Page2	2680	
Translation Buffer 6 Page 3	TB6_Page3	26C0	
Translation Buffer 7 Page 0	TB7_Page0	2700	RW
Translation Buffer 7 Page 1	TB7_Page1	2740	
Translation Buffer 7 Page 2	TB7_Page2	2780	
Translation Buffer 7 Page 3	TB7_Page3	27C0	

Table 4-8 Address Translation Registers

Register Name	Mnemonic	Sparse Memory Offset	PCI Memory Offset	Access

Table 4-9 EISA/ISA & XBUS Registers
(see Table at the end of this chapter)

4.2 PCI Device Configuration Registers

Every PCI device³¹ has a configuration space header as specified in the PCI standard. Registers in the configuration space header are used by configuration code and sometimes by device drivers for error symptom collection. PCI Devices typically also include device specific registers in memory and/or IO space.

31		16		15		0		
Device ID				Vendor ID				00h
Status				Command				04h
Class Code						Revision ID		08h
BIST		Header Type		Latency Timer		Cache Line Size		0Ch
Base Address Registers								10h
								14h
								18h
								1Ch
								20h
								24h
Cardbus CIS Pointer								28h
Subsystem ID				Subsystem Vendor ID				2Ch
Expansion ROM Base Address								30h
Reserved								34h
Reserved								38h
Max_Lat		Min_Gnt		Interrupt Pin		Interrupt Line		3Ch

Figure 4-1 Pre-defined PCI Configuration Space Header

³¹ The MC-PCI bus bridge does not respond to PCI Configuration space transactions.

4.3 EV5 Configuration Registers

All Processor configuration registers are internal to the EV5 or are specified in the Cached CPU daughter card specification. Detailed descriptions of these registers may be found in the EV5 specification.

Name	Mnemonic	Uncached CUD Firmware Initialized State	Cached CUD Firmware Initialized State
SCache Control Register	SC_CTL	00003002	0000F000
BCache Control Register	BC_CONTROL	18000050	04060055
BCache Configuration Register	BC_CONFIG	00011550	1FF14581 (1MB cache) 1FF12581 (2MB cache) 5FF13583 (EV56 4MB cache)
Icache Control & Status Register	ICSR	14C000000 (VMS) 15C000000 (WNT) 16C000000 (OSF)	14C000000 (VMS) 15C000000 (WNT) 16C000000 (OSF)
PAL Base Register	PAL_BASE	TBD	TBD
Miss Addr File - Mode Register	MAF_MODE	00000000	00000000
Dcache mode Register	DC_MODE	00000001	00000010

Table 4-10 EV5 Internal Processor Registers

For 4MByte EV56 Module:

BC_CONTROL	484000055
ST_CLK delay	(BC_CONTROL<34>)
Wave_pipeline = 4 cycles	(BC_CONTROL<31>)
Synch_Bcache= TRUE	(BC_CONTROL<26>)
ECC	(BC_CONTROL<6>)
CORR_FILL_DAT	(BC_CONTROL<4>)
EI_CMD_GRP2	(BC_CONTROL<2>)
BC_ENABLED2	(BC_CONTROL<0>)

BC_CONFIG	5FF136A3
ST_CLK W -> R extra cycle	(BC_CONFIG<30>)
WE_CTL = Synchronous	(BC_CONFIG<28:20>)
FILL_WE_OFFSET = 1	(BC_CONFIG<18:16>)
BC_RD_WR_SPC = 3	(BC_CONFIG<14:12>)
BC_WR_SPD = 6	(BC_CONFIG<11:8>)
BC_RD_SPEED = A	(BC_CONFIG<7:4>)
BCACHE_SIZE = 4 MBytes	(BC_CONFIG<3:0>)

For 4MByte EV56 Module @ 400mhz, sysclock ratio = 6:

BC_CONTROL	4C4000055
ST_CLK delay	(BC_CONTROL<34>)
Wave_pipeline = 4 cycles	(BC_CONTROL<31>)
ST_CLK W -> R extra cycle	(BC_CONTROL<30>)
Synch_Bcache= TRUE	(BC_CONTROL<26>)
ECC	(BC_CONTROL<6>)
CORR_FILL_DAT	(BC_CONTROL<4>)
EI_CMD_GRP2	(BC_CONTROL<2>)
BC_ENABLED2	(BC_CONTROL<0>)

BC_CONFIG	1FF136A3
WE_CTL = Synchronous	(BC_CONFIG<28:20>)
FILL_WE_OFFSET = 1	(BC_CONFIG<18:16>)
BC_RD_WR_SPC = 3	(BC_CONFIG<14:12>)
BC_WR_SPD = 6	(BC_CONFIG<11:8>)
BC_RD_SPEED = A	(BC_CONFIG<7:4>)
BCACHE_SIZE = 4 MBytes	(BC_CONFIG<3:0>)

4.3.1 Ponderosa Specific Registers

Ponderosa is the CPU module with a backup cache. There several Ponderosa specific write-only registers that are peculiar in that state bits appear as addresses. A write to a specific address sets or clears the bits in Ponderosa configuration and control registers.

4.3.1.1 CC_CONFIG_CSR

This register is used to configure the VCTY ASIC on the Cached_Cud. The register is write-only, and must be initiated by the 21164 coresident on its node (CC_CONFIG cannot be updated over the MC_Bus). This write only register has the property that address bits are used to set state in the register. The address bits contained in the "CC_CONFIG_VALUE" field, are written to CC_CONFIG<23:0> as follows:

A write to address (A<39> = 1, A<38:33> = CudID, A<32:28> = 10110, A<27:4> = "value") will load "value" into the register, CC_CONFIG<23:0>. CC_CONFIG Description:

Control and Status Registers

Address of 39	CC_CONFIG Reg 32:28	Bit Range 27:4	Value Description		
1	10110	12:0	Lockout Counter Threshold	Legal Values	# cycles
				0x0040	64
				0x0080	128
				0x0100	256
				0x0200	512
				0x0400	1024
				0x0800	2048
				0x1000	2096
1	10110	13	0: Lockout assertion Enabled 1: Lockout assertion disabled		
1	10110	14	0: Lockout reaction Enabled 1: Lockout reaction Disabled		
1	10110	15	0: Disable MB CACK by gate array 1: Enable MB CACK by gate array		
1	10110	16	0: Shared & Dirty mutually exclusive 1: Shared & Dirty not mutually exclusive (GCD support hook)		
1	10110	23:17	reserved		

Control and Status Registers

Name	Type	Extent	Initial State	Firmware Init'd State	Description																
Lockout_counter_threshold	WO	<12:0>	0x0100	0x0100	The number of cycles an outgoing MC_Bus request must be pending before the Cached CUD asserts MC_LOCKOUT_L <table><tr><th>Legal Values</th><th># cycles</th></tr><tr><td>0x0040</td><td>64</td></tr><tr><td>0x0080</td><td>128</td></tr><tr><td>0x0100</td><td>256</td></tr><tr><td>0x0200</td><td>512</td></tr><tr><td>0x0400</td><td>1024</td></tr><tr><td>0x0800</td><td>2048</td></tr><tr><td>0x1000</td><td>2096</td></tr></table>	Legal Values	# cycles	0x0040	64	0x0080	128	0x0100	256	0x0200	512	0x0400	1024	0x0800	2048	0x1000	2096
Legal Values	# cycles																				
0x0040	64																				
0x0080	128																				
0x0100	256																				
0x0200	512																				
0x0400	1024																				
0x0800	2048																				
0x1000	2096																				
Lockout_assert_disable	WO	<13>	0	0	0 = the Cached CUD will assert MC_LOCKOUT_L when the lockout_counter reaches threshold. 1 = the Cached CUD will never assert MC_LOCKOUT_L.																
Lockout_react_disable	WO	<14>	0	0	0 = the Cached CUD will respond to MC_LOCKOUT_L assertion from other nodes 1 = the Cached CUD will ignore MC_LOCKOUT_L.																
MB_Cack	WO	<15>	0	0	0 = the Cached CUD will ignore Memory_Barrier (MB) commands from the 21164. 1 = the Cached CUD will Cack the MB command when driven by the 21164 ³²																
SD_Fill_Enable	WO	<16>	0	0	0 = SHARED and DIRTY are mutually exclusive. 1 = Cache CUD will assert SHARED and DIRTY to the EV5 during fills for READ_MODs when MC_SHARED_L is asserted.																
reserved	WO	<23:17>																			

Table 4-11 CONFIG_VALUE field of the CC_CONFIG register

³² The 21164 can be programmed to drive the MB command to the system for acknowledgment, or be Cacked internally - which will be the default setting.

4.3.1.2 DTAG_ENABLE

This register is in the VCTY ASIC on the Cached CPU Daughter Card. On initialization the DTAG is disabled. This register should be written prior to initiating normal operation.

Address of 39	DTAG Registers 32:28		Initial State	Firmware Init'd State	Value Description
1	10100	DTAG_ENABLE	1	1	A write enables the DTAG controller in VCTY to monitor MC Bus to maintain cache coherency with other nodes, without the EV5 snooping it's internal cache for all memory transactions.
1	10101	DTAG_DISABLE	0	0	A write causes the DTAG controller in VCTY to treat all MC Bus transactions as non-cacheable transactions. This mode should only be set in a diagnostic environment

4.4 General Registers - Description

4.4.1 Device Type and Revision Register (PCI_REV - Offset = 000)

This register identifies the device as a Host bus to PCI bus bridge. It's purpose is sizing the system, all MC bus devices with globally readable registers are expected to implement this register. It also contains the module revision number.

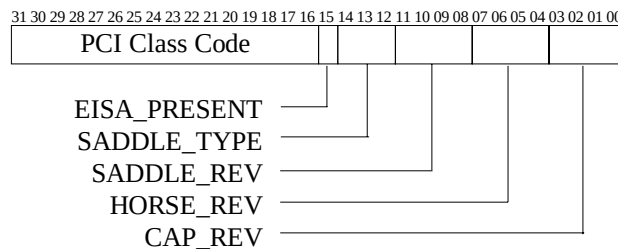


Figure 4-2 PCI Revision Register

Name	Extent	Access	Initial State	
CAP_REV	<3:0>	RO		Revision of the CAP chip
HORSE_REV	<7:4>	RO		Serial shift in from the Horse: REV<7:4> = Horse Module Revision
SADDLE_REV	<11:8>	RO		Shift in from Saddle (using interrupt shift register): REV<11:8> = Saddle Module Revision ³³
SADDLE_TYPE	<14:12>	RO		REV<14:12> = Saddle Module Type ³³ 000 - Left hand 001 - Right hand (Durango Type) 010 - 111 Reserved

³³ Unused inputs to the interrupt signal shift register are used as the source for this field.

EISA_PRESENT or EXT_ARBITER	<15>	RO		When set there is a PCI-EISA bus bridge on the PCI segment connected to this bridge. This bit disables the internal arbiter on the CAP chip.
PCI_CC	<31:16>	RO	0600h	PCI Host Bus Bridge class & subclass code (similar to how it is specified in PCI std)

4.4.2 Who Am I register(WHOAMI - Offset = 040)

This read only register is used by CPU daughter cards (CUDs) to determine their device number and hardware revision. The CUD issues a read of this register; encoded on the address lines of the read transaction are the device number and revision of the bus master. The MC-PCI Bridge takes the device number and revision information off the address lines and provides that as Fill data. This register is on PCI bus bridges

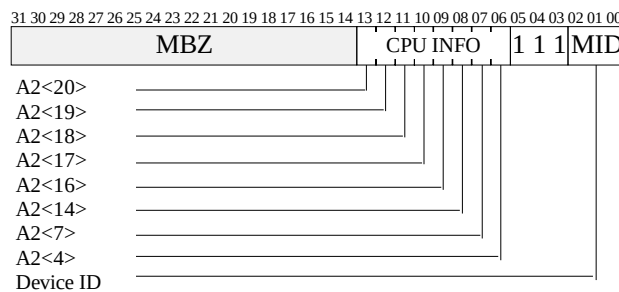


Figure 4-3 WHOAMI register

Name	Extent	Access	
Device ID	<2:0>	RO	This field echoes the Device Identification passed by the master in the second half of the address. It corresponds with the unique MID of the requester.
reserved	<5:3>	RO	Always 1
CPU Info<7:0>	<13:6>	RO	This register will return as data bits information passed on the address lines in the second cycle: A2<4> BIT<6> A2<7> BIT<7> A2<14> BIT<8> A2<16> BIT<9> A2<17> BIT<10> A2<18> BIT<11> A2<19> BIT<12> A2<20> BIT<13> Typically revision information is passed by the bus master on A2. CPU Daughter cards use this field to pass the Device ID of a FILL_ERROR source to software.
reserved	<31:14>		

The table above shows what the IOD will do with a WHOAMI register read. The table below shows what the Cached CPU Daughter card will put on the address lines to be accessed through this register. It is presumed that the Uncached CUD does the same as the Cached version.

The WHOAMI CPU_INFO bits are defined as follows for the Ponderosa (Cached-CUD) module:

CPU_INFO<7:0> = WHOAMI<13:6>

Name	Type	Description																
CPU_INFO<7>	RTC ³⁴	If 1, FILL_ERROR has occurred																
CPU_INFO<6>	RTC ³⁴	If 1, DTAG parity error has occurred																
CPU_INFO<5:0>	RO	If CPU_INFO<7> = 1 THEN ADDR<38:33> of reference causing FILL_ERROR If CPU_INFO<7> = 0 THEN CPU_REV_INFO<5:0> <table border="1"><tr><td>CPU_REV_INFO<5:3></td><td>Module revision & VCTY ASIC revision</td></tr><tr><td>CPU_REV_INFO<2:0></td><td>BCache Size:</td></tr></table> BCache Size Field:<table border="1"><tr><th>Value</th><th>Meaning</th></tr><tr><td>0</td><td>Uncached CPU</td></tr><tr><td>1</td><td>1 MB</td></tr><tr><td>2</td><td>2 MB</td></tr><tr><td>3</td><td>4 MB</td></tr><tr><td>4 - 7</td><td>reserved</td></tr></table>	CPU_REV_INFO<5:3>	Module revision & VCTY ASIC revision	CPU_REV_INFO<2:0>	BCache Size:	Value	Meaning	0	Uncached CPU	1	1 MB	2	2 MB	3	4 MB	4 - 7	reserved
CPU_REV_INFO<5:3>	Module revision & VCTY ASIC revision																	
CPU_REV_INFO<2:0>	BCache Size:																	
Value	Meaning																	
0	Uncached CPU																	
1	1 MB																	
2	2 MB																	
3	4 MB																	
4 - 7	reserved																	

4.4.3 PCI Latency Timer Register (PCI_LAT Offset = 080)

This register specifies, in units of PCI bus clocks, the value of the PCI Master Latency Timer for this PCI bus master. At reset, the register is set to 0. (The field starts at bit 8 for consistency with ALCOR & with the PCI Standard.)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
0																PCI_LAT						0									

Figure 4-4 PCI Latency Register

Name	Extent	Access	Initial State	Firmware Initialized State	
reserved	<7:0>	RO	0	00	
PCI_LAT	<15:8>	RW	0	FF	PCI Master Latency timer in PCI Clock Cycles
reserved	<31:16>	RO	0	0000	

³⁴ RTC means this field has a side effect - it is cleared when read. These fields are always 0 when read by an uncached CPU daughter card.

Bit field diagram for the PCI Configuration Space (bits 31-0). The diagram shows a 32-bit register with various fields mapped to specific bit ranges. Fields include LED, EV56_BW_EN, reserved, DLY_RD_EN, PCI_MEM_EN, PCI_REQ64_EN, PCI_ACK64_EN, PCI_ADR_PAR_EN, MC_CA_PAR_EN, MC_NXM_EN, MC_BUS_MON_EN, reserved, PEND_NUM, RD_TYPE, RL_TYPE, RM_TYPE, PARTIAL_WR_EN, reserved, and ARB_MODE<1:0>.

Figure 4-5 CAP_CTRL

Control and Status Registers

Name	Extent	Access	Initial State	Firmware Init'd State	
LED	<0>	RW	0	1	0: Selftest passed LED off 1: Selftest passed LED on
EV56_BW_EN	<1>	RW	0	0	0: Dense Memory has longword write/quadword read granularity 1: Dense Memory is linear and supports byte/word ³⁵
reserved	<3:2>		0	0	
DLY_RD_EN	<4>	RW	0	1	0: Delayed reads on the PCI bus will not be done 1: In cases where the target latency is expected to exceed 32 PCI cycles, use the PCI delayed read protocol
PCI_MEM_EN	<5>	RW	0	1	0: Bridge will not respond to PCI transactions 1: Bridge will respond to PCI transactions
PCI_REQ64_EN	<6>	RW	0	1	0: Bridge will not request 64-bit PCI data transactions 1: Bridge will request 64 bit data transactions
PCI_ACK64_EN	<7>	RW	0	1	0: Bridge will not accept 64-bit data transactions 1: Bridge will accept 64 bit data transactions
PCI_ADR_PAR_EN	<8>	RW	0	1	0: Do not check PCI Address parity 1: Check PCI Address parity and Enable SERR# drive onto the PCI bus
MC_CA_PAR_EN	<9>	RW	0	1	0: Do not check MC bus CMD/Address Parity 1: Check MC bus CMD/Address Parity
MC_NXM_EN	<10>	RW	0	1	0: Do not check for MC bus NXM 1: Check for MC bus NXM (absence of PEND) on IO space reads, and for TOP_OF_MEM errors.
MC_BUS_MON_EN	<11>	RW	0	1	0: Do not check for errors when a bystander 1: Check all transactions for errors
reserved	<15:12>	RO	0	00	
PEND_NUM	<19:16>	RW	0	xxxx ³⁶	This field controls the number of posted write transactions the MC-PCI bridge will acknowledge before pending writes. See Table 4-12
RD_TYPE	<21:20>	RW	00	00	This field controls the Prefetch algorithm used for PCI memory read command. See Table 4-14 below.
RL_TYPE	<23:22>	RW	00	01	This field controls the Prefetch algorithm used for PCI memory read line command. See Table 4-14 below.
RM_TYPE	<25:24>	RW	00	10	This field controls the Prefetch algorithm used for PCI memory read multiple command. See Table 4-14 below.
PARTIAL_WR_EN (Partial Write)	<26>	RW	00	0 uncached CPU 1 Cached CPUs	This field controls how 16 byte aligned blocks are written to memory. 0: All less than 64 byte blocks use READ/MODIFY/WRITE 1: 16 byte aligned blocks are written using MC_BMSK. If Dirty is signaled then transaction replayed as READ/MOD/WRITE.
reserved	<29:27>	RO	00	00	
ARB_MODE<1:0>	<31:30>	RW	0	00	This field controls the arbitration algorithm used on the PCI. If Saddle module type matches a value that includes an EISA bridge then arbitration is disabled. See Table 4-13

³⁵ Not currently supported. When there are EV56 CPU daughter cards that support this then it can be tested and enabled.³⁶ Configuration Dependent

4.4.4.1 Pended Write Threshold

The value of the PEND_NUM field controls the number of outstanding write transactions that will be acknowledged (ACKed). When more than **PEND_NUM** posted write transactions are outstanding subsequent write transactions to (or through) the MC-PCI bus bridge will be pended. This PEND_NUM threshold value should be set such that it allows sufficient buffer space for at least one outstanding write from each of the other nodes in the building block. Firmware is expected to initialize this field as a function of how many devices are in the building block.

Name	Algorithm
PEND_NUM	$12 - [((x-1)*2) + y]$ Where 12 is the total number of write buffers available on each MC-PCI bus bridge x = Number of MC-PCI bus bridges in the building block ³⁷ . y = Number of CPUs in the building block.

Table 4-12 Write Pend Number Threshold

For Example, a Durango System configured with 2 CPU's and 4 PCIs:

Number of write buffer entries in each MC-PCI bus bridge:	12	
(number of MC-PCI bus bridges in the building block minus one) * 2		6
(number of CPU's in the building block) * 1		2
Total number allocation of buffers (sum)		8
Difference is the value loaded in the PEND_NUM field	4	

For this example, the PEND_NUM field would be set to a value of 4.

4.4.4.2 Arbitration Algorithm

Value	Name	Algorithm
00	MC-PCI Bridge Priority Mode	Grant order between devices is round robin with exception that once the MC-PCI Bridge wins it continues to be granted PCI bus on successive arbitration cycles as long as there is work in the incoming command queue or until its latency timer expires. Latency timer starts on 1st grant and continues to count across successive requests that are won by the MC-PCI Bridge as if they were a single request. Device grant order is MC-PCI Bridge, D1, D2, D3, D4, D5, MC-PCI Bridge, This is the default power up arbitration mode for the MC-PCI Bridge.
01	Round Robin	Device grant order is MC-PCI Bridge, D1, D2, D3, D4, D5, MC-PCI Bridge.
10	Modified Round Robin	Round robin with MC-PCI Bridge winning every other cycle. - device grant; order is MC-PCI Bridge, D1, MC-PCI Bridge, D2, MC-PCI Bridge, D3, MC-PCI Bridge, D4, MC-PCI Bridge, D5, MC-PCI Bridge, D1, ...
11	reserved	

Table 4-13 PCI Arbitration Mode

The Priority mode is the default because it minimizes the time MC bus devices with pended reads and writes are kept from making forward progress, maximizes the bandwidth that can be delivered to peer to peer transfers.

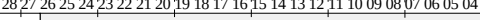
4.4.4.3 Prefetch Algorithm

Value	Prefetch algorithm
0 0	Short
0 1	Medium
1 0	Long
1 1	reserved

Fetch requested LW/QW and remainder of cache block
 Prefetch next block and no more. Will not cross 8KB page boundary
 Keep pre-fetching until PCI transaction completes. Will not cross 8KB page.
 (Defaults to 0 0 - Short Prefetch)

Table 4-14 PCI READ Prefetch algorithm

³⁷ MC-PCI bus bridge will not allocate a buffer for self-directed transfers across the MC bus.



The diagram shows a horizontal bar representing a 32-bit register. Above the bar, bit positions are numbered from 31 down to 0. The bar is divided into several sections by vertical lines. A label 'MBZ' (Meaningless Bit Zone) is placed in the center of the bar, spanning from bit 15 to bit 10. A line points from the text 'HAE SPARSE MEM' below to the first section of the bar, which corresponds to bits 31 down to 27.

System Address to Sparse PCI Memory Address Translation

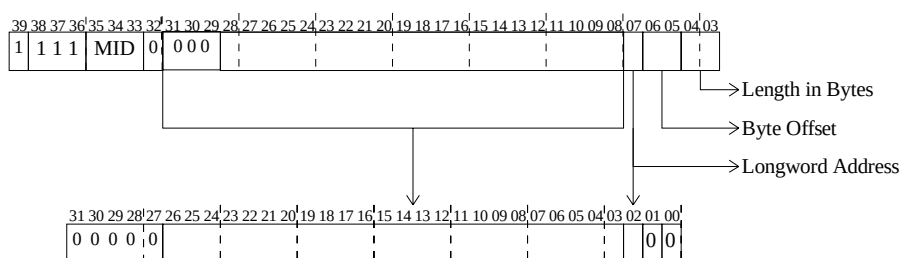


Diagram illustrating the structure of the `HAE_SPARSE_MEM` register (bits 31:27):

- Bit 31: 1
- Bits 30:28: 111
- Bit 27: MID
- Bit 26: 0
- Bits 25:03: Non-Zero (indicated by an arrow pointing to bit 25)
- Bits 24:03: Length in Bytes (indicated by an arrow pointing to bit 24)
- Bits 24:02: Byte Offset (indicated by an arrow pointing to bit 24)
- Bits 24:00: Longword Address (indicated by an arrow pointing to bit 24)

The register is labeled `HAE_SPARSE_MEM<31:27>`.

Address translation for the remaining 112 MB of PCI Sparse Memory Space

Figure 4-7 PCI Sparse Memory Address Translation

Name	Extent	Access	Initial State	Firmware Initialized State	
reserved	<26:0>	RO	0	0	
HAE SPARSE MEM	<31:27>	RW	0	0	PCI Sparse address bits <31:27>

4.4.6 Sparse IO Host Address Extension (HAE_IO - Offset = 440)

This HAE_IO Host Address Extension Register is used to extend a PCI Sparse-Space memory address up to the full 32-bit PCI address. In Sparse Addressing mode, the CPU address provides the low-order PCI address bits, while the HAE_IO provides the higher order bits. To insure that PIO transactions that target sparse PCI IO space use the most recent version of this register, software must read this register after writing it



Figure 4-8 Host Address Extension Register:(HAE_IO)

System Address to Sparse PCI IO Address Translation

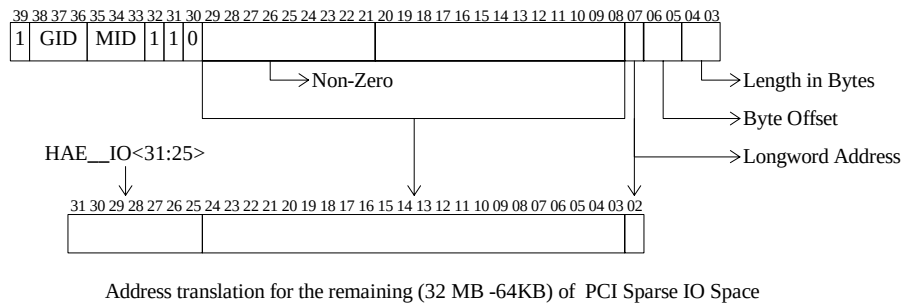
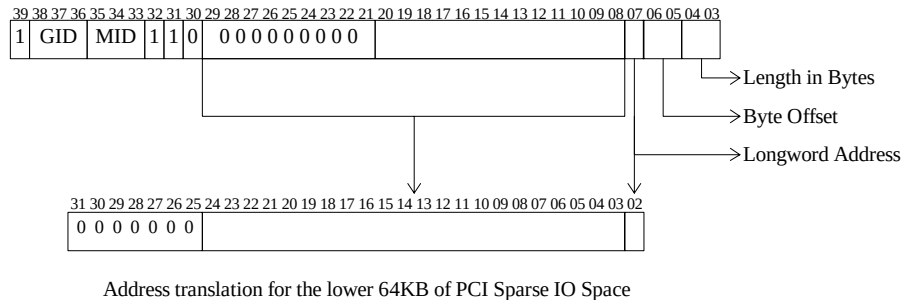


Figure 4-9 PCI Sparse IO space Address Translation³⁸

Name	Extent	Access	Initial State	Firmware Initialized State	
reserved	<24:0>	RO	0	0	
HAE_IO	<31:25>	RW	0	0	PCI address bits <31:25> for the upper 24 MB of sparse IO space

³⁸ The size of the fixed window is the same as specified for the Sable T2.

4.4.7 Special Cycle/Interrupt Register (IACK_SC - Offset = 480)

The PCI Special Cycle command provides a simple message broadcasting mechanism on the PCI. The Intel processor uses this cycle to broadcast processor status; but in general it may be used for logical sideband signaling between PCI agents.

The Special cycle contains no explicit destination address, but is broadcast to all agents. Rawhide will drive TBD as the Special Cycle address. Each receiving agent must determine if the message contained in the data field is applicable to it.

A write to the IACK_SC register causes a special cycle on the PCI. The write data will be passed unmodified to the PCI. Software must write data with the following field definitions:

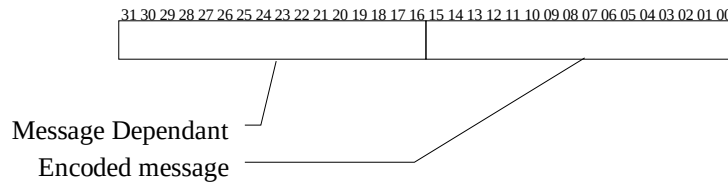


Figure 4-10 IACK_SC Register: **Write Data Format**

A PCI Interrupt Acknowledge cycle is a simple mechanism to collect an interrupt vector from a PCI-(E)ISA bus bridge. A read of the IACK_SC register will result in a PCI Interrupt Acknowledge cycle and the return data is the vector provided by the PCI-EISA bridge.

4.4.8 Dense Memory Host Address Extension (HAE_DENSE - Offset = 4C0)

This HAE_DENSE Host Address Extension Register is used to extend a PCI Dense-Space memory address up to the full 32-bit PCI address. In Dense Addressing mode, the CPU address provides the low-order PCI address bits, while the HAE_DENSE provides the higher order bit. To insure that PIO transactions that target dense memory space use the most recent version of this register, software must read this register after writing it

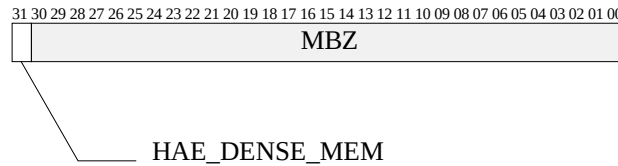


Figure 4-11 Host Address Extension Register: Dense Memory

System Address to Dense PCI Memory Address Translation

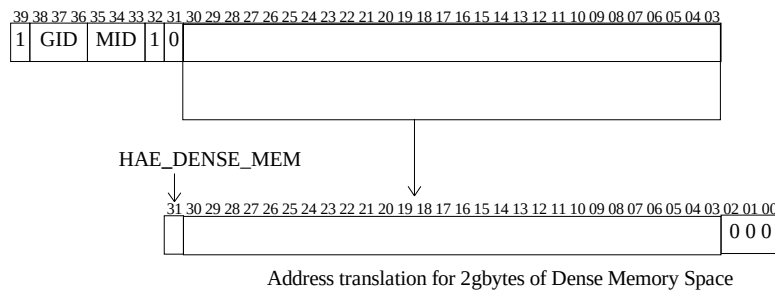


Figure 4-12 PCI Dense Memory Address Translation

Name	Extent	Access	Initial State	Firmware Initialized State	
reserved	<30:0>	RO	0	0	
HAE_DENSE_MEM	<31>	RW	0	0	PCI Dense address bits <31>

4.4.9 Top of Memory Register (TOP_OF_MEM; Offset = 7C0)

This register is used to detect when a rogue application (or software bug) causes transactions that are above the top of memory. When CAP_CTL:MC_NXM_EN is set, all MC bus memory addresses will be compared to the TOP_OF_MEM register. If the MC bus memory address is greater than MC_ADR<38:26> contained in the TOP_OF_MEM register, the IOD will set its CAP_ERR:NXM bit and lock up MC_ERR0 and MC_ERR1 (if not already locked by a previous HARD error). If a previous HARD error existed, then CAP_ERR:LOST_MC_ERR will be set instead.

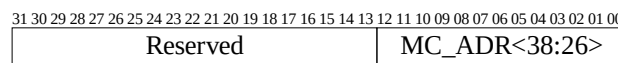


Figure 4-13 TOP_OF_MEM

Name	Extent	Access	Initial State	Firmware Initialized State	
MC_ADR<12:0>	<12:0>	RW	0	XX ³⁹	Contains the address of the top of memory.
reserved	<31:13>	RO	0	0	

The basic top-of-mem table is as follows:

TOP_OF_MEM Register	Memory Size
0000000000000	64Mb
0000000000001	128Mb
0000000000010	192Mb
0000000000011	256Mb
...	
1111111111111	256Gb

4.5 Interrupt Registers

Interrupts are dispatched to software by a combination of hardware and PAL or HAL code. The hardware mechanism that causes an IO interrupt to be posted is a bridge initiated write to a CPU daughter card. Data from which an interrupt vector is derived is written to a data structure in memory and also stored in a register (INT_REQ).⁴⁰

From the point of view of the CPU Daughter card hardware, the following addresses are used in the context of interrupt posting and acknowledgment. This is typically managed by PAL/HAL code and invisible to the operating system:

Address	Bits					
39	38:33	32:28	27:4	Function	Direction	CPU Daughter Card (CUD) Action
1	CUD_ID	01111	xx	IO_INTR	IOD to CUD	Increment INTR_CTR<4:0> (Assert IRQ<1>)
1	IOD_ID	11111	0003F0	IO_ACK0	CUD to IOD	If CUD initiates the write, Decrement INTR_CTR <4:0>
1	IOD_ID	11111	0003F4	IO_ACK1	CUD to IOD	If CUD initiates the write, Decrement INTR_CTR <4:0>
1	IOD_ID	11110	xx	All other IOD CSRs		
1	CUD_ID	00111	xx	NodeHalt_INTR	IOD to CUD	Assert MCH_HLT_IRQ
1	CUD_ID	10111	xx	NodeHalt_ACK	CUD to itself	Release MCH_HLT_IRQ
1	CUD_ID	10000	xx	INTTIM_ACK	CUD to itself	Release IRQ<2> (Interval Timer Interrupt)
1	CUD_ID	00001	xx	IP_INTR	CUD to another CUD	Assert IRQ<3> (Interprocessor Interrupt)
1	CUD_ID	10001	xx	IP_ACK	CUD to itself	Release IRQ<3>
1	CUD_ID	10010	xx	MCHK_ACK	CUD to itself	Release SYS_MCH_CHK_IRQ
1	CUD_ID	10011	xx	rsvd (IRQ0_ACK?)	CUD to itself	Release IRQ<0>

³⁹ Loaded with the value Firmware determines to represent the top of memory. Firmware uses the I²C bus to size memory.

⁴⁰ The intended audience of this section are those writing PAL or HAL code. OS software is not expected to know about the details of how hardware and PAL/HAL convert hardware signals into interrupts.

1	CUD_ID	11110	xx	reserved	CUD to itself	May be used to cause posted write buffers to be flushed.
---	--------	-------	----	----------	---------------	--

Table -4-154-16 Global Interrupt Target Address

4.5.1 Interrupt Control (INT_CTL - Offset = 500)

Interrupt Transactions for debug may be disabled for hardware and software debug. This register contains bits that disable interrupt transactions and the write of interrupt information to memory.



Figure 4-14 INT_CTL Register

Name	Extent	Access	Initial State	Firmware Initialized State	
EN_INT	<0>	RW	0	1	0: Disables MC-PCI bus bridge issuing MC Bus IO interrupt transactions and disables writing Device Interrupt information to memory. 1: Enables MC-PCI bus bridge to issue MC Bus IO Interrupt transactions.
EN_INT_NUM	<1>	RW	0	1	0: Device Interrupt information is not written to memory. If this bit is clear then hardware signals the interrupt to Interrupt Target 0 and PAL, HAL or other interrupt related software is expected to use INT_ACK0 to acknowledge the event. 1: If EN_INT = 1 Then this enables writing Device Interrupt information structure identified in INT_ADR and INT_ADR_EXT
reserved	<31:2>	RO	0	0	

4.5.2 Interrupt Request (INT_REQ - Offset = 540)

This register is used to read the state of the interrupt lines from the IO subsystem. With interrupts disabled this register may be polled to sense when an interrupt is active. This register is for use by Firmware and will reflect the state of the INT lines. It can not be disabled.

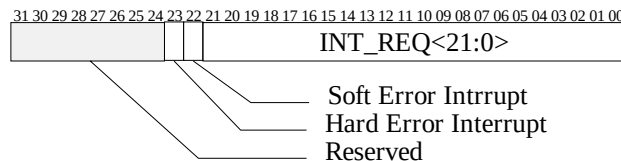


Figure 4-15 INT_REQ Register

Control and Status Registers

Name	Extent	Access	Initial State	
INT_REQ	<21:0>	R/W1C	0	0: no interrupt asserted 1: interrupt asserted
Soft ERR	<22>	RO	0	Correctable error interrupt flag. Reflects state of CAP_ERR<28:27>
Hard ERR	<23>	RO	0	Hard error flag. Reflects the logical or of the state in CAP_ERR<30:29,26:25,23,3:0>.
reserved	<31:24>	RO	0	

4.5.3 Interrupt Target Device (INT_TARG - Offset = 580)

Interrupt Target Device ID & Interrupt Target Address registers control the action taken in the event of an IO or error interrupt. The interrupt Target Device number is used as an address offset to a CPU specific data structure in memory (as shown in Figure 4-24 Address of interrupt data structure in memory) and as address bits 38:33 of the IO_INTR target CPU (as shown in Table -4-154-16 Global Interrupt Target Address). Restrictions: INT1_TARG_DEV_ID and INT0_TARG_DEV_ID must be initialized to different values. Otherwise the two target data structures would address the same location in memory and interrupts could be lost.

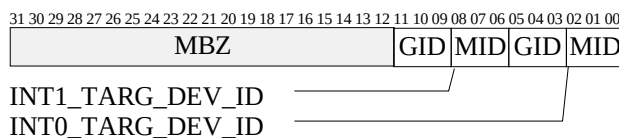


Figure 4-16 Interrupt Target ID

Name	Extent	Access	Initial State	Firmware Initialized State	
INT0_TARG_DEV_ID	<5:0>	RW	0	Boot Processor ID	Identifies the physical address to be used when generating <u>Interrupt Target 0</u> transactions on MC bus. Used by HALT and IO interrupts. ⁴¹
INT1_TARG_DEV_ID	<11:6>	RW	0	Boot Processor ID	Identifies the physical address to be used when generating <u>Interrupt Target 1</u> transactions on MC bus. Used only by IO interrupts.
reserved	<31:12>	RO	0	0	

4.5.4 Interrupt Target Address (INT_ADR - Offset = 5C0)

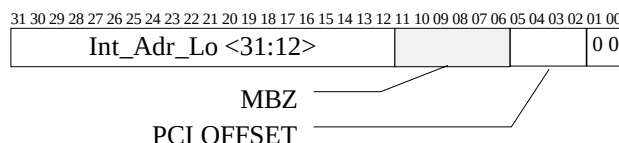


Figure 4-17 Interrupt Target Address Register

⁴¹ The Interrupt Target for HARD_ERROR interrupts must be programmed to the same CPU on platforms that expect SRM PAL code to collect symptoms in the event of HARD_ERROR interrupts (INT_REQ<23>).

Control and Status Registers

Name	Extent	Access	Initial State	Firmware Initialized State	
reserved	<1:0>	RO	0	0	Longwords only
PCI OFFSET	<5:2>	RW	0	TBD ⁴²	Specifies the offset of the interrupt target CPU data structure.
reserved	<11:6>	RO	0	0	
INT_ADDR_LO	<31:12>	RW	0	TBD ⁴²	Specifies the page address of the target for writes of interrupt information to memory

4.5.5 Interrupt Target Address Extension (INT_ADR_EXT - Offset = 600)

Interrupt Target CUD Device ID & Interrupt Target Word Address registers control the action taken by the MC-PCI Bridge in the event of an IO or error interrupt. The address used is the location in memory to which the MC-PCI Bridge will write the vector. This register specifies the top 7 bits of the address.

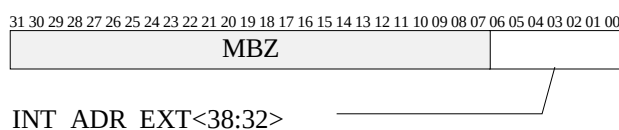


Figure 4-18 Interrupt Address Extension

Name	Extent	Access	Initial State	Firmware Initialized State	
INT_ADR_EXT<38:32>	<6:0>	RW	0	TBD ⁴²	Specifies the upper address bits to be used when writing interrupt information into memory.
reserved	<31:7>	RO	0	0	

4.5.6 Interrupt Mask Register (INT_MASK0 - Offset = 640)

This register controls which bits in the INT_REQ register will cause an interrupt to be posted to Interrupt Target 0 on the MC bus as specified in the MC bus interrupts section of the Rawhide Bus Specification. It also controls which interrupt bits in INT_REQ should be cleared when INT_ACK0 is written.

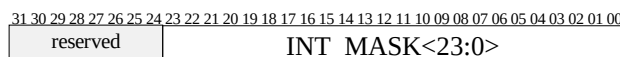


Figure 4-19 INT_MASK0 register

Name	Extent	Access	Initial State	Firmware Initialized State	
INT_MASK	<23:0>	RW	0	FFFFFF	0: Interrupt IRQ Disabled 1: Interrupt IRQ Enabled
reserved	<31:24>	RO	0	00	

If configuration firmware or other software programs the INT_MASK0 and INT_MASK1 registers to direct the same interrupt to different processors then software should expect passive release.

⁴² Configuration Dependent.

4.5.7 Interrupt Mask Register (INT_MASK1 - Offset = 680)

This register controls which bits in the INT_REQ register will cause an interrupt to be posted to Interrupt Target 1 on the MC bus as specified in the MC bus interrupts section of the Rawhide Bus Specification. It also controls which interrupt bits in INT_REQ should be cleared when INT_ACK1 is written.

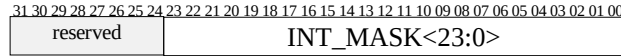


Figure 4-20 INT_MASK1 register

Name	Extent	Access	Initial State	Firmware Initialized State	
INT_MASK	<23:0>	RW	0	000000	0: Interrupt IRQ Disabled 1: Interrupt IRQ Enabled
reserved	<31:24>	RO	0	00	

4.5.8 Interrupt Acknowledge Register (INT_ACK0 - Offset = 10003F00)

This register is used to acknowledge an interrupt and permit the MC-PCI Bridge to signal another to the CPU Daughter Card identified as Interrupt Target 0.

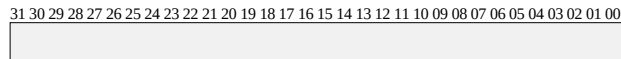


Figure 4-21 INT_ACK0

Name	Extent	Access	
INT_ACK0	<31:0>	WO	When written the MC-PCI Bridge may issue another interrupt targeting the CPU identified by INT0_TARG_DEV_ID

4.5.9 Interrupt Acknowledge Register (INT_ACK1 - Offset = 10003F40)

This register is used to acknowledge an interrupt and permit the MC-PCI Bridge to signal another to the CPU Daughter Card identified as Interrupt Target 1.

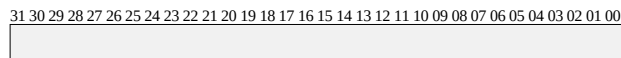


Figure 4-22 INT_ACK1

Name	Extent	Access	
INT_ACK1	<31:0>	WO	When written the MC-PCI Bridge may issue another interrupt targeting the CPU identified by INT1_TARG_DEV_ID

4.5.10 Interrupt Hardware

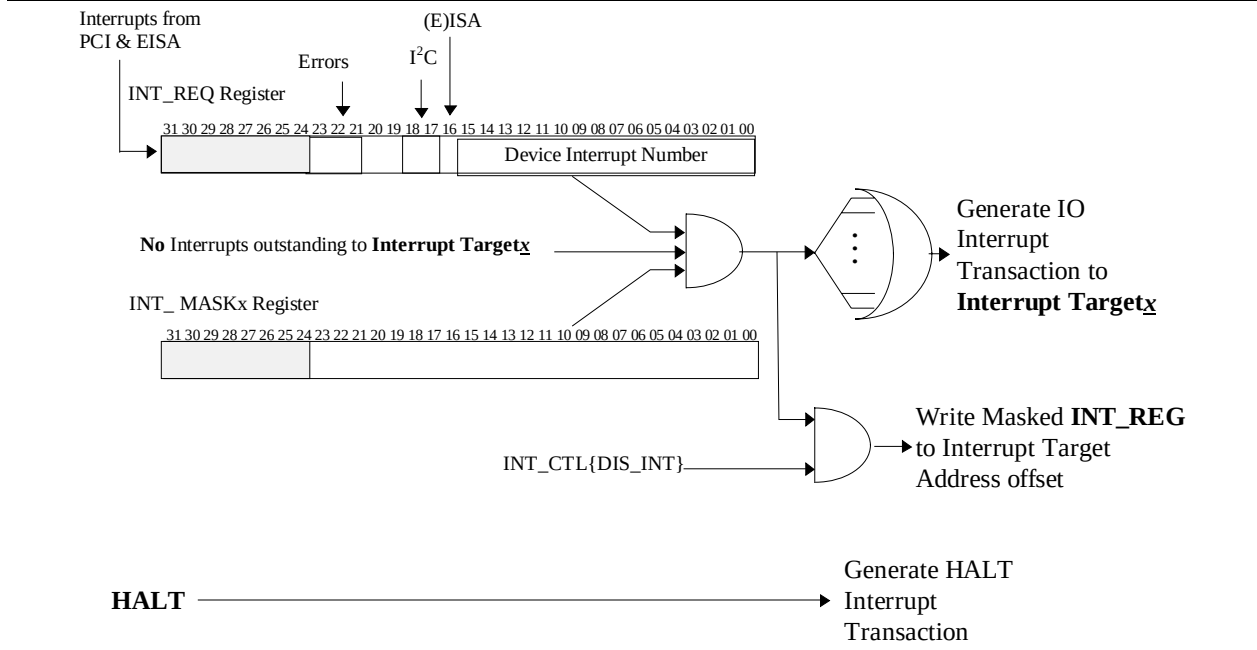
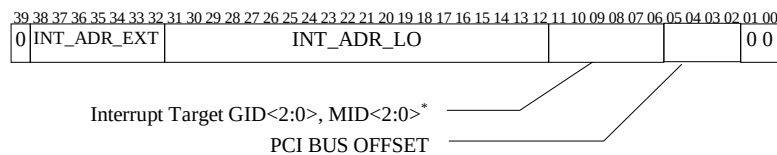


Figure 4-23 Interrupt Generation on PCI bus 0

When an interrupt is signaled to the CAP chip, the vector is written to a location in memory established by configuration software (by writing INT_ADR and INT_ADR_EXT). The Data structure in memory is 4 bytes [times up to 16 MC bus to PCI bus bridges]:



* Specified in the INT_TARG CSR

Figure 4-24 Address of interrupt data structure in memory

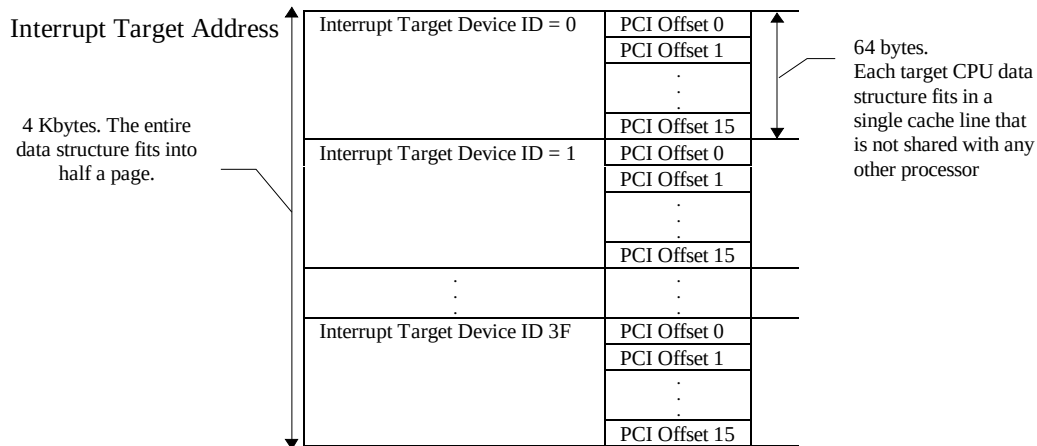


Figure 4-25 Interrupt Target Data Structure in Memory

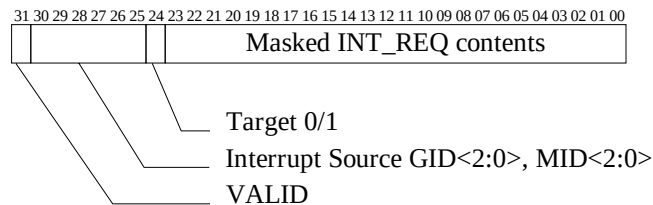


Figure 4-26 Interrupt Target Memory Data Structure Record Format

Name	Extent	Description
<i>Masked INT_REQ</i>	<23:0>	This is the contents of the INT_REQ, masked by the contents of the INT_MASK register. PAL or HAL code use this to calculate an interrupt vector.
Target 0/1	<24>	This identifies which of the INT_ACKx registers PAL/HAL will write when the posted interrupts have been dispatched.
Interrupt Source	<30:25>	Identifies the PCI that is the source of the interrupt.
Valid	<31>	Set when the MC-PCI bus bridge writes to this location; cleared by PAL code.

Table 4-17 Interrupt Target Data Structure Record Format

The combination of the Interrupt Source and each bit in the Masked INT_REQ field is mapped by PAL or HAL code into an interrupt vector.

Interrupt information is also stored unmasked in the INT_REQ register. As all interrupts are level sensitive; the state of INT_REQ is updated by hardware each time the MC-PCI bus bridge INT_ACKx register is written.

When it is an EISA device interrupt, PAL/HAL, or first level interrupt code will read the PCI Interrupt acknowledge register to cause an interrupt acknowledge cycle which in turn fetches the 8 bit vector from the PCI-EISA bridge.

Software (PAL/HAL or ??) will write to INT_ACKx on the MC-PCI bridge identified in the Interrupt Source and Target 0/1 field. When the write to INT_ACKx is observed the CPU Daughter Card will decrement a pending interrupt counter which is causing the IRQ signal to be asserted.

4.5.11 Interrupt logic vector assignments

Firmware or configuration software are expected to load the PCI Standard specified 8 bit Interrupt Line configuration space header entry on each device with the IRQ number. See Chapter 5 for details on interrupt vector assignments

4.6 Performance Monitor Registers

ALCOR implemented a set of registers to count a variety of events in the system. A limited number of registers have been implemented on Rawhide.

4.6.1 Performance Monitor Register (PERF_MON; Offset = 300)

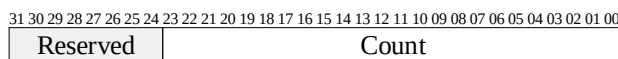


Figure 4-27 PERF_MON; Offset = 300)

Name	Extent	Access	Initial State	Firmware Initialized State	
Count	<23:0>	RW	0	00000000	Counts what is specified in PERF_CON.
reserved	<31:24>	RO	0	0	

4.6.2 Performance Monitor Control Register (PERF_CON; Offset = 340)

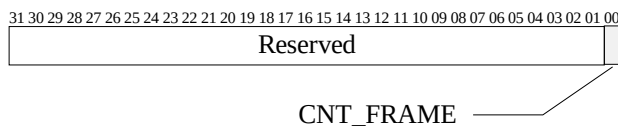


Figure 4-28 PERF_CON; Offset = 340)

Name	Extent	Access	Initial State	Firmware Initialized State	
CNT_FRAME	<0>	RW	0	0	0: Increment PERF_MON every PCI cycle that FRAME is asserted 1: Increment PERF_MON on every TLB miss
reserved	<31:1>	RO	0	0	

A 32 bit counter incrementing every PCI cycle FRAME is asserted to measure how busy the PCI is.

A 32 bit counter incrementing every time a TLB miss/refill occurs.

4.7 Diagnostic Registers - Description

As the PCI master and slave state machines are independent, it is expected that diagnostics will be able to do self directed transactions by reading or writing regions of dense space that are mapped into global memory through one of the programmable DMA windows. This provides a mechanism for testing all data lines though the CAP and MDP chips.

4.7.1 MC-PCI Bridge Diagnostic Control Register (CAP_DIAG - Offset = 700)

Figure 4-29 Force Error Register CAP_DIAG

Name	Extent	Access	Initial State	Firmware Initialized State	
PCI_RESET	<0>	WO	0	0	0: Reset not asserted on the PCI 1: Reset asserted on the PCI. This bit should be cleared by software at least 100μSeconds after it was set. WARNING: The only transaction to address space allocated to this bridge permitted when this bit is set is a write to this CSR to clear this bit. Attempts to read bridge CSRs when this bit is set are likely to result in deadlock. To avoid choking this bridge do not read or write to this address space while this bit is set.
reserved	<29:1>	RO	0	0	
F_MC_ADRPE	<30>	RW	0	0	0: Normal parity is output to the MC bus 1: MC Command/Address parity is inverted.
F_PCI_ADRPE	<31>	RW	0	0	0: Normal parity is output to the PCI 1: Bad parity is forced onto the low 32 bits of the PCI during an address cycle.

4.7.2 General Purpose Register for Firmware (SCRATCH; Offset = 740, Alias = 780)

This register is used by firmware for communication and synchronization between processors during power-up and reset. When read at offset 740, the contents of SCRATCH<31:0> are returned with no side effects. When read at the aliased offset of 780, Lock (bit 31) is set after the read as a side effect.

Bit 31 is used as a "read-lock" bit. Firmware on a CPU that finds bit 31 of SCRATCH Alias clear may assume that it has locked the CSR. Firmware may want to re-read the CSR to verify that SCRATCH<31> has been set by hardware, especially during power-up self-test when the underlying hardware may be suspect. Once a CPU has locked the CSR, all other CPU's that read the CSR will find bit 31 set.

Any CPU can write or read the SCRATCH CSR at any time. It is up to software to use the lock feature for synchronization purposes. Software is expected to write the CSR only after it has acquired the lock. A lock-acquiring reader of SCRATCH is expected to later write SCRATCH with bit 31 clear. This will avoid deadlock and permit other CPUs to gain the lock.

The aliased nature of the SCRATCH register permits software to choose whether it desires the lock side effects or not. For example, console firmware EXAMINE command code or system software would likely not desire the lock side effect.

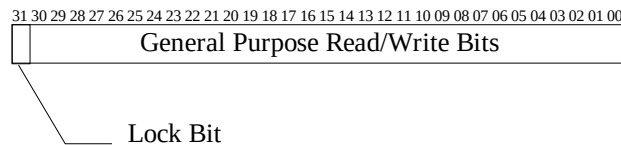


Figure 4-30 General Purpose Register for Firmware (SCRATCH; Offset = 740)

Name	Extent	Access	Initial State	Firmware Initialized State	
SCRATCH	<30:0>	RW	0	XX ⁴³	General purpose CSR. This is a scratch pad register to be used by Firmware in selection of a boot processor etc. The low four bits are used for a CPU present bit mask.
Lock Bit	<31>	RW	0	XX ⁴³	Set and cleared when written. Also set <i>after</i> it is read at address 780 (previous contents returned before bit set).

By convention, the SCRATCH register will be used by firmware as follows:

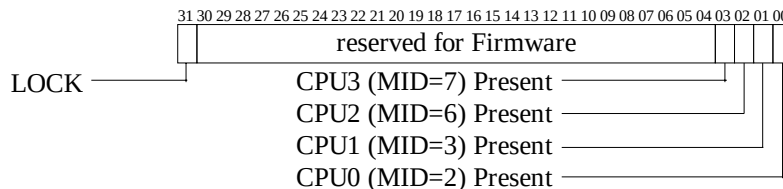


Figure 4-31 Firmware conventions for using SCRATCH register

⁴³ Firmware/Configuration Dependent.

4.7.3 Top of Memory Register (TOP_OF_MEM; Offset = 7C0)

This register is used to detect when a rogue application (or software bug) causes transactions that are above the top of memory. When CAP_CTL:MC_NXM_EN is set, all MC bus memory addresses will be compared to the TOP_OF_MEM register. If the MC bus memory address is greater than MC_ADR<38:26> contained in the TOP_OF_MEM register, the IOD will set its CAP_ERR:NXM bit and lock up MC_ERR0 and MC_ERR1 (if not already locked by a previous HARD error). If a previous HARD error existed, then CAP_ERR:LOST_MC_ERR will be set instead.

4.8 Error Symptom Registers

4.8.1 MC Error Information Register 0 (MC_ERR0 - Offset = 800)

The low order MC bus address bits are latched into this register when the MC-PCI Bridge detects an error event. If the event is a hard error the register bits are locked. A write to clear symptom bits in the CAP_ERR register unlocks this register. When the valid bit (MC_ERR_VALID) in the CAP_ERROR register is clear the contents are undefined.



Figure 4-32 MC_ERR0

Name	Extent	Access	Initial State	
reserved	<3:0>	RO	0	
ADDR<31:4>	<31:4>	RO	0	Contains the address of the current transaction on the MC bus when an error is detected.

4.8.2 MC Error Information Register 1 (MC_ERR1 - Offset = 840)

The high order MC bus address bits and error symptoms are latched into this register when the MC-PCI Bridge detects an error event. If the event is a hard error the register bits are locked. A write to clear symptom bits in the CAP_ERR register unlocks this register. When the valid bit (MC_ERR_VALID) in the CAP_ERROR register is clear the contents are undefined.

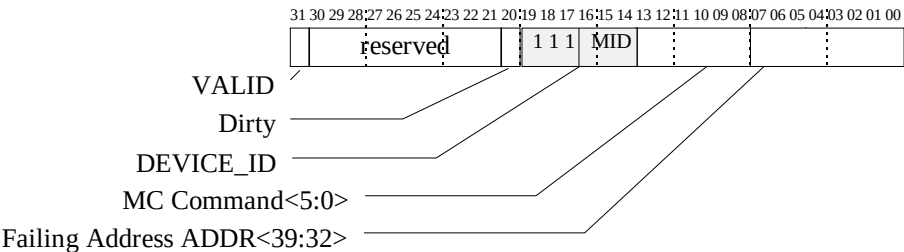


Figure 4-33 MC_ERR1

Name	Extent	Access	Initial State	
ADDR<39:32>	<7:0>	RO	0	Address bits <39:32> of the transaction on the MC bus when an error is detected.
MC_CMD<5:0>	<13:8>	RO	0	Active command at the time the error was detected ⁴⁴
DEVICE_ID	<19:14>	RO	0	MC slot number of bus master at the time of the error.
Dirty	<20>	RO	0	Set if the MC bus error was associated with a Read/Dirty transaction. When set the device ID field <19:14> does not indicate the source of the data.
reserved	<30:21>	RO	0	
VALID	<31>	RO	0	Logical OR of bits 30:23 in the CAP_ERR register. Set if MC_ERR0 & MC_ERR1 Contain a valid address

⁴⁴ See the Rawhide System Bus Specification for specification of this field.

4.8.3 CAP Error Register (CAP_ERR - Offset = 880)

CAP_ERR is used to log information pertaining to an error detected by the CAP or MDP ASIC. If the error is a hard error the register is locked. All bits, except the LOST_MC_ERR bit, are locked on hard errors. CAP_ERR will remain locked until the CAP_ERR is written to clear each individual error bit.

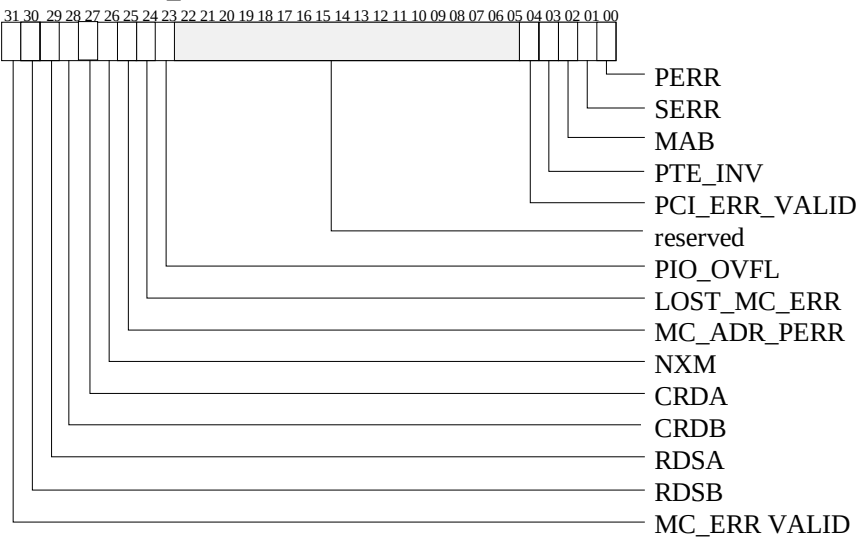


Figure 4-34 CAP_ERR

Control and Status Registers

Name	Extent	Access	Initial State	
PERR	<0>	RW1C	0	PCI master state machine observed PERR#
SERR	<1>	RW1C	0	PCI target state machine observed SERR#. CAP asserts SERR when it is Master and detects Target abort.
MAB	<2>	RW1C	0	PCI master state machine detected PCI Target Abort (likely cause: NXM) (except Special Cycle). On reads FILL error is also returned.
PTE_INV	<3>	RW1C	0	Invalid page table entry on Scatter/Gather access.
PCI_ERR_VALID	<4>	RO	0	Logical OR of bits 3:0 of this register. When set, the PCI error address register is locked.
Reserved	<22:5>	RO	0	
PIO_OVFL	<23>	RW1C	0	Set when a transaction that targets this MC-PCI bus bridge is not serviced because the buffers are full. This is a symptom of setting the PEND_NUM field in CAP_CNTL to an incorrect value.
LOST_MC_ERR	<24>	RW1C	0	Set when an error is detected but not logged because the associated symptom fields and registers are locked with the state of an earlier error.
MC_ADR_PERR	<25>	RW1C	0	Set when an MC bus command/address parity error is detected
NXM	<26>	RW1C	0	MC bus master transaction status NXM (Read with Address bit 39 set but transaction not pended or transaction target above the top of memory register) CPU will also get FILL error on reads.
CRDA	<27>	RW1C	0	Correctable ECC error detected by MDPA. Clear state in MDPA_STAT before clearing this bit.
CRDB	<28>	RW1C	0	Correctable ECC error detected by MDPB. Clear state in MDPB_STAT before clearing this bit.
RDSA	<29>	RW1C	0	Uncorrectable ECC error detected by MDPA. Clear state in MDPA before clearing this bit.
RDSB	<30>	RW1C	0	Uncorrectable ECC error detected by MDPB. Clear state in MDPB before clearing this bit.
MC_ERR_VALID	<31>	RO	0	Logical OR of bits 30:23 in this register. When set MC_ERR0 and MC_ERR1 are latched.

4.8.4 PCI Error Status Register 1 (PCI_ERR1 - Offset = 1040)

The PCI_ERR1 register is used by the MC-PCI Bridge to log bus address <31:0> pertaining to an error condition logged in CAP_ERR. This register always captures PCI address <31:0>, even for a PCI DAC cycle. When the PCI_ERR_VALID bit in CAP_ERR is clear the contents are undefined.



Figure 4-35 PCI_ERR1

Name	Extent	Access	Initial State	
ADDR<31:0>	<31:00>	RO	0	Contains address bits <31:0> of the current transaction on the PCI bus when an error is detected.

Table 4-18 PCI_ERR1

4.8.5 MDPA Status Register (MDPA_STAT - Offset = 4000)

Contains status information related to the error stored in the CAP_ERR register.



Figure 4-36 MDPA_STAT

Name	Extent	Access	Initial State	
MDPA_REV	3:0	RO	REV	Chip Revision Level
reserved	29:4	RO	0	reserved
CRD	30	R/W1C	0	MDPA_SYN contains syndrome of a CRD. Write one clears both bits <31:30>
RDS	31	R/W1C	0	MDPA_SYN contains syndrome of a RDS. Write one clears both <31:30>

4.8.6 MDPA Error Syndrome Register (MDPA_SYN - Offset = 4040)

The MDPA_SYN register is used by the MC-PCI Bridge to log information pertaining to an error in the low 64 bits on the MC bus detected by the ECC checkers. The syndrome is latched into the MDPA_SYN register upon an MC-PCI Bridge error. A write to clear MDPA_STAT<31 or 30> unlocks this register.

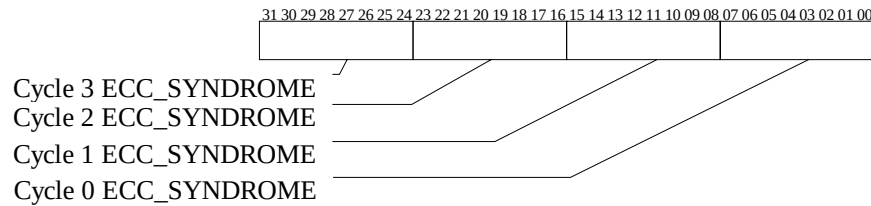


Figure 4-37 MDPA_SYN

Name	Extent	Access	Initial State	
Cycle 0 ECC_SYNDROME	<7:0>	RO	undefined	ECC Syndrome bits
Cycle 1 ECC_SYNDROME	<15:8>	RO	undefined	ECC Syndrome bits
Cycle 2 ECC_SYNDROME	<23:16>	RO	undefined	ECC Syndrome bits
Cycle 3 ECC_SYNDROME	<31:24>	RO	undefined	ECC Syndrome bits

4.8.7 MDPA Diagnostic Check Register (MDPA_DIAG - Offset = 4080)

The MDPA_DIAG is used for diagnostic DMA writes to memory. It is loaded with a known ECC pattern to be written to memory if the USE_CHK bit is set in the MDPA_DIAG register.

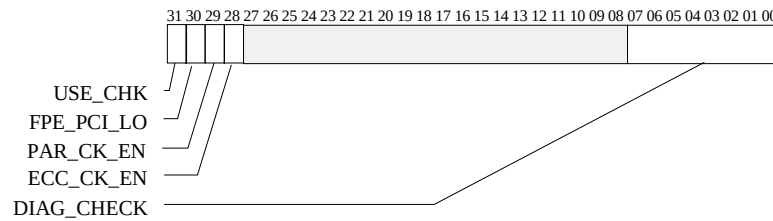


Figure 4-38 MDPA_DIAG

Name	Extent	Access	Initial State	Firmware Initialized State	
DIAG_CHECK	<7:0>	RW	0	00	Data used for ECC in diagnostic DMA writes.
reserved	<27:8>	RO	0	0	
ECC_CHK_EN	<28>	RW	0	1	0: ECC is not checked. 1: ECC is checked on all MC bus transactions.
PAR_CHK_EN	<29>	RW	0	0	0: PCI Data Parity is not checked 1: PCI data Parity is checked on all transactions that this interface consumes data.
FPE_PCI_LO	<30>	RW	0	0	Bad parity is forced onto the low 32 bits of the PCI during data cycles
USE_CHK	<31>	RW	0	0	When set, DMA write cycles use the value in DIAG_CHECK field as ECC sent on the System bus to memory

4.8.8 MDPB Status Register (MDPB_STAT - Offset = 8000)

Contains status information related to the error stored in the CAP_ERR register.



Figure 4-39 MDPB_STAT

Name	Extent	Access	Initial State	
MDPB_REV	3:0	RO	REV	Chip Revision Level
reserved	29:4	RO	0	
CRD	30	R/W1C	0	MDPB_SYN contains syndrome of a CRD. Write one clears both bits <31:30>
RDS	31	R/W1C	0	MDPB_SYN contains syndrome of a RDS. Write one clears both <31:30>

4.8.9 MDPB Error Syndrome Register (MDPB_SYN - Offset = 8040)

The MDPB_SYN register is used by the MC-PCI Bridge to log information pertaining to an error in the high 64 bits on the MC bus detected by the ECC checkers. The syndrome is latched into the MDPB_SYN register upon an MC-PCI Bridge error. A write to clear MDPB_STAT<31 or 30> unlocks this register.

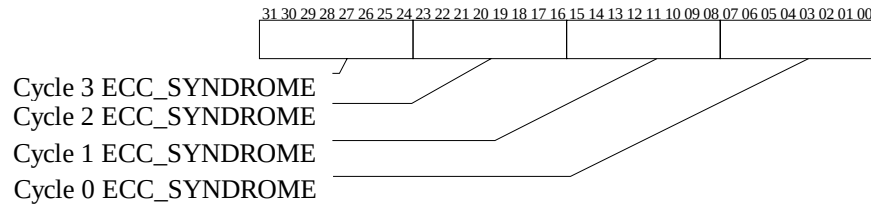


Figure 4-40 MDPB_SYN

Name	Extent	Access	Initial State	
Cycle 0 ECC_SYNDROME	<7:0>	RO	undefined	ECC Syndrome bits
Cycle 1 ECC_SYNDROME	<15:8>	RO	undefined	ECC Syndrome bits
Cycle 2 ECC_SYNDROME	<23:16>	RO	undefined	ECC Syndrome bits
Cycle 3 ECC_SYNDROME	<31:24>	RO	undefined	ECC Syndrome bits

4.8.10 MDPB Diagnostic Check Register (MDPB_DIAG - Offset = 8080)

The MDPB_DIAG is used for diagnostic DMA writes to memory. It is loaded with a known ECC pattern to be written to memory if the USE_CHK bit is set.

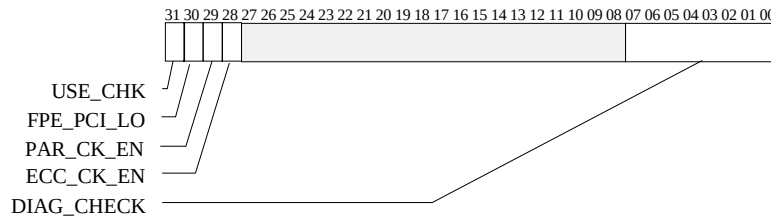


Figure 4-41 MDPB_DIAG

Name	Extent	Access	Initial State	Firmware Initialized State	
DIAG_CHECK	<7:0>	RW	0	00	Data used for ECC in diagnostic DMA writes.
reserved	<27:8>	RO	0	0	
ECC_CHK_EN	<28>	RW	0	1	0: ECC is not checked. 1: ECC is checked on all MC bus transactions.
PAR_CHK_EN	<29>	RW	0	0	0: PCI Data Parity is not checked 1: PCI data Parity is checked on all transactions that this interface consumes data.
FPE_PCI_LO	<30>	RW	0	0	Bad parity is forced onto the low 32 bits of the PCI during data cycles
USE_CHK	<31>	RW	0	0	When set, DMA write cycles use the value in DIAG_CHECK field as ECC sent on the MC bus to memory

4.9 PCI Address Related Registers: descriptions

4.9.1 Scatter/Gather Translation Buffer Invalidate (SG_TBIA - Offset = 1300)

A write to the SG_TBIA register will cause the specified group of scatter gather TLB TAGs to be marked invalid.

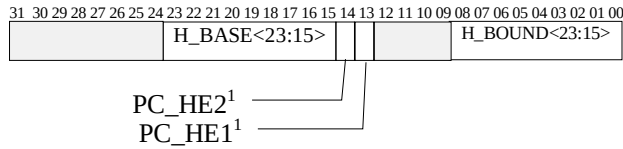


Figure 4-42 SG_TBIA

Name	Extent	Access	
CSR_WR_DATA<31:0>	<31:0>	WO	A Write to this register will invalidate the scatter/gather Translation Buffers. Data is "don't care"

4.9.2 PC “Hole” Compatibility Register (HBASE Offset = 1340)

This register provides the base and bounds address for a PC Compatibility hole and enables for that hole and one at a fixed address offset.



Note:
(1) W_BASE(0) only

Figure 4-43 HBASE⁴⁵

Name	Extent	Access	Initial State	Firmware Initialized State	
H_BOUND<23:15>	<8:0>	RW	0	00	When HBASE<14> is set , these bits are compared to PC_AD<23:15> of the incoming PCI Cycles to determine if the address falls below the upper bound of the programmable PC Compatibility “Hole”. The address contained in this register signifies the last valid upper PCI address in the hole. Bits <8:5> of this register must be non-zero when this hole is enabled.
reserved	<12:9>	RO	0	0	must be zero
PC_HE1	<13>	RW	0	1	When set the MC-PCI bus bridge will not respond to PCI addresses which fall into the fixed PC “Hole” between 512KB and 1MB.
PC_HE2	<14>	RW	0	0	When set the MC-PCI bus bridge will not respond to PCI addresses which fall between the programmable PCI “Hole” base and bounds address contained in HBASE<23:26> and HBASE<8:0> respectively.
H_BASE<23:15>	<23:15>	RW	0	0	When HBASE<14> is set , these bits are compared to PC_AD<23:15> of the incoming PCI Cycles to determine if the address falls above the lower bound of the programmable PC Compatibility “Hole”. Bits <23:20> of this register must be non-zero when this hole is enabled.
reserved	<31:24>	RO		0	

⁴⁵ This is intended to be identical to the Sable T2 implementation.

4.9.3 Window Base (W_x_BASE, x=0-3 - Offset = 1400, 1500, 1600, 1700)

The Window Base register provides the base address for a particular target window. There are 4 Window Base registers: W_BASE[0], W_BASE[1], W_BASE[2], and W_BASE[3]. The W_BASE[] registers should not be modified unless software insures that no PCI traffic is targeted for the window being modified.

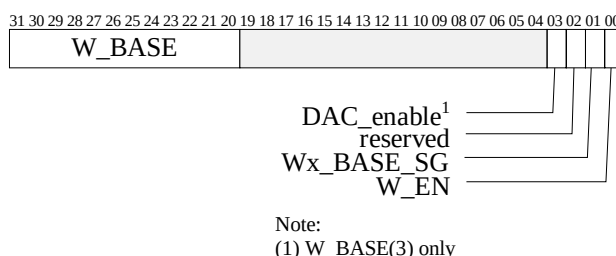


Figure 4-44 W_x_BASE, x=0-3

Determining a Hit in the Target Window:

The incoming PCI address bits <31:20> are compared with the each of the four Window Base registers; where the W_MASK register determines which bits are involved in the comparison. The target window is “Hit” when the masked addresses match a valid Window Base register. (Also see the section in the PCI to Memory chapter on PC Compatibility address windows).

When DAC_enable is set in Window 3 then W_DAC base register is used to compare PCI address <39:32> of a DAC cycle.

Name	Extent	Access	Initial State	Firmware Initialized State	
W_EN	<0>	RW	0	tbd ⁴⁶	When clear, the PCI Target Window is disabled and will not respond to PCI initiated transfers. When set, the PCI Target Window is enabled and will respond to PCI transactions that hit in the address range of the target window
Wx_BASE_SG	<1>	RW	0	tbd ⁴⁶	When clear the PCI Target Window uses direct mapping to translate a PCI address to an MC bus address. When set the PCI Target Window uses scatter gather mapping to translate a PCI address to a physical memory address.
reserved	<2>	RO		0	
DAC_enable only in W_BASE[3]	<3>	RW	0	tbd ⁴⁶	When set the address in W_DAC register is compared against the PCI address <39:32> for a PCI DAC cycle. If this compare hit, and the 32 bit portion of the PCI address hit, then a DAC cycle hit occurs.
reserved	<19:4>	RO	0	0	
W_BASE	<31:20>	RW	0	tbd ⁴⁶	Specifies the PCI Base address of the PCI Target Window and is used to determine a hit in the target window.

⁴⁶ See Chapter 3

4.9.4 Window Mask (W_x_MASK, x=0-3 - Offset = 1440, 1540, 1640, 1740)

The window Mask register provides a mask corresponding to bits <31:20> of an incoming PCI address. The size of each window can be programmed to be from 1 MByte to 4 GBytes in powers of two by masking bits of the incoming PCI address via the Window Mask register as shown in Table 5-40.

There are 4 Window Mask registers: W_MASK(0), W_MASK(1), W_MASK(2), and W_MASK(3). The W_MASK registers should not be modified unless software insures that the no PCI traffic is targeted for the window being modified.

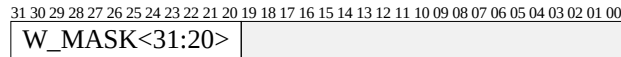


Figure 4-45 W_x_MASK, (x=0-3)

Determining a Hit in the Target Window:

The incoming PCI address bits <31:20> are compared with the each of the Window Base registers; where the W_MASK register determines which bits are involved in the comparison. The target window is “Hit” when the masked addresses match a valid Window Base register.

Name	Extent	Access	Initial State	Firmware Initialized State																															
reserved	<19:0>	RO	0	0																															
W_MASK	<31:20>	RW	0	tbd ⁴⁶	Specifies the size of the PCI Target Window (see table below) and used to mask out address bits not used when determining a PCI target window “HIT”. <table><tr><th>W_MASK<31:20></th><th>Size of Window</th></tr><tr><td>0000 0000 0000</td><td>1 Megabyte</td></tr><tr><td>0000 0000 0001</td><td>2 Megabytes</td></tr><tr><td>0000 0000 0011</td><td>4 Megabytes</td></tr><tr><td>0000 0000 0111</td><td>8 Megabytes</td></tr><tr><td>0000 0000 1111</td><td>16 Megabytes</td></tr><tr><td>0000 0001 1111</td><td>32 Megabytes</td></tr><tr><td>0000 0011 1111</td><td>64 Megabytes</td></tr><tr><td>0000 0111 1111</td><td>128 Megabytes</td></tr><tr><td>0000 1111 1111</td><td>256 Megabytes</td></tr><tr><td>0001 1111 1111</td><td>512 Megabyte</td></tr><tr><td>0011 1111 1111</td><td>1 Gigabytes</td></tr><tr><td>0111 1111 1111</td><td>2 Gigabytes</td></tr><tr><td>1111 1111 1111</td><td>4 Gigabytes</td></tr><tr><td>otherwise</td><td>not supported</td></tr></table>	W_MASK<31:20>	Size of Window	0000 0000 0000	1 Megabyte	0000 0000 0001	2 Megabytes	0000 0000 0011	4 Megabytes	0000 0000 0111	8 Megabytes	0000 0000 1111	16 Megabytes	0000 0001 1111	32 Megabytes	0000 0011 1111	64 Megabytes	0000 0111 1111	128 Megabytes	0000 1111 1111	256 Megabytes	0001 1111 1111	512 Megabyte	0011 1111 1111	1 Gigabytes	0111 1111 1111	2 Gigabytes	1111 1111 1111	4 Gigabytes	otherwise	not supported
W_MASK<31:20>	Size of Window																																		
0000 0000 0000	1 Megabyte																																		
0000 0000 0001	2 Megabytes																																		
0000 0000 0011	4 Megabytes																																		
0000 0000 0111	8 Megabytes																																		
0000 0000 1111	16 Megabytes																																		
0000 0001 1111	32 Megabytes																																		
0000 0011 1111	64 Megabytes																																		
0000 0111 1111	128 Megabytes																																		
0000 1111 1111	256 Megabytes																																		
0001 1111 1111	512 Megabyte																																		
0011 1111 1111	1 Gigabytes																																		
0111 1111 1111	2 Gigabytes																																		
1111 1111 1111	4 Gigabytes																																		
otherwise	not supported																																		

4.9.5 Translated Base (Tx_BASE, x=0-3 - Offset = 1480, 1580, 1680, 1780)

The Translated Base register is used to map PCI addresses into memory. There are 4 Translated Base registers: TBR0, TBR1, TBR2, and TBR3, one for each window. If the Scatter/Gather bit of the Window Base Register is set, then the Translated Base register provides the base address of the Scatter/Gather map for this window. If the Scatter/Gather bit is clear, the Translated Base register provides the base physical address of this window.

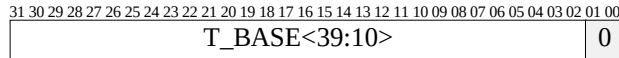


Figure 4-46 Tx_BASE, (x=0,3)

The TBR registers should not be modified unless software insures that the no PCI traffic is targeted for the window being modified.

Name	Extent	Access	Initial State	Firmware Initialized State	
reserved	<1:0>	RO		0	
T_BASE<39:10>	<31:2>	RW	0	tbd ⁴⁶	If Scatter Gather Mapping is disabled, T_BASE <39:20> specifies the base CPU address of the translated PCI address for the PCI Target Window T_BASE<19:10> MBZ. (refer to Table 4-20). If Scatter Gather Mapping is enabled, T_BASE <39:10> specifies the base CPU address for the Scatter Gather Map Table for the PCI Target Window (refer to Table 4-21).

Table 4-19 Translated Base Registers (Tx_BASE, x=0-3)

The field W_MASK<31:20> sets the size of the PCI Target window and the number of 8K pages that fall into the window. Every 8K page requires one 8 byte scatter gather map entry. Table 4-21 shows the relationship of W_MASK to the size of the scatter gather map in memory.

$$\begin{aligned}
 &(\text{size of the window in bytes}) / (8 \text{ Kbytes}) = \text{number of entries required} \\
 &(\text{number of entries}) * (8 \text{ bytes}) = \text{size of the scatter gather table}
 \end{aligned}$$

The quadword address used to index into the table is formed from concatenating the appropriate T_BASE and PCI address bits based on the size of the scatter gather map. The PCI address forms the index into the table while the T_BASE forms the naturally aligned base of the table.

For example, for a mask of 0000 0000 0000, there are 128 entries in the scatter gather table and the table size is 1 Kbytes. Entries are quadwords so the lower three bits <2:0> of the address are always zero. Now, mask off PCI bits <31:20> (because of the W_MASK). Then use PCI address <19:13> (7 bits, $2^7 = 128$ entries in the table) as the table index. Use the T_BASE <39:10> to get the other bits of the 40 bit address.

W_MASK<31:20>	Translated Address<39:0>	Unused Translated Base Register Bits
0000 0000 0000	TBASE<39:20>:PCI_AD_H<19:0>	T_BASE<19:10>
0000 0000 0001	TBASE<39:21>:PCI_AD_H<20:0>	T_BASE<20:10>
0000 0000 0011	TBASE<39:22>:PCI_AD_H<21:0>	T_BASE<21:10>
0000 0000 0111	TBASE<39:23>:PCI_AD_H<22:0>	T_BASE<22:10>
0000 0000 1111	TBASE<39:24>:PCI_AD_H<23:0>	T_BASE<23:10>
0000 0001 1111	TBASE<39:25>:PCI_AD_H<24:0>	T_BASE<24:10>
0000 0011 1111	TBASE<39:26>:PCI_AD_H<25:0>	T_BASE<25:10>
0000 0111 1111	TBASE<39:27>:PCI_AD_H<26:0>	T_BASE<26:10>
0000 1111 1111	TBASE<39:28>:PCI_AD_H<27:0>	T_BASE<27:10>
0001 1111 1111	TBASE<39:29>:PCI_AD_H<28:0>	T_BASE<28:10>
0011 1111 1111	TBASE<39:30>:PCI_AD_H<29:0>	T_BASE<29:10>
0111 1111 1111	TBASE<39:31>:PCI_AD_H<30:0>	T_BASE<30:10>
1111 1111 1111	TBASE<39:32>:PCI_AD_H<31:0>	T_BASE<31:10>

Note: (1) unused Translation Base must be zero for correct operation

(2) For peer to peer DMA T_BASE<39> must be set.

Table 4-20 PCI Address Translation - Scatter Gather Mapping Disabled

W_MASK<31:20>	SG Map Table Size	Scatter Gather Map Address<38:0> (used to index S/G Table in memory)
0000 0000 0000	1 Kbytes	TBASE<38:10>:PCI_AD_H<19:13>:000
0000 0000 0001	2 Kbytes	TBASE<38:11>:PCI_AD_H<20:13>:000
0000 0000 0011	4 Kbytes	TBASE<38:12>:PCI_AD_H<21:13>:000
0000 0000 0111	8 Kbytes	TBASE<38:13>:PCI_AD_H<22:13>:000
0000 0000 1111	16 Kbytes	TBASE<38:14>:PCI_AD_H<23:13>:000
0000 0001 1111	32 Kbytes	TBASE<38:15>:PCI_AD_H<24:13>:000
0000 0011 1111	64 Kbytes	TBASE<38:16>:PCI_AD_H<25:13>:000
0000 0111 1111	128 Kbytes	TBASE<38:17>:PCI_AD_H<26:13>:000
0000 1111 1111	256 Kbytes	TBASE<38:18>:PCI_AD_H<27:13>:000
0001 1111 1111	512 Kbytes	TBASE<38:19>:PCI_AD_H<28:13>:000
0011 1111 1111	1 MBytes	TBASE<38:20>:PCI_AD_H<29:13>:000
0111 1111 1111	2 MBytes	TBASE<38:21>:PCI_AD_H<30:13>:000
1111 1111 1111	4 MBytes	TBASE<38:22>:PCI_AD_H<31:13>:000

Note: TBASE<39> is "don't care"

Table 4-21 PCI Address Translation - Scatter Gather Mapping Enabled

4.9.6 Window DAC Base Register (W_DAC - Offset = 17C0)

The Window DAC Base register provides the <7:0> address bits for comparison against PCI address <39:32> during the DAC cycle. PCI address <63:40> has to be zero for a PCI window hit. The DAC BASE register is used in conjunction with the W_BASE register. For more details refer to the PCI Addressing Chapter.

The DAC BASE register is only applicable to window 3 and only if enabled by the DAC_enable bit in W_BASE [3].

Determining a Hit in the Target Window:

The Target Window is a “Hit” when the following is satisfied:

- The incoming PCI address bits <31:20> matches one of the four Window Base registers; the W_MASK register determines which bits are involved in the comparison.
- PCI Address <63:40> is zero
- PCI address matches window 3 and PCI address <39:32> match the DAC BASE register.

The Target Window is ‘Hit’ when the masked address match a valid Window Base register.

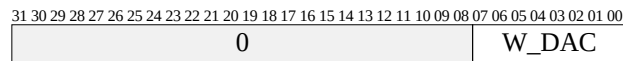


Figure 4-47 Window DAC Base Register (W_DAC)

Name	Extent	Access	Initial State	Firmware Initialized State	
W_DAC	<7:0>	RW	0	TBD	DAC_BASE specifies bits <39:32> of the PCI base address used to determine a hit in the target window for a DAC cycle.
reserved	<31:8>	RO	0	0	

4.10 Scatter/Gather Address Translation registers

4.10.1 Translation Buffer Tag (TB_TAGx, x=0-7 - Offset = 1800, 1840, 1880, 18C0, 1900, 1940, 1980, 19C0)

Software can write to these TB TAG entries, but they may be evicted by the hardware on a SG TLB miss.

Determining a Hit in the Translation buffer:

The incoming PCI address bits <31:15> are compared with each of the eight Translation Buffer Tag Registers. If there is a match, the corresponding Translation Buffer Page register group is indexed by PCI address bits <14:13>, and if it is valid then there is a Translation Buffer Hit.

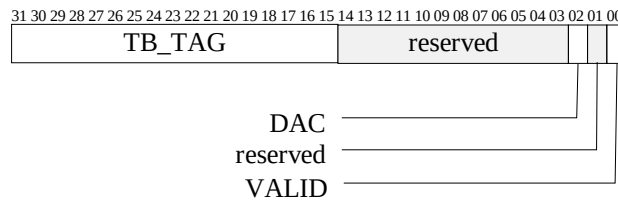


Figure 4-48 TB_TAG0 - TB_TAG3

Name	Extent	Access	Initial State	Firmware Initialized State	
VALID	<0>	RW	0	0	If VALID and CAP_CTRL[SG_TLB_EN] are set, then this entry will be used for address translation
reserved	<1>	RW	0	0	
DAC	<2>	RW	U	0	If set then this TAG entry corresponds to a 64-bit PCI address (DAC cycle); otherwise it belongs to a 32-bit PCI address (SAC cycle)
reserved	<14:3>	RW	0	0	
TB_TAG	<31:15>	RW	U	0	TB_TAG<31:15> is the TAG for each translation buffer entry

4.10.2 Translation Buffer Page Register (TBx_PAGEn - Offset = 2000:27C0)

There are thirty two translation buffer data registers, a group of four for each of the eight translation buffer entries. The TBx_PAGEn registers are automatically updated on a TLB miss (a group of four at a time) by the MC-PCI Bridge hardware. The replacement algorithm is Least Recently Used (LRU).

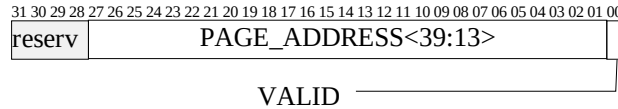


Figure 4-49 Translation Buffer Data Register TBx_PAGEn

Name	Extent	Access	Initial State	Firmware Initialized State	
VALID	<0>	RW	U	0	
PAGE_ADDRESS	<27:1>	RW	U	0	The PAGE_ADDRESS <27:1> forms Physical address <39:13>. PCI_AD_H<12:0> forms physical address <12:0>. For Rawhide address <39> may be set for peer to peer DMA Transactions.
reserved	<31:28>	RO	0	0	

Determining a Hit in the Translation Buffer:

The incoming PCI address bits <31:15> are compared with the each of the eight Translation Buffer Tag Registers. If there is a match, the corresponding Translation Buffer Page register group is indexed by PCI address bits <14:13> and if it is valid then there is a Translation Buffer Hit.

If the Address bits do not match the TAG, or the page entry is invalid then a TLB miss occurs. If the PTE fetched by the hardware TLB-miss handler is still invalid then a hard (RDS) error interrupt is asserted for a DMA write. See Hardware Exceptions and Interrupts chapter.

4.11 Memory Configuration Registers

There are no memory control registers on Rawhide. There are Memory Window registers on the MC-PCI bridge.

4.12 EV5 Configuration Registers - description

Detailed description of the EV5 registers are available in the EV5 spec. Initialization of EV5 specific registers is specified in section 4.3 EV5 Configuration Registers. The only Rawhide specific CPU configuration register is on the Cached CPU module and is specified in 4.3.1 Ponderosa Specific Registers.

4.13 Standard IO CSRs (EISA/PCI Bus Bridge ESC chip etc.)

There are CSRs at fixed addresses in the PCI-EISA bus bridge and CSRs whose location is defined by loading PCI-EISA bus bridge registers. GPCSR registers are initialized by firmware to define the offset and extent of registers implemented on the XBUS in PALs on the Saddle module. Once GPCSRs are initialized, the Flash ROM Control Register and Reset Register are found at offset 26 and 27 in PCI memory address space.

4.13.1 B3050-AA - Saddle Flash ROM

There are two blocks of PROM expected to be mapped in address space on this host bus bridge. AXP Firmware is discussed in this section. EISA Configuration NVRAM is at address offset 0x0800-0x08FF⁴⁷.

The Saddle Flash ROM storage is made up of two flash ROMs of 1 Megabyte storage each. The flash ROMs are referred to as primary and secondary with the secondary part optional to the design⁴⁸.

4.13.1.1 Write Protect & Fail-Safe loader Jumpers

The ROMs can be write protected by removing a jumper that disconnects the programming voltage. A second one megabyte part is included to support the WNT Alpha BIOS project. It is selected via register bit FROM_SEL. It is also write protected by removing the programming voltage jumper.

4.13.1.2 Addressing ROM

The ROM implementation is based on Lynx/Sable. ROM is used on PCI logical bus 0 to store the system console. It is implemented with 1Mbyte required and 1MB optional FLASH ROM. The FLASH ROM is logically divided into two "banks", with one bank containing the System Console and Fail Safe Loader and the other (alternate) bank containing the ARC (WNT) Firmware.

Each flash ROM is accessed as sixteen 64K Byte blocks of memory space in the address range F0000h to FFFFFh. The EISA address bits SA<15:0> are applied directly to the flash ROMs. Flash Control Register bits are used to supply the remaining address lines as FROM_ADR<2:0> and 64K_SEL.

The FROM Control Register and the Reset Register are accessed using the General Purpose Chip Select 1 control signals from the EISA bridge chip set. The ESC bridge chip configuration registers that define the address range for GPCSR1 control signals should be set to PCI address 26 with a depth of 2. This places the Flash ROM Control Register at the address defined for FRCR and the Reset Register at the address defined for RESET.

PCI Memory Address Offset	FRCR[7] ⁴⁹ : ADR<19>	FRCR[6]: ADR<18>	FRCR[5]: ADR<17>	FRCR[4]: 64K_SEL	FROM_SEL	Selection
0F FFFFh 0F 0000h	x	x	x	x	0	SRM Console, Extended SROM, VMS & UNIX PAL code, & Fail Safe Loader
	x	x	x	x	1	ALPHA BIOS (16 segments)

Table 4-22 Flash ROM Control Register (FRCR) Bit Selection of Flash ROM space

This is how the 2 MByte FLASH looks like on Rawhide. The BIOS memory address range for the 64KB section is from offset 000F0000h to offset 000FFFFFh relative to the PCI Memory window.

⁴⁷ same as specified in the PCI-EISA bus bridge chip set data books and Sable/Gamma/Lynx.

⁴⁸ The second 1MB of FROM is now a WNT requirement.

⁴⁹ (or Jumper XX)

4.13.2 Reset Register: (PCI IO Address Offset = 26)

The Reset Register controls and monitors some aspects of the system reset logic. The register contains a writable bit that will reset the entire system box and four read-only status bits that indicate the source of system resets generated by the Saddle module. This register can be read by firmware at startup time to determine the cause of the most recent firmware restart. Only one of the status bits should be set at any time thereby reporting the source of the last system reset.

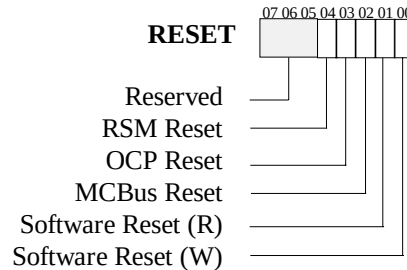


Figure 4-50 Reset Register

Name	Extent	Access	
Software Reset Request	<0>	WO	When software write this bit a system-wide reset request is sent to the Power Control Module.
Software Reset	<1>	RO	The most recent reset was the result of Software writing Reset Register bit 0.
Mobcaps Reset	<2>	RO	The most recent Reset was received from the MCBus interface chip(CAP Chip) via the PCI0 reset signal without a request generated by the Saddle module. This could be due to initial power up, a reset directed to PCI0 of the Saddle module by a CAP register write or the assertion of MCBus reset by any agent capable of driving it.
OCP Reset	<3>	RO	The most recent reset was the result of the Operator Control Panel reset switch being actuated.
RSM Reset	<4>	RO	The most recent reset was generated by a Remote System Management Module residing in a local EISA slot.
reserved	<7:5>		

4.13.3 Flash ROM Control Register (PCI IO Address Offset = 27)

This register is used to control the Flash ROM.

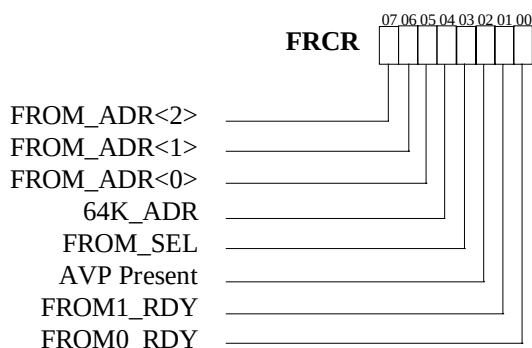


Figure 4-51 Flash ROM Control Registers

Name	Extent	Access	Initial State	Firmware Initialized State	
FROM0_RDY	<0>	RO			When set this indicates that the primary flash ROM is ready to accept commands.
FROM1_RDY	<1>	RO			When set this indicates that the secondary flash ROM is ready to accept commands.
AVP Present	<2>	RO			When set this bit indicates that the programming voltage is present at the Flash ROMs, implying that the programming jumper is inserted.
FROM_SEL	<3>	RW			This bit selects the secondary Flash ROM. When set the second 1MB FROM is selected.
64K_ADR	<4>	RW			These four bit are used as the high order address bits to the flash ROMs. They select one of sixteen 64K byte address ranges in the flash ROM.
FROM_ADR<2:0>	<7:5>	RW			

4.13.4 Standard IO addressing

Table 4-23 STANDARD I/O ADDRESSING (Sparse IO Space)

Register Name or Address Space	Address offset into Sparse IO Space	PCI IO Address	Length	Attributes
DMA1 CH0 Base and Current Address	.0000.0000	0000	1 byte	R/W
DMA1 CH0 Base and Current Count	.0000.0020	0001	1 byte	R/W
DMA1 CH1 Base and Current Address	.0000.0040	0002	1 byte	R/W
DMA1 CH1 Base and Current Count	.0000.0060	0003	1 byte	R/W
DMA1 CH2 Base and Current Address	.0000.0080	0004	1 byte	R/W
DMA1 CH2 Base and Current Count	.0000.00A0	0005	1 byte	R/W
DMA1 CH3 Base and Current Address	.0000.00C0	0006	1 byte	R/W
DMA1 CH3 Base and Current Count	.0000.00E0	0007	1 byte	R/W
DMA1 Status(r) Command(w) register	.0000.0100	0008	1 byte	R/W
DMA1 Write Request register	.0000.0120	0009	1 byte	WO
DMA1 Write Single Mask Bit	.0000.0140	000A	1 byte	WO
DMA1 Write Mode register	.0000.0160	000B	1 byte	WO
DMA1 Clear Byte Pointer	.0000.0180	000C	1 byte	WO
DMA1 Master Clear	.0000.01A0	000D	1 byte	WO
DMA1 Clear Mask register	.0000.01C0	000E	1 byte	WO
DMA1 Read/Write All Mask Register Bits	.0000.01E0	000F	1 byte	R/W
Reserved		0010-001F ⁵⁰		
INT 1 Control	.0000.0400	0020	1 byte	R/W
INT 1 Mask	.0000.0420	0021	1 byte	R/W
Configuration Address Index register	.0000.0440	0022	1 byte	R/W
Configuration Data Index register	.0000.0460	0023	1 byte	R/W
Spare		0024-25		
Reset Register	.0000.04C0	0026	1 byte	R/W
FRCR - Flash ROM Control Register	.0000.04E0	0027	1 byte	R/W
Aliased		0028-3F		
Timer 1 - Counter 0 System Clock	.0000.0800	0040	1 byte	R/W
Timer 1 - Counter 1 Refresh Request	.0000.0820	0041	1 byte	R/W
Timer 1 Counter 2 Speaker Tone	.0000.0840	0042	1 byte	R/W
Timer 1 Command Mode register	.0000.0860	0043	1 byte	WO
Spare		0044-0047		
Timer 2 - Counter 0 Fail-Safe Timer	.0000.0880	0048	1 byte	R/W
Timer 2 - Counter 1 RESERVED	.0000.08A0	0049	1 byte	R/W
Timer 2 - Counter 2 CPU Speed Control	.0000.08C0	004A	1 byte	R/W
Timer 2 Command Mode register	.0000.08E0	004B	1 byte	WO
Spare		004C-004F		
Reserved		0050-005B ⁵⁰		
Spare		005C-005F		
Keyboard Controller: Data	.0000.0C00	0060	1 byte	R/W
NMI Status and Control	.0000.0C20	0061	1 byte	R/W
Spare		0062		
Reserved		0063 ⁵⁰		
Keyboard Controller: CMD/STAT	.0000.0C80	0064	1 byte	R/W

⁵⁰ Lost to “don't care”s in address decode in ESC chip

Control and Status Registers

Register Name or Address Space	Address offset into Sparse IO Space	PCI IO Address	Length	Attributes
Reserved		0065 ⁵⁰		
Spare		0066		
Reserved		0067 ⁵⁰		
Spare		0068-006F		
Real Time Clock & NMI Enable Register ⁵¹	.0000.0E00	0070	1 byte	WO
Real Time Clock Control	.0000.0E20	0071	1 byte	
Reserved		0072 ⁵⁰		
Spare		0073		
Reserved		0074 ⁵⁰		
Spare		0075		
Reserved		0076 ⁵⁰		
Spare		0077-007F		
DMA Page register - reserved	.0000.1000	0080	1 byte	R/W
DMA Channel 2 Page register	.0000.1020	0081	1 byte	R/W
DMA Channel 3 Page register	.0000.1040	0082	1 byte	R/W
DMA Channel 1 Page register	.0000.1060	0083	1 byte	R/W
DMA Page register - reserved	.0000.1080	0084	1 byte	R/W
DMA Page register - reserved	.0000.10A0	0085	1 byte	R/W
DMA Page register - reserved	.0000.10C0	0086	1 byte	R/W
DMA Channel 0 Page register	.0000.10E0	0087	1 byte	R/W
DMA Page register - reserved	.0000.1100	0088	1 byte	R/W
DMA Channel 6 Page register	.0000.1120	0089	1 byte	R/W
DMA Channel 7 Page register	.0000.1140	008A	1 byte	R/W
DMA Channel 5 Page register	.0000.1160	008B	1 byte	R/W
DMA Page register - reserved	.0000.1180	008C	1 byte	R/W
DMA Page register - reserved	.0000.11A0	008D	1 byte	R/W
DMA Page register - reserved	.0000.11C0	008E	1 byte	R/W
DMA Refresh Page register	.0000.11E0	008F	1 byte	R/W
Reserved		0090-0091 ⁵⁰		
System Control Port	.0000.1240	0092	1 byte	R/W
Reserved		0093-00BF ⁵⁰		
INT 2 Control	.0000.1400	A0	1 byte	R/W
INT 2 Mask	.0000.1420	A1	1 byte	R/W
Reserved		00A2-00BF ^{50, 52}		
DMA2 CH0 Base and Current Address	.0000.1800	00C0	1 byte	R/W
Reserved		00C1 ⁵⁰		
DMA2 CH0 Base and Current Count	.0000.1840	00C2	1 byte	R/W
Reserved		00C3 ⁵⁰		
DMA2 CH1 Base and Current Address	.0000.1880	00C4	1 byte	R/W
Reserved		00C5 ⁵⁰		
DMA2 CH1 Base and Current Count	.0000.18A0	00C6	1 byte	R/W
Reserved		00C7 ⁵⁰		
DMA2 CH2 Base and Current Address	.0000.18C0	00C8	1 byte	R/W
Reserved		00C9 ⁵⁰		

⁵¹ Care should be taken when writing to this address. Integrity of bit 7 should be maintained when writing the Real Time Clock Address.

⁵² Due to decoding, 2 are spare, 2 are reserved, etc. See ESC chip specification.

Control and Status Registers

Register Name or Address Space	Address offset into Sparse IO Space	PCI IO Address	Length	Attributes
DMA2 CH2 Base and Current Count	.0000.1940	00CA	1 byte	R/W
Reserved		00CB ⁵⁰		
DMA2 CH3 Base and Current Address	.0000.1980	00CC	1 byte	R/W
Reserved		00CD ⁵⁰		
DMA2 CH3 Base and Current Count	.0000.19C0	00CE	1 byte	R/W
Reserved		00CF ⁵⁰		
DMA2 Status(r) Command(w) register	.0000.1A00	00D0	1 byte	WO
Reserved		00D1 ⁵⁰		
DMA2 Write Request register	.0000.1A40	00D2	1 byte	WO
Reserved		00D3 ⁵⁰		
DMA2 Write Single Mask Bit	.0000.1A80	00D4	1 byte	WO
Reserved		00D5 ⁵⁰		
DMA2 Write Mode register	.0000.1AC0	00D6	1 byte	WO
Reserved		00D7 ⁵⁰		
DMA2 Clear Byte Pointer	.0000.1B00	00D8	1 byte	WO
Reserved		00D9 ⁵⁰		
DMA2 Master Clear	.0000.1B40	00DA	1 byte	WO
Reserved		00DB ⁵⁰		
DMA2 Clear Mask register	.0000.1B80	00DC	1 byte	WO
Reserved		00DD ⁵⁰		
DMA2 Read/Write All Mask Register Bits	.0000.1BC0	00DE	1 byte	R/W
Reserved		00DF ⁵⁰		
Spare		00E0-EF		
Reset IRQ13	.0000.1E00	00F0	1 byte	WO
Spare		00F1-02F7		
Serial Port 1	.0000.5F00- .0000.5FE0	2F8-2FF ⁵³		R/W
Spare		300-356		
Secondary Floppy Disk Controller	.0000.6AE0- .0000.6E00	0357 370 ⁵⁴		WO
Spare		371-397		
Combo Chip Configuration	.0000.7300	0398	1 byte	R/W
Combo Chip Configuration	.0000.7320	0399	1 byte	R/W
Spare		39A-3BB		
Parallel Port	.0000.7780- .0000.77C0	03BC-03BE ⁵⁵		R/W
Spare		03BF-03EF		
Primary Floppy Disk	.0000.7E00- .0000.7EE0	3F0-3F7 ⁵⁴		R/W
Serial Port 0	.0000.7F00- .0000.7FE0	3F8-3FF ⁵⁶		R/W

⁵³ Accesses to these addresses produce chip selects on the XBUS for Serial Port 1.⁵⁴ Accesses to these addresses produce chip selects on the XBUS for the Floppy Disk Controllers.⁵⁵ Accesses to these addresses produce chip selects on the XBUS for the Parallel Port.⁵⁶ Accesses to these addresses produce chip selects on the XBUS for Serial Port 0.

Control and Status Registers

Register Name or Address Space	Address offset into Sparse IO Space	PCI IO Address	Length	Attributes
Reserved	.000.0000	0400	1 byte	R/W
DMA1 CH0 Base /Current Count	.0000.8020	0401	1 byte	R/W
Reserved	.0000.8040	0402	1 byte	R/W
DMA1 CH1 Base/Current Count	.0000.8060	0403	1 byte	R/W
Reserved	.0000.8080	0404	1 byte	R/W
DMA1 CH2 Base/Current Count	.0000.80A0	0405	1 byte	R/W
Reserved	.0000.80C0	0406	1 byte	R/W
DMA1 CH3 Base/Current Count	.0000.80E0	0407	1 byte	R/W
Reserved	.0000.8100	0408	1 byte	R/W
Reserved	.0000.8120	0409	1 byte	R/W
DMA Chaining Mode Status/Interrupt Pending	.0000.8140	040A	1 byte	R/W
DMA1 Extended Mode register	.0000.8160	040B	1 byte	WO
Chaining Buffer Control Register	.0000.8180	040C	1 byte	WO
Reserved	.0000.81A0	040D	1 byte	R/W
Reserved	.0000.81C0	040E	1 byte	R/W
Reserved	.0000.81E0	040F	1 byte	R/W
DMA CH0 S-G Command Register	.0000.8200	0410 ⁵⁷	1 byte	WO
DMA CH1 S-G Command Register	.0000.8220	0411 ⁵⁷	1 byte	WO
DMA CH2 S-G Command Register	.0000.8240	0412 ⁵⁷	1 byte	WO
DMA CH3 S-G Command Register	.0000.8260	0413 ⁵⁷	1 byte	WO
DMA CH5 S-G Command Register	.0000.82A0	0415 ⁵⁷	1 byte	WO
DMA CH6 S-G Command Register	.0000.82C0	0416 ⁵⁷	1 byte	WO
DMA CH7 S-G Command Register	.0000.82E0	0417 ⁵⁷	1 byte	WO
DMA CH0 S-G Status register	.0000.8300	0418 ⁵⁷	1 byte	WO
DMA CH1 S-G Status register	.0000.8320	0419 ⁵⁷	1 byte	WO
DMA CH2 S-G Status register	.0000.8340	041A ⁵⁷	1 byte	WO
DMA CH3 S-G Status register	.0000.8360	041B ⁵⁷	1 byte	WO
DMA CH5 S-G Status register	.0000.83A0	041D ⁵⁷	1 byte	WO
DMA CH6 S-G Status register	.0000.83C0	041E ⁵⁷	1 byte	WO
DMA CH7 S-G Status register	.0000.83E0	041F ⁵⁷	1 byte	WO
DMA CH0 S-G Descriptor Pointer register	.0000.8400	0420 ⁵⁷	1 byte	RO
DMA CH0 S-G Descriptor Pointer register	.0000.8420	0421 ⁵⁷	1 byte	RO
DMA CH0 S-G Descriptor Pointer register	.0000.8440	0422 ⁵⁷	1 byte	RO
DMA CH0 S-G Descriptor Pointer register	.0000.8460	0423 ⁵⁷	1 byte	RO
DMA CH1 S-G Descriptor Pointer register	.0000.8480	0424 ⁵⁷	1 byte	RO
DMA CH1 S-G Descriptor Pointer register	.0000.84A0	0425 ⁵⁷	1 byte	RO
DMA CH1 S-G Descriptor Pointer register	.0000.84C0	0426 ⁵⁷	1 byte	RO
DMA CH1 S-G Descriptor Pointer register	.0000.84E0	0427 ⁵⁷	1 byte	RO
DMA CH2 S-G Descriptor Pointer register	.0000.8500	0428 ⁵⁷	1 byte	RO
DMA CH2 S-G Descriptor Pointer register	.0000.8520	0429 ⁵⁷	1 byte	RO
DMA CH2 S-G Descriptor Pointer register	.0000.8540	042A ⁵⁷	1 byte	RO
DMA CH2 S-G Descriptor Pointer register	.0000.8560	042B ⁵⁷	1 byte	RO
DMA CH3 S-G Descriptor Pointer register	.0000.8580	042C ⁵⁷	1 byte	RO
DMA CH3 S-G Descriptor Pointer register	.0000.85A0	042D ⁵⁷	1 byte	RO
DMA CH3 S-G Descriptor Pointer register	.0000.85C0	042E ⁵⁷	1 byte	RO
DMA CH3 S-G Descriptor Pointer register	.0000.85E0	042F ⁵⁷	1 byte	RO

⁵⁷ The Scatter-Gather DMA registers are relocatable. The default I/O address for the Scatter-Gather Registers on Power-up are 0410-043F. These registers can be mapped in the I/O range xx10-xx3F.

Control and Status Registers

Register Name or Address Space	Address offset into Sparse IO Space	PCI IO Address	Length	Attributes
Spare		0430-0433		
DMA CH5 S-G Descriptor Pointer register	.0000.8680	0434 ⁵⁷	1 byte	RO
DMA CH5 S-G Descriptor Pointer register	.0000.86A0	0435 ⁵⁷	1 byte	RO
DMA CH5 S-G Descriptor Pointer register	.0000.86C0	0436 ⁵⁷	1 byte	RO
DMA CH5 S-G Descriptor Pointer register	.0000.86E0	0437 ⁵⁷	1 byte	RO
DMA CH6 S-G Descriptor Pointer register	.0000.8700	0438 ⁵⁷	1 byte	RO
DMA CH6 S-G Descriptor Pointer register	.0000.8720	0439 ⁵⁷	1 byte	RO
DMA CH6 S-G Descriptor Pointer register	.0000.8740	043A ⁵⁷	1 byte	RO
DMA CH6 S-G Descriptor Pointer register	.0000.8760	043B ⁵⁷	1 byte	RO
DMA CH7 S-G Descriptor Pointer register	.0000.8780	043C ⁵⁷	1 byte	RO
DMA CH7 S-G Descriptor Pointer register	.0000.87A0	043D ⁵⁷	1 byte	RO
DMA CH7 S-G Descriptor Pointer register	.0000.87C0	043E ⁵⁷	1 byte	RO
DMA CH7 S-G Descriptor Pointer register	.0000.87E0	043F ⁵⁷	1 byte	RO
Spare		0440-0460		
Extended NMI and Reset Control	.0000.8C20	0461	1 byte	R/W
NMI I/O Interrupt port	.0000.8C40	0462	1 byte	R/W
Spare	.0000.8C60	0463		
Last EISA Bus master granted (L)	.0000.8C80	0464	1 byte	RO
Spare		0465-047F		
Reserved	.0000.9000	0480	1 byte	R/W
DMA CH2 High Page register	.0000.9020	0481	1 byte	R/W
DMA CH3 High Page register	.0000.9040	0482	1 byte	R/W
DMA CH1 High Page register	.0000.9060	0483	1 byte	R/W
Reserved	.0000.9080	0484	1 byte	R/W
Reserved	.0000.90A0	0485	1 byte	R/W
Reserved	.0000.90C0	0486	1 byte	R/W
DMA CH0 High Page register	.0000.90E0	0487	1 byte	R/W
Reserved	.0000.9100	0488	1 byte	R/W
DMA CH6 High Page register	.0000.9120	0489	1 byte	R/W
DMA CH7 High Page register	.0000.9140	048A	1 byte	R/W
DMA CH5 High Page register	.0000.9160	048B	1 byte	R/W
Reserved	.0000.9180	048C	1 byte	R/W
Reserved	.0000.91A0	048D	1 byte	R/W
Reserved	.0000.91C0	048E	1 byte	R/W
DMA High Page Refresh register	.0000.91D0	048F	1 byte	R/W
Spare		0490-4C1		
Reserved	.0000.9840	04C2	1 byte	R/W
Spare		04C3-4C5		
DMA CH5 High Base & Current Count register	.0000.98C0	04C6	1 byte	R/W
Spare		04C5-04C9		
DMA CH5 High Base & Current Count register	.0000.9940	04CA	1 byte	R/W
Spare		04CB-04CD		
DMA CH5 High Base & Current Count register	.0000.99C0	04CE	1 byte	R/W
Spare		04CF		
INT-1 edge/Level Control register	.0000.9A00	04D0	1 byte	R/W
INT-2 edge/Level Control register	.0000.9A20	04D1	1 byte	R/W

Control and Status Registers

Register Name or Address Space	Address offset into Sparse IO Space	PCI IO Address	Length	Attributes
Reserved	.0000.9A40	04D2	1 byte	R/W
Reserved	.0000.9A60	04D3	1 byte	R/W
DMA2 Chaining Mode	.0000.9A80	04D4	1 byte	R/W
Reserved	.0000.9AA0	04D5	1 byte	R/W
DMA2 Extended Mode Register	.0000.9AC0	04D6	1 byte	WO
Reserved	.0000.9AE0	04D7	1 byte	R/W
Reserved	.0000.9B00	04D8	1 byte	R/W
Reserved	.0000.9B20	04D9	1 byte	R/W
Reserved	.0000.9B40	04DA	1 byte	R/W
Reserved	.0000.9B60	04DB	1 byte	R/W
Reserved	.0000.9B80	04DC	1 byte	R/W
Reserved	.0000.9BA0	04DD	1 byte	R/W
Reserved	.0000.9BC0	04DE	1 byte	R/W
Reserved	.0000.9BE0	04DF	1 byte	R/W
DMA CH0 Stop Register Bits [7:2]	.0000.9C00	04E0	1 byte	R/W
DMA CH0 Stop Register Bits [15:8]	.0000.9C20	04E1	1 byte	R/W
DMA CH0 Stop Register Bits [23:16]	.0000.9C40	04E2	1 byte	R/W
Reserved	.0000.9C60	04E3	1 byte	R/W
DMA CH1 Stop Register Bits [7:2]	.0000.9C80	04E4	1 byte	R/W
DMA CH1 Stop Register Bits [15:8]	.0000.9CA0	04E5	1 byte	R/W
DMA CH1 Stop Register Bits [23:16]	.0000.9CC0	04E6	1 byte	R/W
Reserved	.0000.9CE0	04E7	1 byte	R/W
DMA CH2 Stop Register Bits [7:2]	.0000.9D00	04E8	1 byte	R/W
DMA CH2 Stop Register Bits [15:8]	.0000.9D20	04E9	1 byte	R/W
DMA CH2 Stop Register Bits [23:16]	.0000.9D40	04EA	1 byte	R/W
Reserved	.0000.9D60	04EB	1 byte	R/W
DMA CH3 Stop Register Bits [7:2]	.0000.9D80	04EC	1 byte	R/W
DMA CH3 Stop Register Bits [15:8]	.0000.9DA0	04ED	1 byte	R/W
DMA CH3 Stop Register Bits [23:16]	.0000.9DC0	04EE	1 byte	R/W
Reserved	.0000.9DE0	04EF	1 byte	R/W
Reserved	.0000.9E00	04F0	1 byte	R/W
Reserved	.0000.9E20	04F1	1 byte	R/W
Reserved	.0000.9E40	04F2	1 byte	R/W
Reserved	.0000.9E60	04F3	1 byte	R/W
DMA CH5 Stop Register Bits [7:2]	.0000.9E80	04F4	1 byte	R/W
DMA CH5 Stop Register Bits [15:8]	.0000.9EA0	04F5	1 byte	R/W
DMA CH5 Stop Register Bits [23:16]	.0000.9EC0	04F6	1 byte	R/W
Reserved	.0000.9EE0	04F7	1 byte	R/W
DMA CH6 Stop Register Bits [7:2]	.0000.9F00	04F8	1 byte	R/W
DMA CH6 Stop Register Bits [15:8]	.0000.9F20	04F9	1 byte	R/W
DMA CH6 Stop Register Bits [23:16]	.0000.9F40	04FA	1 byte	R/W

Control and Status Registers

Register Name or Address Space	Address offset into Sparse IO Space	PCI IO Address	Length	Attributes
Reserved	.0000.9F60	04FB	1 byte	R/W
DMA CH7 Stop Register Bits [7:2]	.0000.9F80	04FC	1 byte	R/W
DMA CH7 Stop Register Bits [15:8]	.0000.9FA0	04FD	1 byte	R/W
DMA CH7 Stop Register Bits [23:16]	.0000.9FC0	04FE	1 byte	R/W
Reserved	.0000.9FE0	04FF	1 byte	R/W
Reserved	.0000.0000- .0000.A3E0	0500-051F	32 bytes	R/W
Spare		0520-052F		
I ² C	.0000.A600	0530	1 byte	R/W
I ² C	.0000.A620	0531	1 byte	R/W
Spare		0540-07FF		
NVRAM on Saddle module	.0001.0000- .0001.1FE0	0800-08FF		R/W
Spare		0900-0BFF		
Configuration RAM Page register	.0001.8000	0C00	1 byte	R/W
Spare		0C01-0FFF		
Flash ROM address window	.01E0.0000- .01FF.FFE0	000F.0000 - 000F.FFFF	64KB	R/W

5. Hardware Interrupts, and Exceptions

5.1 Overview

This chapter addresses:

- IO interrupts
- Error interrupts
- Interval Timer interrupts
- Inter-Processor interrupts
- HALT interrupts
- System bus and PCI bus errors
- Machine-check logout frame.

EV5 Detected errors are not discussed in detail; see the EV5 specification.

5.2 System Interrupts

In summary, PAL or HAL code converts all hardware signaled interrupt and exception events into machine checks.

5.2.1 IO Interrupts

PAL, HAL, and Diagnostic software developers should read Chapter 4 for a description of IO interrupt registers. All other programmers only need to know how interrupt vectors are assigned. Interrupt vector assignment is documented in Table 5-1, Table 5-2, and Table 5-3.

Each PCI bus has a four bit “PCI Offset” number⁵⁸ assigned to it at system configuration time. This number is used to identify an *interrupt target processor specific* offset into the interrupt information data structure. It is also used by PAL code in calculating an SCB vector. Interrupt vectors are assigned as follows:

SCB Offset	Interrupting Device
800	All EISA device interrupts are offset from this vector (10 times EISA vector)
900	All PCI device interrupts are offset from this vector. With one exception ⁵⁹ the offset is: $((\text{PCI Offset}) * 200) +$ $((\text{INT_REQ Bit Number} + 2) * 10)$

Table 5-1 SCB Vector Offset Calculation⁶⁰

Implicit in Table 5-2 are configuration limitations for operating systems that do not share interrupts. The Rawhide limitations are less onerous than on other platforms because there are only three 3 (E)ISA expansion slots. Rawhide has the same interrupt sharing driven (E)ISA configuration limitations as Mikasa because the assignment of interrupt lines is identical.

⁵⁸ Assigned by loading the Interrupt Target Address Register (INT_ADR<5:2> = PCI OFFSET field)

⁵⁹ The NCR810 SCSI interrupt line is a special case.

⁶⁰ The SCB vector is an OSF/VMS and SRM concept that is not valid for WNT. (WNT folks:want to provide text?)

Hardware Interrupts and Exceptions

Priority	Label	Controller	Available independent interrupt (not shared)	Internal/ External	Interrupt Vector	Interrupt Source (This assignment of interrupts to IRQ pins is identical to Mikasa)
1	IRQ 0	1		Internal	800	Internal Timer 1 Counter 0 out.
2	IRQ 1	1		External	810	Keyboard
3-10	IRQ 2	1		Internal		Interrupt from Controller 2
3	IRQ 8#	2		External	880	<i>Reserved (normally Real Time Clock)</i>
4	IRQ 9	2	X	External	890	EISA bus pin B04
5	IRQ 10	2	X	External	8A0	EISA bus pin D03
6	IRQ 11	2	X	External	8B0	EISA bus pin D04
7	IRQ 12	2		External	8C0	EISA bus pin D05 Mouse Intr (ABFULL)
8	IRQ 13	2		External	8D0	<i>Reserved (normally 487 coprocessor error)</i>
9	IRQ 14	2	X	External	8E0	EISA bus pin D07
10	IRQ 15	2	X	External	8F0	EISA bus pin D06
11	IRQ 3	1		External	830	EISA bus pin B25 (COM2)
12	IRQ 4	1		External	840	EISA bus pin B24 (COM1)
13	IRQ 5	1	X	External	850	EISA bus pin B23
14	IRQ 6	1		External	860	EISA bus pin B22 (Floppy Disk Contrl)
15	IRQ 7	1		External	870	EISA bus pin B21 (Parallel Port)

Table 5-2 (E)ISA Interrupt assignment

Hardware Interrupts and Exceptions

Target EV5 IRQ	INT_REQ Bit	Interrupt	Usage	PCI Offset	Vector
EV5 IRQ<1> Identified in INT_TARG_DEV_ID register		reserved	PCI Slot 0 & 1	0	900-970
	0	IntA	PCI Slot 2		980
	1	IntB			990
	2	IntC			9A0
	3	IntD			9B0
	4	IntA	PCI Slot 3		9C0
	5	IntB			9D0
	6	IntC			9E0
	7	IntD			9F0
	8	IntA	PCI Slot 4		A00
	9	IntB			A10
	10	IntC			A20
	11	IntD			A30
	12	IntA	PCI Slot 5		A40
	13	IntB			A50
	14	IntC			A60
	15	IntD			A70
	16 ⁶¹	Reserved (EISA_INT)			A80
	17	I ² C	8584 I ² C Ctrl		A90
	18	I ² C	I ² C Bus ⁶²		AA0
	19:20	unused			AB0-AF0
	21 ⁶¹	Reserved (NMI)	8259 NMI		MCHK 660
	22 ⁶³	SOFT ERR	CAP Chip		MCHK 620
	23 ⁶³	Reserved HARD ERR	CAP Chip		MCHK 660
EV5 IRQ<1> Identified in INT_TARG_DEV_ID register		reserved	PCI Slot 0	1	B00-B30
	16	SCSI	NCR810 SCSI ⁶⁴		B40
		reserved	PCI Slot 1: IntB, C, D		B50-B70
	0	IntA	PCI Slot 2		B80
	1	IntB			B90
	2	IntC			BA0
	3	IntD			BB0
	4	IntA	PCI Slot 3		BC0
	5	IntB			BD0
	6	IntC			BE0
	7	IntD			BF0
	8	IntA	PCI Slot 4		C00
	9	IntB			C10
	10	IntC			C20
	11	IntD			C30
	12	IntA	PCI Slot 5		C40
	13	IntB			C50
	14	IntC			C60
	15	IntD			C70
	21:17	unused			C80-CF0
	22 ⁶³	SOFT ERR	CAP Chip		MCHK 620
	23 ⁶³	Reserved HARD ERR	CAP Chip		MCHK 660

⁶¹ This INT_REQ line is tied to the EISA_INT signal from the 8259 internal to the PCI to EISA bus bridge.

⁶² Power Fail, Fan Fail etc. Are I²C bus interrupts.

⁶³ Converted into MCHK by PAL code

⁶⁴ PCI Logical Slot 1 - the NCR810 is addressed at configuration address 1.

Target EV5 IRQ	INT_REQ Bit	Interrupt	Usage	PCI Offset	Vector
EV5 IRQ<1> Identified in INT_TARG_DEV_ID register		reserved	PCI Slot 0 & 1	2	D00-D70
	0:15		PCI Slots 2-5		D80-E70
	16:21	Unused			E80-EF0
	22 ⁶³	SOFT ERR	CAP Chip		MCHK 620
	23 ⁶³	Reserved HARD ERR	CAP Chip		MCHK 660
(Repeat Bus 2)	.	.	.	3-	F00-10F0

Table 5-3 PCI Interrupt Vector Assignment

CUD Identified in INT0_TARG_DEV_ID: EV5 MCH_HLT_IRQ	HALT	OCP Logic
--	------	-----------

Table 5-4 Halt Interrupts (independent of IO interrupt mechanism)

5.2.2 EISA Interrupts

There is one EISA bus in a Rawhide system. The PCI-EISA bridge chip (ESC) set provides two cascaded 8259's and 14 interrupt lines. The PCI Interrupt-Acknowledge command is used to read the interrupt request vector out of the 8259s, and to flush the EISA-to-PCI write buffers in the (E)ISA bridge chip (used to ensure coherency between data written to memory, and the ensuing interrupt which signals the completion of the write).

The ESC chip provides four pins which can be used for routing PCI interrupts to the cascaded 8259s. This is not sufficient to support the 3 (E)ISA slots (11 interrupts) and the 20 interrupt lines (4 slots times four lines per slot, 4 spares for native PCI devices)⁶⁵. Consequently the 8259s in the ESC chip are used for the (E)ISA interrupts and independent **PCI Interrupt** logic is provided for the PCI interrupts. This is shown in Table 5-6. Note that the EISA interrupt signal passes through the PCI Interrupt logic block.

At the completion of an (E)ISA interrupt, software must reset the interrupt latch, wait at least 500ns⁶⁶ before issuing the EOI⁶⁷ command to the 8259s. Table 5-2 defines the EISA interrupt assignment.

5.2.3 EISA NMI Interrupt Assignment

Table 5-5 defines the hardware and software events which generate the ESC NMI Signal⁶⁸. All the NMI sources can be separately enabled/disabled in the ESC chip; and the NMI itself can be enabled/disabled. SERR# and PERR# must be disabled⁶⁹.

⁶⁵ Software prefers that PCI interrupts not be wire-ORed.

⁶⁶ this is an EISA spec. Requirement - it is a consequence of the weak pull-ups on the IRQ lines.

⁶⁷ What is this? I thought the INT ACK cycle on PCI did this??

⁶⁸ This is the standard EISA NMI signal.

⁶⁹ Is this true even if we restrict peer to peer DMA to exclude EISA devices? Yes - because of bugs in the Intel PCI-EISA bus bridge chips. *Since all instances of PERR# and SERR# will result in system crashing it may not matter.*

NMI Source	IO port to Enable/disable	Rawhide Usage
PCI PERR# PCI Data Parity Error	Port 0061h, bit<2>	DISABLE
PCI SERR# PCI Address parity error and other system errors.	Port 0040h, bit<3>	DISABLE ⁷⁰
IOCHK# EISA add-in board parity error or some other catastrophic error.	Port 0061h, bit<3>	Allowed
Fail-Safe timer time-out. This timer generates an NMI at regular intervals, preventing the system from being tied up in a tight loop. The operating system should reset timer 2 counter 0 at regular intervals to prevent it from causing an NMI under normal operating conditions.	Port 0461h, bit<2>	Allowed
EISA BUS time-out Set when a bus master uses the bus longer than 8 microseconds.	Port 0461h, bit<3>	Allowed
Software generated NMI Write to I/O port 0462h	Port 0461h, bit<1>	Allowed

Table 5-5 ESC NMI Generation

The majority of error conditions detected by the CAP chip generate a hard error 660 machine check interrupt and are covered later in this chapter.

The conditions that generate a soft error (620 MCHK) interrupt are unmasked correctable errors which are also covered later in this chapter. These PCI and EISA error conditions are signaled using IRQ<1> as device interrupt priority level (hardware treats these as if they were IO device interrupts). PAL code will dispatch the appropriate condition handler.

5.2.4 PCI Interrupts

The PCI interrupt logic is distributed between discrete logic and the CAP chip. Software has direct visibility into the interrupt logic via the INT_MASKx and INT_REQ registers. Software controls the target of each interrupt through the INT_TARGx and INT_MASKx registers. Software controls where CPU daughter card specific interrupt information is stored in memory through the INT_ADR and INT_ADR_EXT registers. Table 5-3 defines the interrupt assignment for the Main Interrupt Logic.

Table 5-7 gives the system interrupt assignment for the EV5 interrupt pins. An interrupt is enabled if the current IPL is less than the target IPL of the device. All interrupts are disabled when the processor is executing in PAL mode.

⁷⁰ CAP sees all SERR# assertions and will generate the interrupt.

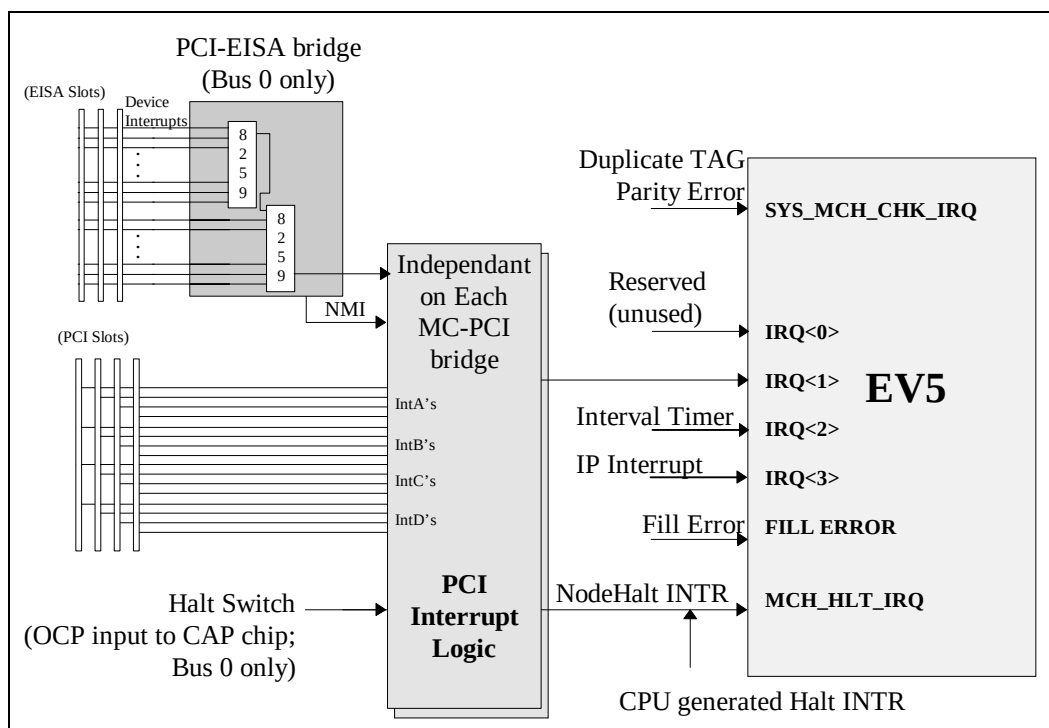


Table 5-6 Interrupt Figure

5.2.5 Interrupt Pin Assignments

EV5 Interrupt Pin	IPL	EV5 Suggested Usage	Rawhide Usage
IRQ<0>	20	Corrected System Error	reserved (unused)
IRQ<1>	21	Medium Priority External Interrupt	PCI and (E)ISA Device and Error Interrupts
IRQ<2>	22	Timer, inter processor interrupt	Interval Timer
IRQ<3>	23	High Priority, Real time devices	IP Interrupt
SYS_MCH_CHK_IRQ	31	Serious System Error	Duplicate TAG Parity Error ⁷¹
FILL ERROR	--		MC Bus read transaction terminated in a Fill error
MCH_HLT_IRQ	--		Halt button, CPU halt

Table 5-7 EV5 Interrupt Assignment

RESET changes the meaning of the EV5 IRQ inputs as described in the EV5 specification.

5.2.6 PCI Device Interrupts

The CAP chip contains an interrupt request register (INT_REQ) that reflects the state of all PCI and error interrupts on the specific MC-PCI bus bridge. It also provides a pair of interrupt target and interrupt mask registers that allow configuration software to route any interrupt to either of the two CPU Daughter Cards specified in the Interrupt Target register. Table 5-2 (E)ISA Interrupt assignment and Table 5-3 PCI Interrupt Vector Assignment specifies how interrupt lines are tied to the CAP chip and converted into vectors.

⁷¹ Cached CPU Daughter Card only

5.2.7 SYS_MCH_CHK_IRQ

The Rawhide system connects the SYS_MCH_CHK_IRQ interrupt pin to the error output of the Cache control ASIC on the Cached CPU Daughter Card. On uncached CUD this pin is not connected.

5.2.8 FILL ERROR

The Rawhide system connects the FILL ERROR pin to MC bus protocol logic. It is asserted when an MC Bus read (Load) transaction terminates with FILL ERROR. PAL exception handling code addresses this condition in an operating system dependant fashion as specified in the Alpha SRM.

5.2.9 Halt/Reset Switches

Depressing the Reset button on the operator control panel will cause a complete hardware reset of the system (just like a power cycle).

The halt button is signaled to the CPU daughter card which is target of the halt interrupt. It is presented to EV5 as a MCH_HLT_IRQ. This interrupt is not masked by an IPL level but is masked if the EV5 is in PAL mode.

How MCH_HLT_REQ is handled depends on the operating system and is handled uniquely by O/S specific PAL or HAL code. If all processors are to be halted PAL code will generate halt interrupt transactions on the MC bus (writes to CPU daughter card specific IO space addresses). If a system reset is required then the handler should write the RESET bit in a CSR?⁷² to cause the same effect as depressing the reset button. Logic generating the MCH_HLT_IRQ is triggered by the high to low edge on the **CHALTL** signal from the OCP. Firmware can read the CAP_ERROR register to sense the state of the OCP HALT button and, if it is specified to do so, Firmware may keep the system quiescent until the reset button has been released.

5.3 Error Reporting Philosophy

Errors are the manifestation of temporary or permanent faults that become detectable to software. As often as possible errors should be signaled though the layers of the system or protocol stack to the application program that is the ultimate victim of the error. The victim application program should decide whether to retry, continue, or abort.

Faults may exist in a system and never manifest as errors or effect performance. For example, Correctable Read Data-CRD memory errors detected by the MC-PCI bus bridge are corrected on the fly with no performance degradation. These are visible to software when CRD interrupts are enabled and may be recorded in an event or error log-but for correct operation it is not necessary to do anything about CRD errors detected by the IO subsystem.

Operating systems we have consulted with report that all hard (Uncorrectable) errors result in a system crash. Because of this, all hard errors detected by the PCI or EISA bus bridge are converted to 660 Machine check interrupts.

5.4 Rawhide System bus to PCI bus bridge errors

In all cases:

- If it is a PCI error, and error detection is enabled, assert SERR or PERR on PCI
- Set SOFT_ERR or HARD_ERR in the interrupt register (INT_REQ) to signal the 620 or 660 MCHK.
- Capture error symptoms in MC or PCI side error registers.

⁷² The RESET CSR is a bit in the ELCR register on the XBUS.

Hardware Interrupts and Exceptions

Error Type	Transaction	What happens to the data?	Symptoms logged
MC bus Command/Address Parity Error	PIO write	No transaction. Data dropped	If enabled, IOD captures symptoms, failing address and CMD/ADDR, and generates a hard error interrupt (660 MCHK)
	Peer-Peer DMA write		
	Memory Write	Data written to memory	
	PIO read	Appears as an MC bus NXM (read not pended)	
	Peer-Peer DMA read		
	Memory Read	Fill from apparent address	
Uncorrectable Data Error (RDS)	PIO write	Data Parity bit flipped on PCI when data is written.	If enabled, IOD captures symptoms, failing address, And ECC error syndrom bits. A hard error interrupt (660 MCHK) is generated
	Peer-Peer DMA write	Byte Enables cleared on bad quadwords.	
	Memory Write	Data written to memory	
	PIO Read	CPU detects error	
	Peer-Peer DMA read	Data parity bit flipped on PCI when data is returned.	
	CPU Memory Read	CPU detects error	
	DMA Memory Read	Data parity bit flipped on PCI when data is returned	
	Read-modify-write, RDS on the read data	Check bits toggled to force bad ECC on corrupt quadwords	
	TLB Fill	PTE Valid bit cleared, Fill completes	
	Bridge CSR write	Write the register	
MCbus NXM	Any read	Target Abort	IOD captures symptoms
Mcbus Fill Error	Peer-peer DMA read		
PTE Invalid	DMA read		
	DMA Write		
Correctable Data Error (CRD)	All	Data is corrected	If enabled, IOD captures symptoms, failing address & syndrom bits and generates a soft error (620 MCHK) interrupt.

Error Type	Transaction	What happens to the data?	Symptoms logged
PCI bus PERR or or SERR	PIO write	Transaction continues until target terminates	If enabled, IOD captures symptoms, failing PCI address and generates a hard error interrupt (660 Machine Check).
	Peer-Peer DMA write		
	DMA Memory Write	Data not written	
	PIO read	Fill error returned	
	Peer-Peer DMA read	Transaction continues until target terminates	
	DMA Memory Read		
PCI NXM AKA Master Abort (MAB)	PIO write	Data dropped; Write Pend Ack	IOD captures symptoms: PCI Failing address & MAB, and generates a hard error interrupt (660 MCHK Chk)
	Peer-Peer DMA write		
	PIO Read	Fill error returned	
	Peer-Peer DMA read		
PCI bus Target Abort	PIO write	Data Dropped transaction terminated	IOD captures symptoms: PCI Failing address & SERR, and generates a hard error
	PIO read	Fill Error	

⁷³In Rev A and B versions of the CAP chip there was a bug that caused memory to be written in this case as if it were a DMA to a direct mapped window

		interrupt (660 MCHK)
--	--	----------------------

5.4.1 Error detection by software (with error interrupts: Firmware initialized as the default)

Error interrupts are converted by PAL code to 620 and 660 Machine checks. These provide a deterministic signal that transaction has failed. Every bridge with symptoms will interrupt. Exception handling code in PAL or HAL can limit state collection and reset to the bridges that assert error interrupts.

Error symptoms do not change when interrupts are masked. In all cases listed below, if error interrupts are not masked, the bridge or bridges detecting the error will interrupt the processor identified as the interrupt target for the event.

5.4.2 Error detection by software (with error interrupts masked)

In the event of an error reported as a 660 or 670 machine check, PAL or HAL code will collect and reset error state on every bridge in the system.

With error interrupts, masked CPU initiated writes and all DMA transactions that fail will be signaled by a device such that the error is (eventually) detected by a device driver. CPU initiated read transactions that fail will terminate with Fill Error machine check.

5.4.2.1 CPU initiated read transactions

Failures will result in a Fill Error machine check. Error symptom bits will be set in one or more PCI bus bridges:

1. PCI Error: only the bus bridge and devices on the bus that was target of the read will have state to collect and reset.
2. MC bus Address Parity Error or Uncorrectable Data error: All PCI bus bridges will have state to collect and reset.

5.4.2.2 CPU initiated write transactions

Failures will be signalled at device IPL using the Hard Error Interrupt. PAL or HAL code will convert these to 660 Machine Checks. Error symptom bits will be set in one or more PCI bus bridges:

1. PCI Error: only the bus bridge and devices on the bus that was target of the read will have state to collect and reset.
2. MC bus Address Parity Error or Uncorrectable Data error: All PCI bus bridges will have state to collect and reset.

5.4.2.3 Device initiated memory transaction (DMA)

Failures will be signalled at device IPL using the Hard Error Interrupt. PAL or HAL code will convert these to 660 Machine Checks. The device is likely to generate an IO interrupt to the driver indicating that an operation did not complete.

1. PCI Error on the initiator bus: only the bus bridge and devices on the bus where the transaction was initiated will have state to collect.
2. MC bus Address Parity Error or Uncorrectable Data error: All PCI bus bridges will have state to collect and reset.

5.4.2.4 Device initiated peer transaction (DMA)

Failures will be signalled at device IPL using the Hard Error Interrupt. PAL or HAL code will convert these to 660 Machine Checks. The device is likely to generate an IO interrupt to the driver indicating that an operation did not complete.

1. PCI Error on the initiator bus: only the bus bridge and the device initiating the transaction will have state to collect.
2. PCI Error on the target bus: only the bus bridge and devices on the target will have state.
3. MC bus Address Parity Error or Uncorrectable Data error: All PCI bus bridges will have state to collect and reset.

5.5 EV5 Error Handling

Please refer to the Error Handling chapter of the EV5 specification for more details. Table 5-8 summarizes the EV5 features.

Region/fault	Description
Memory/BCache/uncached access	ECC checked. A Maskable, corrected ECC error interrupt is provided at IPL 31
BCache Tags	Parity checked
Ibox—No progress time-out	This is intended to prevent a hang. For the Rawhide system this should be programmed to the longest time-out period.
Outgoing BCache data	EV5 does not check outgoing BCache data. Neither does the Cached CUD ASIC. The MDP chips check ECC on every transaction and may be programmed to generate an interrupt in the event of bad ECC detected on the MC bus.
Address/cmd from system	Odd parity checked by the CAP chip. This is likely a fatal error. The CAP chip will capture the symptoms and generate a Maskable error interrupt.

Table 5-8 EV5 error detection features

5.5.1 MC-Bus Sizing Errors

CPU initiated I/O Space reads of non-existent MC-Bus addresses will result in the assertion of `FILL_ERROR` to the EV5. `FILL_ERROR` causes a PALcode MCHK entry, and subsequent delivery of a machine check to system software. If enabled, each IOD on the MC-Bus will set its `CAP_ERR<NXM>`, `CAP_ERR<MC_ERR_VALID>`, and `INT_REQ<HARD_ERR>` bits. Additionally, if enabled, each IOD will post an IPL15 I/O Interrupt to its selected CPU. The interrupted CPU may be different from the CPU which probed the non-existent MC-Bus address. For Cached-CUDs only, the first read of a WHOAMI register will return data with `CPU_INFO<7>` bit set, and the failing target device ID in the `CPU_INFO<5:0>` field. The non-existent failing address information is available in `MC_ERR0` and `MC_ERR1` until the `CAP_ERR<MC_ERR_VALID>` bit is cleared by software.

5.5.2 PCI Sizing Errors

CPU initiated reads of non-existent PCI addresses⁷⁴ will also result in the assertion of `FILL_ERROR` to the issuing EV5, and the delivery of a machine check to system software. The targeted MC-PCI bus bridge will set `CAP_ERR<MAB>`, `CAP_ERR<PCI_ERR_VALID>`, and `INT_REQ<HARD_ERR>` bits. If enabled, the bridge will post an IPL15 I/O Interrupt to its selected CPU. This interrupted CPU may be different from the CPU which probed the non-existent PCI address. For Cached-CUDs only, the first read of a WHOAMI register will return data with `CPU_INFO<7>` bit set, and the DEVICE ID of the failing target IOD in the `CPU_INFO<5:0>` field. The non-existent PCI address will be latched in the target IOD's `PCI_ERR1` register until the `CAP_ERR<PCI_ERR_VALID>` bit is cleared by software.

5.6 PCI Error Handling

The PCI specification allows for parity error detection on both address and data cycles. The parity check includes `AD<31:0>` field, the byte enables as well as the 64-bit PCI extension (`AD<63:32>`). All PCI options must generate parity and nearly all PCI options check parity (the exceptions are devices, such as video frame buffers, which cannot create system integrity problems in the event of undetected errors).

There are two PCI signals provided for error notification: `PERR` and `SERR`.

⁷⁴ Any PCI address space; Config, IO, or Memory.

- PERR# -- used to report DATA parity errors (except for the special cycles since these are broadcast writes and PCI devices are not required to listen to special cycles). The master reports read parity errors, and the target reports write parity errors.
- SERR# -- typically used to report ADDRESS parity errors and Special cycle DATA parity errors, However, SERR# may also be used to report catastrophic system errors (such as certain master⁷⁵ or target aborts). SERR# is asynchronous with respect to a PCI transaction. SERR# is a one clock cycle pulse, but it may linger for a few cycles because of the modest pull-up.

A selected agent that detects an address parity error will do one of the following (see PCI specification, Revision 2.0, section 3.7.2.2):

1. claim the cycle and terminate cleanly as though the address was correct;
2. claim the cycle and terminate with a target abort; or
3. not claim the cycle and let it terminate with a master abort.

For both PERR# and SERR#, the PCI specification suggests that the transaction should complete gracefully, preserving normal PCI operation.

5.6.1 PERR# Implications

A PCI device detecting a PERR# is expected to gracefully complete the transaction and to inform the device driver by posting an interrupt (or setting a flag). Any PCI-PCI bridge will transparently propagate bad parity to the target/master, allowing the receiving device to act accordingly by posting interrupts, etc.

Problems arise with the Intel PCI-EISA bridge which does not propagate bad parity; it simply asserts PERR# and expects the system hardware (i.e. the CAP chip) to notify the driver. Consequently, the CAP chip is compelled to always sample the PERR# signal⁷⁶ and assert an error interrupt anytime PERR# is asserted. The ramification of this is that occasionally two interrupts may be posted ; Table 5-9 shows this could occur if two PCI devices are communicating on the PCI bus (peer-peer) and a data error is observed (in this case the CAP will post a HARD_ERROR (660 MCHK) interrupt and the PCI device will probably assert an interrupt). Also note that this only happens on the PCI bus that also has an EISA bus bridge (CAP chip know this by the device type of the Saddle module).

Rawhide discourages peer-peer DMA that includes the PCI-EISA bus bridge in the transaction.

Data parity error scenario	Device detecting parity error	Interrupt by PCI Master device?	Interrupt by CAP chip?
PCI device Write to CAP/MDP	MDP	probably not	Maskable (yes)
PCI device Write to any other PCI device	PCI target device	probably not	
PCI-EISA bridge Write to CAP/MDP	MDP	no	
PCI-device Write to PCI-EISA bridge (prohibited) ⁷⁷	PCI-EISA	probably not	
PCI device (excluding the PCI-EISA bridge) Read of any address	PCI Master	probably not	
PCI EISA bridge read of any PCI device (excluding the CAP/MDP) (prohibited) ⁷⁷	PCI-EISA	no	
PCI-EISA bridge read of CAP/MDP	PCI-EISA	no	

Table 5-9 Rawhide handling of PCI data errors

⁷⁵ not all Master Aborts are catastrophic -- for instance, a special cycle always enjoys a master abort.

⁷⁶ not just when it is the target of a transaction)

⁷⁷ EISA devices are not permitted to participate in peer-to-peer DMA on Rawhide.

5.6.2 SERR# Implications

This is a serious error and it is expected that a system crash will be the result.

5.6.2.1 CAP Target Abort

If the CAP chip receives a target abort for its own PCI transaction then it will assert SERR#, and it will post a HARD_ERROR (660 MCHK) interrupt.

5.6.2.2 PCI Address parity error-CAP master

As a master device, the CAP will be notified of an address parity error either through a Master abort, or SERR#. In any case, the CAP chip will post a HARD_ERROR (660 MCHK) interrupt.

However, since the PCI-EISA bridge subtractively decodes the PCI address space, and thus absorbs all unclaimed addresses, it is clearly the most-likely target for a bad PCI address. Unfortunately, the PCI-EISA Bridge chip does not report address parity errors; and consequently, the majority of the PCI memory address parity errors will not be detected (and instead may access or corrupt some EISA/ISA device).

5.6.2.3 PCI Address parity Error—CAP target:

There are three possibilities:

- If the (erroneous) address hits in one of the PCI windows in the CAP then the CAP will claim the transaction. The CAP is optimized to accept a PCI transaction (i.e. DEVSEL) as soon as possible, and requiring DEVSEL to be qualified by an address parity check is one of the tight timing paths. One cycle later we will assert SERR#.
- The address does not hit in any of the PCI windows in the CAP, but does hit in some other PCI device's window. In this case, the other PCI device may claim the cycle and generate an SERR#. The CAP will also assert SERR# and will induce a HARD_ERROR (660 MCHK) interrupt on detecting SERR#.
- The address does not hit in any of the PCI windows in the CAP and is not claimed by any other PCI device⁷⁸, the CAP will assert SERR#.

5.6.2.4 DAC cycle address parity error:

The handling for this case is the same as for a standard (SAC) PCI address parity error, as described above. The main difference is that the PCI_ERRx registers will contain the least significant 32 address bits. The most significant bits of the address can be inferred by reading the PCI DAC BASE register and are not available for hits in the monster window.

5.6.2.5 Special cycle data parity error:

The CAP will not detect this error. The handling of this error is as follows:

When a special cycle is issued there are two situations to consider. First, all the PCI devices are deaf to the special cycle, in which case a parity error is immaterial; and second, if a device is listening for the special cycle then it will almost certainly report the data parity error as a SERR#. The CAP will intercept the SERR# and report it VIA HARD_ERROR (660 MCHK) interrupt.

5.6.2.6 TAG hit/TLB invalid error:

This case may be detected after the transaction has completed in the case of a write. As a consequence, this error is signaled on the PCI as an SERR#. The CAP will report this VIA HARD_ERROR (660 MCHK) interrupt.

⁷⁸ only possible for addresses which are not subtractively decoded by the PCI-EISA bridge (outside the first 64 Kbytes of address space).

5.7 Rawhide (MC bus) error Handling

The MC-PCI bus bridge and EV5 detect the vast majority of the Rawhide System Errors. The remaining errors are detected by the PCI-EISA bus bridge chip set and by PCI option devices (SERR# & PERR#). The MC-PCI bus bridge acts as a monitor of all MC bus transactions. Errors may be detected on the MC bus when the bridge is a participant in a transaction or when it is a bystander. Errors on a PCI bus are only detected when the bridge is a participant.

When an error is detected, symptom information is latched and the failing command is completed or terminated as gracefully as possible. The error condition is signaled to the EV5 by an interrupt which invokes the Rawhide specific PAL code. This PAL code reads the error information and initiates the appropriate action.

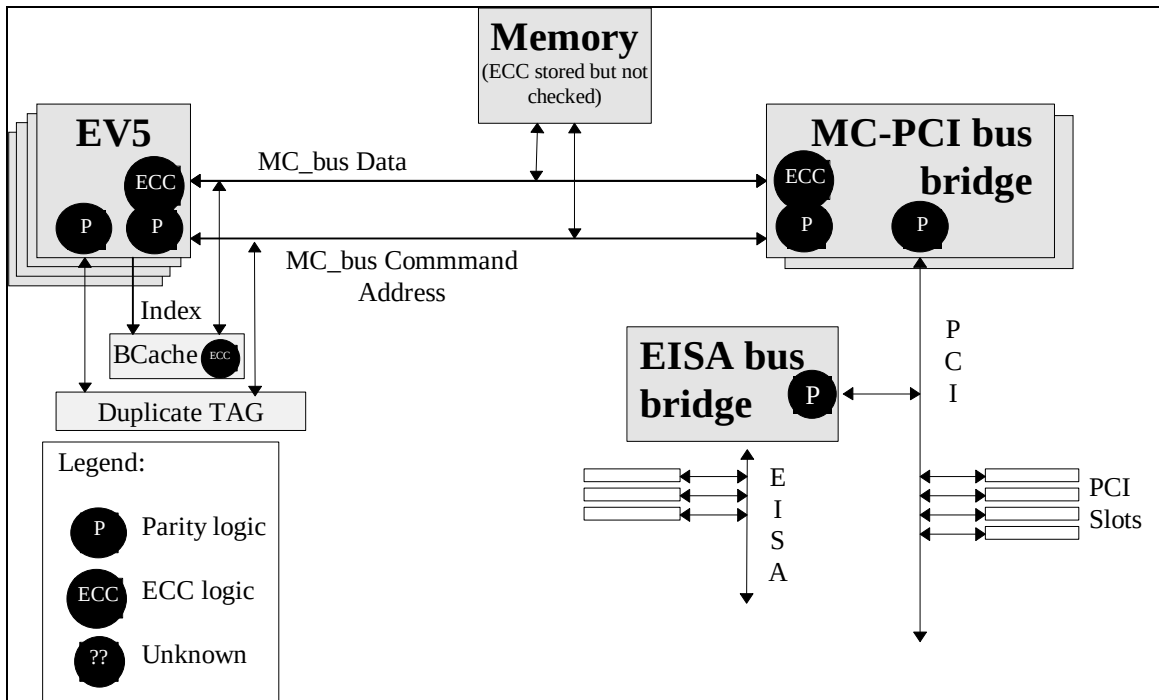


Table 5-10 Rawhide Error Logic

Table 5-10 shows where there is error detection and generation logic:

- **Rawhide system memory**: ECC protected.
- **21164 Internal ICache, DCache, SCache TAG**: Parity Protected
- **MCbus data lines (MC_DAT)**: ECC protected. Data checked by the MC-PCI bus bridge and by the EV5 when it is consumer of the data.
- **MCbus Command/Address**: Parity Protected, parity checked by the MC-PCI bus bridge.
- **BCache**: data is ECC protected.
- **Duplicate TAG RAM**: Parity protected
- **PCI bus**: Parity protected. PERR# (Data Parity Error) and SERR# (address parity and system error) are detected by the MC-PCI bus bridge.
- **EISA bus**: Parity for I/O board memory and other catastrophic errors VIA the ISA IO-CHK signal.
- **Scatter/Gather TLB in the CAP chip**: not protected

- **Flash & EEPROM ROM:** No parity/ECC checking. Checksum if implemented by software.

There is no parity protection on the MCbus arbitration (MC_REQ, MC_GRANT, MC_RHOLD, MC_DIRTY_EN, etc.), Mask lines (MC_BMSK), or other miscellaneous signals.

5.7.1 CAP & MDP ASIC Error Registers

The symptom registers are specified in Chapter 4.

Description	Error Register		Details
Error Indication	CAP Error Register	CAP_ERR	Which error occurred
MC Bus Errors	MC Error Information Registers	MC_ERR0	Failing Address <31:4>
		MC_ERR1	Failing Address <39:32>, command, int4_valid, DEVICE_ID and Dirty bits
PCI detected errors		PCI_ERR1	PCI Address
MDP Status	MDP Data Path Status	MDPA_STAT MDPB_STAT	Which error occurred and chip revision
ECC Syndrome	ECC Syndrome Registers	MDPA_SYN MDPB_SYN	

5.7.2 CAP & MDP Detected Errors

This section may be redundant with Section 5.4 on page 5-121. It is included because it identifies which system bus node may detect each error and may clarify some other aspect of error signaling for some readers.

Hardware Interrupts and Exceptions

Fault	Transaction Type	Source	Detection Point	Detection Point Signal	Signaled
CRD	DMA read (memory or peer-peer)	Memory, CPU (Dirty cache line), Bridge ⁷⁹	All MC Bridges	Maskable Bridge interrupt	630 Machine Check (Soft Error Interrupt)
CRD	CPU Read	Memory, Bridge or CPU (dirty Cache Line)	EV5 & All MC Bridges	Maskable Bridge interrupt	630 Machine Check EI_STAT<30> = 1 (System/Fill) EI_STAT<31> = 1 (COR_ECC_ERR) EI_ADDR<39> = 1; (if I/O space) = 0; (if Memory space)
CRD	Write	CPU or Bridge	All MC Bridges	Maskable Bridge interrupt	630 Machine Check (Soft Error Interrupt)
RDS	DMA Read (memory or peer-peer)	Bridge ⁷⁹ Memory or CPU (dirty cache line)	All MC Bridges	PCI SERR	660 Machine Check (Hard Error Interrupt) & Device Interrupt IRQ1
RDS	CPU PIO or Memory Read	Bridge, Memory or CPU (dirty cache line)	EV5 & All MC Bridges	Maskable Bridge interrupt	660 Machine Check EI_STAT<30> = 1 (System/Fill) EI_STAT<32> = 1 (UNC_ECC_ERR) EI_ADDR<39> = 0;(memory space) or = 1;(I/O space)
RDS	Write	CPU or Bridge	All MC Bridges	Maskable Bridge interrupt	660 Machine Check (Hard Error Interrupt)
PCI Address Parity Error	CPU Write or DMA	Bridge ⁷⁹ or PCI device	Bridge & All devices on PCI	PCI SERR	660 Machine Check (Hard Error Interrupt) & Device Interrupt IRQ1
PCI Address Parity Error	CPU Read	PCI Device or Bridge	PCI Device or Bridge	PCI SERR FILL Error & Maskable Bridge interrupt	660 Machine Check (FILL_ERROR_H asserted to EV5)

5.7.3 MC bus Error and PCI bus Error Interrupts

These are serviced at device interrupt priority level (IPL). They are converted by PAL code into bus specific machine checks and symptoms are logged by software.

5.7.3.1 MC Errors

In general, errors on the MC bus are detected by the target of a transaction. MC bus protocol specifies a Fill Error as the error signaling mechanism on failed read transactions. Failed write transactions are signaled as imprecise

⁷⁹ Error occurred on the Horse module or one of the three bridge ASICs.

machine check (660) exceptions. Both read and write errors will be signaled as bystander error Machine check (660) exceptions if bystander error reporting is enabled in the CAP chip.

5.7.3.2 PCI Error Interrupts

The PCI Specification requires that PERR# be signaled to the master of the transaction, allowing the originator of the request, at the hardware or software level, the prerogative of recovery (see section 3.7.2.1 of the PCI Specification, Rev 2.0). Thus, when the MC-PCI bus bridge interrupts the EV5 because of a PERR#, it is possible that the originating PCI master has also set its own interrupt or flag, signaling to its device driver to handle the parity error gracefully.

As such, it is important that we use care in specifying which errors should be enabled by configuration code to signal the EV5 using Bridge (Hard error) interrupts (RDS). Hard (RDS) error interrupts (660 Machine checks) will always cause VMS and Digital UNIX to crash. Current assumption is that all errors are enabled.

5.8 Error Insertion

For diagnosis and DVT there are design features for insertion of errors in the system

5.8.1 Force Error Registers

The CAP/MDP chips have CSRs that contain force error bits.

If the force parity error bit in the register is asserted then a parity error will be forced on all PCI transactions for which the MDP is source of the data.

If the USE_CHK bit is set to cause the MDP to select substitute ECC then on all DMA Write transactions the MDP will substitute the contents of the DIAG_CHECK field of the MDPx_DIAG register for the ECC field on all MC bus data cycles for which the MDP is the producer of the data.

Description	Register	Details
Force Error Control	DIAG_CHKA, DIAG_CHKB	Contains force bad parity on the PCI data lines and force substitute ECC on the MC bus.

5.8.2 Accessing main memory via CPU Uncached space

The Rawhide IO system can be configured to convert or loop-back an IO space transaction into an MC bus transaction.

The mechanism is to configure the PCI DMA window to a certain address space, and perform CPU uncached IO read/writes to the same address space. The CPU IO read/write will be sent out on the PCI bus and will be acknowledged by the CAP chip because the target address of the transaction will hit in the PCI window. The CPU IO read/write will be converted by the CAP/MDP MC-PCI bus bridge into an MC bus transaction which may target main memory.

5.8.3 Writing Bad ECC to memory

The Force Error Control registers may be used instead of ECC generation logic in the MDPA/MDPB to load bad ECC into memory. Thus diagnostics can load bad ECC into memory using the loop-back mechanism described in section 5.8.2. This bad ECC can be accessed using a CPU fill or using loop-back as specified in 5.8.2.

With the "loop-back" path the complete DMA data path inside the CAP/MDP is tested. Furthermore, Read and Flush operations to the EV5/BCache are always performed (since the width of a CPU uncached operation is less than a cache block in size) and the scatter/gather, direct addressing, or monster windows may all be used.

5.9 Digital UNIX and Open VMS Errors

5.9.1 Rawhide Hard Errors

Hard errors in the Rawhide platform consist of any non recoverable EV5 detected errors (MCHK670) , System dependent unrecoverable errors (MCHK660) and any Hard Errors generated by the IOD. In each case PALcode will build a Hard Error Frame and pass a pointer of this structure to the OS. The OS will format a Hard Error Log entry and pass it to the error logger. Table 5-11 provides a list of hard errors for the Rawhide platform and how they will be reported.

Hardware Interrupts and Exceptions

ERROR TYPE	SYMPTOM	Decode System area of frame ⁸⁰	MCHK MASK	Description and Other Information
MACHINE CHECK 670	DC_PERR_STAT_SEO<0>	WHOAMI only	0086	Second Dcache parity error in cycle AFTER register was locked.
	DC_PERR_STAT_LOCK<1>		0086	Parity error detected in Dcache
	DC_PERR_STAT_DP0<2>		0086	Data Parity error Dcache bank 0
	DC_PERR_STAT_DP1<3>		0086	Data Parity error Dcache bank 1
	DC_PERR_STAT_TP0<4>		0086	TAG Parity error Dcache bank 0
	DC_PERR_STAT_TP1<5>		0086	TAG Parity error Dcache bank 1
	SC_STAT_SC_TPERR<2:0>		0086	TAG Parity error during Scache TAG lookup
	SC_STAT_SC_DPERR<10:3>		0086	Data Parity error during Scache read
	SC_STAT_SC_SCND_ERR<16>		0086	Parity Error while SC_TPERR or SC_DPERR was already set
	EL_STAT_BC_TPERR<28>		0086	Bcache read encountered bad parity in the TAG control RAM
	EL_STAT_BC_TC_PERR<29>		0086	Bcache read encountered bad parity in the TAG control RAM
	ICPERR_STAT_DPE<11>		0086	Icache Data Parity error
	ICPERR_STAT_TPE<12>		0086	Icache Tag Parity error
	EI_STAT_UNC_ECC_ERR<32> && EI_STAT_EI_ES == 0		0086	Uncorrectable error from Bcache.
	EI_STAT_UNC_ECC_ERR<32> && EI_STAT_EI_ES == 1	Whole System Area	0086	Uncorrectable error from outside the CPU module
	ICPERR_STAT_TMR		0086	Timeout. If PCI errors set on a PCI bridge then grab snapshot from that PCI.
MACHINE CHECK 660 caused by a CPU detected Error	WHOAMI<6>	WHOAMI only	0086	Duplicate TAG Parity Error
	WHOAMI FILL_ERROR_H<7> ⁸¹	Whole System Area	0086	Fill Error. If PCI errors set on a PCI bridge then grab snapshot from that PCI.
Machine Check 660 caused by IOD detected MC_Bus Errors	CAP_ERR_RDSA		0086	Uncorrectable error detected by the MDPA in the IOD module. This causes the bridge to generate a hard error interrupt. PAL converts this to a MCHK 660.
	CAP_ERR_RDSB		0086	Uncorrectable error detected by the MDPB in the IOD module. This causes the bridge to generate a hard error interrupt. PAL converts this to a MCHK 660.
	CAP_ERR_NXM		0086	Non-Existant Memory.
	CAP_ERR_MC_ADR_PERR		0086	MC_Bus Parity Error
	CAP_ERR_MC_LOST		0086	Lost MC_Bus Error.
	CAP_ERR_PIO_OVFL		0086	Transaction not serviced, buffers are full.
Machine Check 660 caused by IOD detected PCI Errors	CAP_ERR_PTE_INV		0086	Invalid Page Table Entry on Scatter Gather access.
(Gather PCI snapshot for PCI bus)	CAP_ERR_MAB		0086	PCI Master state machine detected PCI Target abort.
	CAP_ERR_SERR		0086	Serious PCI Error detected.
	CAP_ERR_PERR		0086	PCI Parity Error detected.

Table 5-11 Rawhide Hard Errors

⁸⁰ This field defines if DECEvent need decode each set of IOD registers in the hard error frame system area or if only the WHOAMI need be decoded.

⁸¹ If WHOAMI<7> == 1, then ADDR<38:33> of the reference causing the Fill Error are contained in WHOAMI<5:00>. If WHOAMI<7> == 0, then WHOAMI<5:00> will contain revision information.

5.9.1.1 Machine Check 670

The Rawhide Machine Check 670 errors are EV5 detected errors. These errors are specifically detected by the EV5 and are fatal system errors. PAL will invoke the Hard Error error handler, build the Hard Error Frame, and pass pointer of the frame to the OS. The operating system will then generate a Hard Error Log Packet.

PALcode will build the CPU and system areas of the hard error frame. Text translators should translate the system area of the hard error frame except as noted in table Table 5-11 Rawhide Hard Errors.

5.9.1.2 Machine Check 660

MCHK 660 is defined as system dependant hard errors such as, errors detected and reported by the WHOAMI register and IOD Hard Error Interrupts are also converted to MCHK 660 by PAL. PAL builds the Hard Error Frame and passes the pointer to the OS. The operating system will then generate a Hard Error Log Packet.

While collecting error state PALcode will scan the error registers of each MC_Bus bridge for PCI errors and set a bit in the PCI Mask field in the Hard Error Frame, corresponding to the IODx. A one set indicates to the OS that a PCI snapshot should be taken for that PCI bus.

5.9.1.3 Hard Error Logout Frame

PALcode will statically allocate space for a total of 16 EV5 Logout frames and 16 IOD register subpackets. When a hard error occurs PAL passes a pointer to the CPU area of the Logout frame to the OS.

The Hard Error Logout Frame itself is not contiguous. PAL passes to the OS, a pointer to the CPU area of the Logout frame. The CPU area contains a offset to a System area. Both areas are to be logged as the Hard Error Logout Frame by the OS. It should be noted that only to one EV5 Logout frame is passed while the pointer in the CPU area points to the complete system area (all 16 IOD frames).

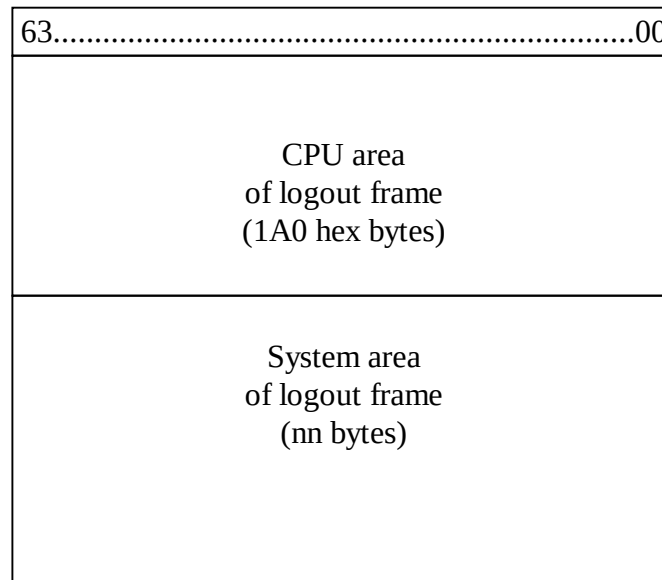


Figure 5-52 Hard Error Frame format for Digital UNIX and OpenVMS

5.9.1.3.1 Common EV5 Logout frame.

The EV5 Logout frames are allocated on a per-CPU basis and indexed based on the WHOAMI value of the processor. For a given error PALcode will save the EV5 registers to its EV5 portion of the Logout Frame.

Hardware Interrupts and Exceptions

	63.....48	47.....32	31.....16	15.....00	
Common Area:	R	S	D	C	Frame Size
	System Area Offset			CPU Area Offset	
PAL Code Area:	MCHK Frame Rev			MCHK	PCI
	PAL SHADOW REGISTERS (0-7)				
	PAL TEMP REGISTERS (0-23)				
CPU Area:	EXC_ADDR				
	EXC_SUM				
	EXC_MASK				
	PAL_BASE				
	ISR				
	ICSR				
	IC_PERR_STAT				
	DC_PERR_STAT				
	VA				
	MM_STAT				
	SC_ADDR				
	SC_STAT				
	BC_TAG_ADDR				
	EI_ADDR				
	FIL_SYN				
	EI_STAT				
	LD_LOCK				

Figure 5-53 EV5 Logout Frame (VMS & OSF)

The Logout Frame is as defined in the SRM. The following table defines bits <63:60> of the first Quadword of the frame.

Bit location	Name	Description
63	R	Retry bit as defined in the SRM
62	S	Second error bit as defined in the SRM
61	D	Diagnostics Bit. PAL will not build Logout Frame when delivering Machine Checks to diagnostics.
60	C	Scrubbed bit. Pal successfully scrubbed single bit error (620/630 Machine Checks only).

The CPU Area consists of registers defined in the EV5 specification. Other Definitions that may be of interest are in the table that follows:

Field Name	Description
Frame Size	Size of both the EV5 Logout Frame and System area of the hard error frame.
PCI	PCI Mask bits (15:00 of longword) used by PAL to indicate a PCI error was detected by the Bridge. The OS should grab PCI snapshot for each of the PCI bus(s) specified by the mask. Ex: Bit 0 = PCI 0, Bit 3 = PCI 3
MCHK	Machine Check reason mask field (31:16 of longword) For more information on Machine Check Reason Mask values see Table 5-11 Rawhide Hard Errors".

Table 5-12 Additional EV5 Logout Frame Definitions

5.9.1.3.2 System area of Hard Error Frame

The system area of the Hard Error Frame will be filled with IOD register subpackets for each IOD on the system, providing a snapshot of the I/O interfaces. For a definition of the IOD subpacket see, Figure 5-75 IOD Register Subpacket Format on page 5-154. Indexing into each IOD Subpacket and finding valid PCI_REV values will define which IOD subpacket contains valid register information in the system area.

The system area itself is a contiguous area pointed to by the System Area Offset in the EV5 Error Frame.

63.....00
IOD 0 Register Subpacket (60 hex bytes)
IOD 1 Register Subpacket (60 hex bytes)
IOD 2 Register Subpacket (60 hex bytes)
IOD 3 Register Subpacket (60 hex bytes)
IOD 4 Register Subpacket (60 hex bytes)
.
.
.
.
IOD 14 Register Subpacket (60 hex bytes)
IOD 15 Register Subpacket (60 hex bytes)

5.9.2 Figure 5-54 System area of Hard Error Logout Frame⁸²Rawhide Soft Errors

Soft errors for Rawhide are single bit correctable errors detected by the system. IO detected errors (on IOD) will generate a soft error interrupt. PALcode will service this interrupt and turn it into a MCHK 620 and invoke the Machine Check 620/630 handler. The OS will log a Machine Check 620/630 Error Log Entry (Reference Figure 5-57). There is no other special handling by the PAL or OS.

⁸² In Rawhide only IOD 0-3 are used. The rest are listed for future use.

Error Type	Symptom	Reason Mask	Description and other Info
Machine Check 630	EI_STAT_COR_ECC_ERR & EI_STAT_EI_ES == 0	0086	Correctable Bcache Error.
Machine Check 620	EI_STAT_COR_ECC_ERR & EI_STAT_EI_ES == 1	0086	Collectable Error detected from source outside the CPU module.
	CAP_ERR_CRDA	204	Correctable Error Detected the IO Bridge Module.
	CAP_ERR_CRDB	204	Correctable Error Detected the IO Bridge Module.

Table 5-13 Soft Errors for Rawhide

5.9.2.1 Machine Check 630

Machine Check 630 are generic EV5 correctable errors. The error handler should be the same across all platforms that use the EV5. A generic EV5 correctable error is one that is sourced from Bcache. PALcode will invoke the 630 error handler, build the Machine Check 620/630 Logout Frame and pass a pointer to the structure to the OS.

The OS will generate a Soft Error Log entry and log it to the system event log. For Machine Check 630 the text translator need only decode up to and including the WHOAMI register of the Frame.

5.9.2.2 Machine Check 620

Machine Check 620 reports system dependent single bit errors. PAL will build the Machine Check 620/630 Logout Frame and pass a pointer to the structure to the OS.

5.9.2.3 The Machine Check 620/630 Logout Frame

63.....48		47.....32		31.....16		15.....00		Offset
R	S	D	C					
				Frame Size				00
System Area Offset				CPU Area Offset				08
MCHK Frame Rev				MCHK Reason Mask				10
EI_STAT								18
EI_ADDR								20
FIL_SYN								28
ISR								30
RESERVED				WHOAMI				38
Base Address of Bridge								40
MCERR0				PCI_REV				48
CAP_ERR				MCERR1				50
MDPA_SYM				MDPA_STAT				58
MDPB_SYM				MDPB_STAT				60
RESERVED				RESERVED				68

Figure 5-55 Machine Check 620/630 Logout Frame (VMS & OSF)

The Logout Frame is as defined in the SRM. The following table defines bits <63:60 > of the first Quadword of the frame.

Bit location	Name	Description
63	R	Retry bit as defined in the SRM
62	S	Second error bit as defined in the SRM
61	D	Diagnostics Bit. PAL will not build Logout Frame when delivering Machine Checks to diagnostics.
60	C	Scrubbed bit. Pal successfully scrubbed single bit error (620/630 Machine Checks only).

The CPU Area consists of registers defined in the EV5. The System Area of the Machine Check 620/630 Logout Frame consists of registers from the IOD on the MC_Bus which has the ESCI bridge on the PCI.

5.9.3 OpenVMS and Digital UNIX Rawhide Error Log Entry Formats

The following sections contain the valid Error log entries for Rawhide.

Event Type	VMS Event Type	OSF Class & Type	Description
Machine Check 670	2	100-2	Used for 670 processor Machine Checks
Machine Check 660	27	100-1	Used for 660 system Machine Checks
630 Error Interrupts	26	100-4	Used for Bcache correctable 630 Interrupts
620 Error Interrupts	6	100-3	Used for MC_Bus detected single bit error
Last Fail	29	110-1	Used to collect MC_Bus registers just prior to crashing
I/O Error interrupt ⁸³			IOD error interrupts
System Environment ⁸⁴	102	110-2	Used to provide status on fans, temp etc...
Configuration	43	110-3	Used to depict the Kernel configuration including I/O buses

Table 5-14 Supported Event Types

5.9.3.1 Hard Error Log Entry Format

This error log format is used for all hard error entries by the Rawhide platform. Digital Unix and OpenVMS will use this format. For a list of hard errors see Table 5-11.

⁸³ For OpenVMS and Digital Unix, PAL will be converting these to Machine Check 660 or 620 errors for servicing.

⁸⁴ Neither OpenVMS or Digital Unix is currently planning on supporting the interrupts to generate this entry. Sufficient information is provided should this be added in a future release.

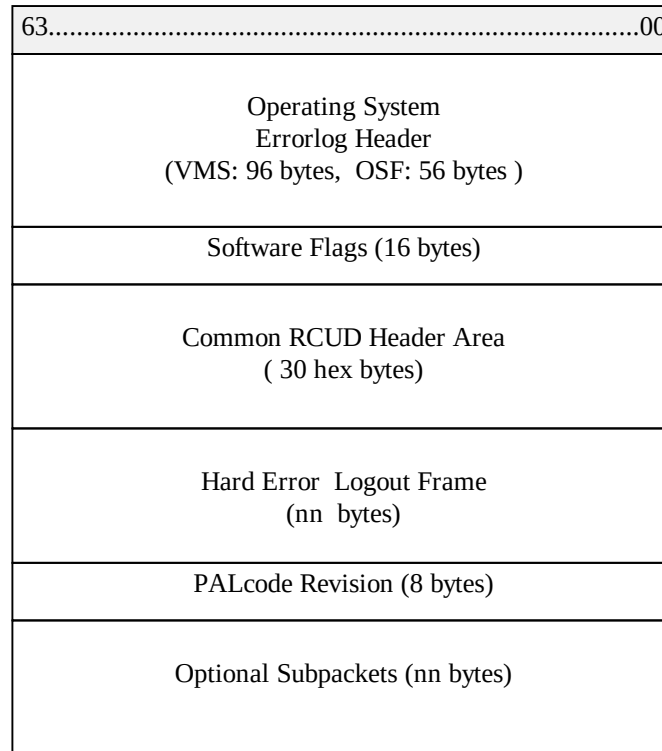


Figure 5-56 Rawhide Machine Check 660/670 Errorlog Entry Format (VMS and OSF)

Optional Subpackets:

- PCI Snapshot
- ESC Register Subpacket⁸⁵

5.9.3.2 Machine Check 620/630 Error Entry Format

The Correctable Error 620/630 Logout frame is used to provide correctable error information to the operating system error handler. When a correctable error occurs, PAL enters the correctable error flow, gathers all module error state, which includes internal EV5 registers and packages them in to a Correctable Error Logout Frame.

⁸⁵ The ESC Subpacket will follow the PCI snapshot(s) if both are present in the same error log entry.

63.....00
Operating System Error Log Header (VMS: 96 Bytes, OSF: 56 Bytes)
Software Flags (16 bytes)
Common RCUD Header Area (30 hex Bytes)
Rawhide 620/630 Error Logout Frame (70 hex Bytes)
PALcode Revision (8 Bytes)

Figure 5-57 Rawhide Machine Check 620/630 Errorlog Entry Format (VMS and OSF)

5.9.3.3 System Configuration Error Log Entry Format

The Rawhide system configuration packet will be generated, formatted and stored in memory (with exception of Error Log Header) by the Rawhide console. A pointer to the FRU TABLE Subpacket will be at HWRPB[D0]. If the pointer contains a 0 then the OS need not log an entry. Otherwise at boot time, the Operating System will extract the FRU TABLE Subpacket, place the OS specific Errorlog Header on the front and log this to the System Error log.

Since the console builds and formats the FRU TABLE Subpacket, specific information about FRU Table format and contents will be contained in the Power Up and Initialization chapter.

63.....00
Operating System Errorlog Header (VMS: 96 Bytes, OSF: 56 Bytes)
FRU TABLE Subpacket (Variable Length)

Figure 5-58 Rawhide System Configuration Errorlog Entry Format

5.9.3.4 System Environmental Error Log Entry Format

At this time, Digital Unix and OpenVMS will not be supporting logging of system enviromental events. This information is provided for future use.

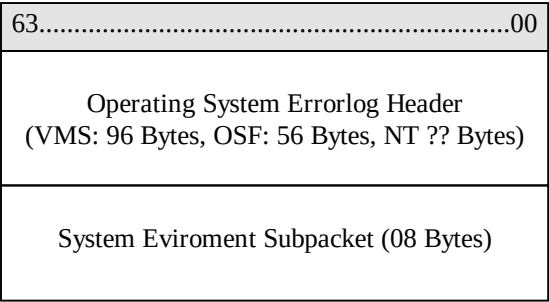


Figure 5-59 Rawhide System Environment Errorlog Entry Format

5.9.3.5 Open VMS Error Log Header

31.....24	23.....16	15.....08	07.....00	Byte Offset
SID				00
XSID_RSV		HDR_REV		04
CPU_ID		XSID_TYP	XSID_REV	08
DEV_TYPE	DEV_CLASS	CPI_ID...cont		0C
SCS_NAME				10
HDRSZ	OS_ID	FLAGS		20
Reserved		HD_ENDRY		24
Date/Time				28
Sequence Number				2C
OS Software Version				30
Error Mask				38
Absolute Time				3C
Dynamic System Recognition (32 Bytes)				40

Figure 5-60 OpenVMS Errorlog Header

5.9.3.6 OSF Error Log Header Format

31.....24	23.....16	15.....08	07.....00	Byte Offset
Sequence Number		Record Length		00
SID				04
Errorlog Version		Priority	Valid Record	08
TIME				0C
Host Name				10
SYS_TYPE				1C
Number of Processors in System				20
Which CPU Serviced the Error				24
ctldev_type	subid_type	subid_class		28
subid_unitnum		subid_num		2C
PCI Device Identification (LO)				30
PCI Device Identification (HI)				34

*Figure 5-61 OSF Errorlog Header***5.10 MicroSoft WindowsNT Error reporting**

All errors will be reported via either Hard Error packet or Soft Error packet. This will apply not only to Rawhide but all platforms with NT supporting error event logging. Each error log will consist of two separate sections:

- The text portion of the event log. This will be identical for all platforms and provide information such as time, date, error type etc.
- The binary portion of the event log will contain either the Generic Hard Error Frame or the Generic Correctable Error Frame. These two generic frames also provide a mechanism for reporting platform specific error information.

5.10.1 Generic Hard Error Frame for NT

This is the error frame that will be logged for all hard errors for Rawhide. Errors which are reported via hard error are those previously defined as Machine Check 670 and 660 errors (Reference Table 5-11 Rawhide Hard Errors for more details).

63.....32	31.....00
Performance Counter Value	Frame Version Number
Uncorrectable header Flags	Performance Counter Value..cont
Error String Pointer	Uncorrectable header Flags cont...
Physical Address	
Extended Error Information	Error Bit
Extended Error Information cont... (32 bytes)	
Processor Information (16 bytes)	
Raw Processor Information Pointer	Raw Processor Information Length
System Information (20 bytes)	
Raw System Information Pointer	Raw System Information Length
Raw System Information (Rawhide specific System error information) (xxx bytes)	

Figure 5-62 Generic Windows NT Hard Error Frame

Field	Description
Version Number	Defines the version of the error structure.
Performance Counter Value	The value of the system performance counter when the error was reported. The value is determined during error processing by the HAL and so may be captured significantly after the hardware detected the error. The value cannot be used for fine grained timing of when errors occurred but can be used for coarse grained timing and approximate timing.
Uncorrectable Flags	Structure and definitions defined later in this section. See Figure 5-63 Generic Error Flags
Error String Pointer	Pointer to a Text string describing the Uncorrectable error.
Physical Address	The physical CPU address of the quadword that caused the error report. This value is optional, its validity is indicated in the Flags
Error Bit	Bit in error
Extended Error Information	A structure that describes interpretation of the error. The values in the structure are optional, for a description of the Extended Error Information field see Extended Error Information Field on page 5-148.
Reporting Processor	A structure that describes the processor type on the system and the processor that reported the error. PROCESSOR_INFO structure is described in section 5.10.3.3. This information is valid only with the Raw Processor information is contained in the error packet.
Raw Processor Error Information length	The length of the raw processor error information structure in bytes. If == 0 then no Raw Processor Information is supplied
Raw Processor Error Pointer	A pointer to the raw processor error information structure. The of the structure is processor-specific. For the definition for the EV5 processors see section 0.
System Information	A structure that describes the type of system for which the error was reported. SYSTEM_INFO structure is described later in section 0.
Raw System Information Length	The length of the raw system error information structure in bytes. If == 0 then no raw system information is provided.
Raw System Pointer	A pointer to the raw system error information structure. The definition of the structure is system/ASIC-specific. This structure is defined in section 5.10.3.6.

Table 5-15 Uncorrectable Error Frame Field Definitions

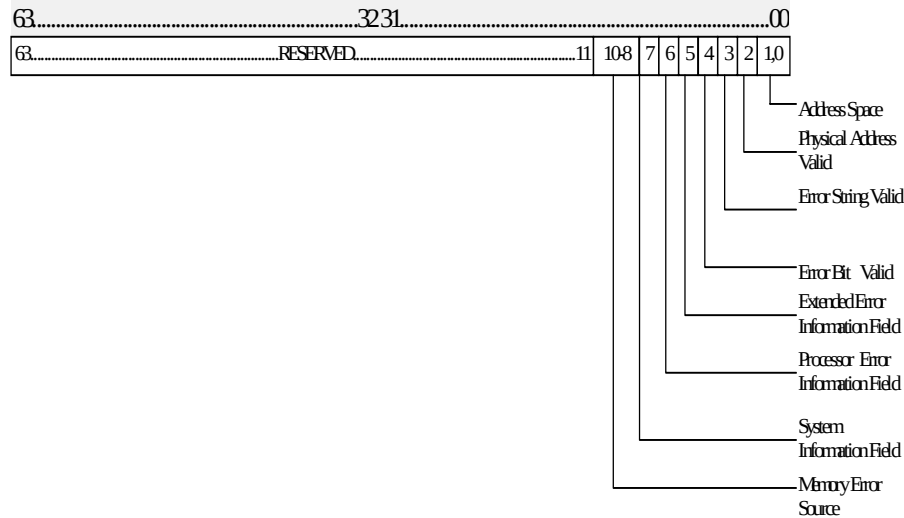


Figure 5-63 Generic Error Flags

Bit(s) Location	Description
0 - 1	Address Space Identifies the address space that was the source of the correctable error. The following apply for bits 1, 0 respectively: 00 Unidentified 01 Memory Space 10 I/O Space 11 reserved
2	Error String Valid.
3	Physical Address Valid. If set the physical address in the Correctable Error structure contains valid information.
4	Error Bit Valid
5	Extended Error Information Field. If set the Extended error information pointer in the Correctable Error structure points to valid information.
6	Processor Information Field is valid. If set the Raw Processor Information pointer in the Correctable Error structure points to valid information.
7	System Information Field is valid. If set the Raw System Information pointer in the Correctable Error structure points to valid information.
8-10	Memory Error Source. Identifies the source of a memory error. The following are valid for bits 10 - 8 respectively: 000 Unidentified 010 ProcessorCache 011 SystemCache 100 ProcessorMemory 101 SystemMemory

Table 5-16 Generic error flag bit definitions

5.10.2 Generic Soft Error Frame for NT

The HALcode passes this to report soft errors. A list of soft errors for Rawhide may be found in Table 5-13 Soft Errors for Rawhide.

63.....32	31.....00
reserved	Frame Version Number
Performance Counter Value	
Physical Address	
Correctable Flags (xxx bytes)	
Data Bit Error Mask	
Check Bit Error Mask	
Extended Error Information (xxx bytes)	
Reporting Processor Information (xxx bytes)	
reserved	Raw Processor Information Length
Raw Processor Information (Rawhide specific CPU error information) (xxx bytes)	
System Information (xx bytes)	
reserved	Raw System Information Length
Raw System Information (Rawhide specific System error information) (xxx bytes)	

NOTE: Byte offset information will be added to frame when definition is final.

Figure 5-64 Rawhide Correctable Error Frame for Windows NT

Field	Description
Version Number	Defines the version of the error structure.
Performance Counter Value	The value of the system performance counter when the error was reported. The value is determined during error processing by the HAL and so may be captured significantly after the hardware detected the error. The value cannot be used for fine grained timing of when errors occurred but can be used for coarse grained timing and approximate timing.
Physical Address	The physical CPU address of the quadword that caused the correctable error report. This value is optional, its validity is indicated in the Flags
Correctable Flags	for Structure and definitions see Figure 5-63 Generic Error Flags
Data Bit Error Mask	A mask that describes which data bits in the corrected word were in error. A value of one in the mask indicates that the corresponding bit in the data word were in error.
Check Bit Error Mask	A mask that describes which check bits in the corrected word were in error. A value of one in the mask indicates that the corresponding bit in the check bits were in error.
Extended Error Information	A structure that describes interpretation of the error. The values in the structure are optional, the EXTENDED_ERROR structure is described in Figure 5-63 Generic Error Flags.
Reporting Processor	A structure that describes the processor type on the system and the processor that reported the error. PROCESSOR_INFO structure is described in section 5.10.3.3.
Raw Processor Error Information length	The length of the raw processor error information structure in bytes.
Raw Processor Error Information	A pointer to the raw processor error information structure. The of the structure is processor-specific. The definition for the Rawhide processor(s) is defined in section 5.10.3.5.
System Information	A structure that describes the type of system for which the error was reported. SYSTEM_INFO structure is described in section 0 .
Raw System Information Length	The length of the raw processor error information structure in bytes.
Raw System Information	A pointer to the raw system error information structure. The definition of the structure is system/ASIC-specific. This structure is provided in section 5.10.3.7 .

Table 5-17 Correctable Error Frame field definitions

5.10.3 Generic Structures for NT Error Frames

In the sections that follow are structures used in both the Generic Hard Error Frame and the Generic Soft Error Frame. Any structure that is specific to either error frame is provided in the section that describes the affected error frame.

5.10.3.1 Extended Error Information Field

The extended error information subpacket used will vary dependent upon the Address Space and Memory Source Fields in the Flags for the Generic Correctable and Uncorrectable frames. Use the table provided to determine how to decode this field

Address Space	Memory Source	Extended Error Subpacket Type
01 (Memory Space)	01x	Extended Cache Error Information
01 (Memory Space)	10x	Extended Memory Error Information
10 (IO Space)	n/a	Extended I/O Error Information

Table 5-18 Determining Extended Error Information Content

5.10.3.1.1 Extended Cache Error contents

31.....00
Extended Cache Error Information Flags
Processor Information Subpacket (xxx bytes)
Cache Level
Cache Board Number
Cache Simm

Figure 5-65 Extended Cache Error Information Subpacket

Subpacket Field Name	Description
Extended Cache Error Information Flags	Identifies which fields in Cache Error Information Subpacket are valid. • bit 0 == 1 == Cache Level Valid is valid • Bit 1 == 1 == Cache Board Number is valid • Bit 2 == 1 == Cache SIMM is valid
Processor Information Subpacket ⁸⁶	Identifies Processor associated with the error.
Cache Level	Identifies level of cache that caused the error. • Primary cache == Level 1 • Secondary cache == Level 2 etc... Extended Cache Error Information flag must indicated that value is valid.
Cache Board Number ⁸⁷	Identifies the board number of the cache that caused the error. Extended Cache Error Information flag must indicated that value is valid.
Cache SIMM Number ⁸⁸	Identifies Cache SIMM that caused the error. Extended Cache Error Information flag must indicated that value is valid.

Table 5-19 Extended Cache Error Information Field Definitions

⁸⁶ Valid only if the Memory Error Source field of the Correctable flags == 010.

⁸⁷ For Rawhide the Cache Board Number Should Equal the reporting Processor unless the transaction is dirty (should be indicated somewhere in error information collected but I haven't sifted through everything yet). If the transaction is dirty, the MC_Bus presently does not save/latch which CPU is supplying data to the bus.

⁸⁸ Rawhide Cache SIMM Number would be invalid for rawhide as well. As far as I know Rawhide is not using SIMM's on the cached version of the Processor. You could call out the DRAM that is causing the failure but you may then get into callout problems is the board goes to layout at a later date.

5.10.3.1.2 Extended Memory Error contents

31.....00
Extended Memory Error Information Flags
Processor Information Subpacket (xxx bytes)
Memory Board Number
Memory Simm

Figure 5-66 Extended Memory Error Information Subpacket

Subpacket Field Name	Description
Extended Memory Error Information Flags	Identifies which fields in memory Error Information Subpacket are valid. • Bit 0 == 1 == Memory Board Number is valid • Bit 1 == 1 == Memory SIMM is valid
Processor Information Subpacket ⁸⁹	Identifies Processor associated with the error.
Memory Board Number	Identifies the board number of the cache that caused the error. Extended Memory Error Information flag must indicated that value is valid.
Memory SIMM Number	Identifies Memory SIMM that caused the error. Extended Memory Error Information flag must indicated that value is valid.

5.10.3.1.3 Extended I/O Error contents

31.....00
Interface Type (xx bytes)
Bus Number
Physical Address (xx Bytes)

Figure 5-67 Extended I/O Error Information Subpacket

Subpacket Field Name	Description
Interface	Identifies Bus Interface type (i.e. PCI) that caused the error
Bus Number	Identifies the bus number of the bus that caused the error.
Physical Address	Identifies the physical address of the bus that caused the error.

*Table 5-20 Extended I/O Error Information Field Definitions*⁸⁹ Valid only if the Memory Error Source field of the Correctable flags == 010.

5.10.3.2 System Information Field

31.....00
System Type
Firmware Revision ID
System Variant
System Revision

Figure 5-68 System Information Structure Format

Field Name	Description
System Type	Identifies the type of system that reported the error. Rawhide Values are TBD.
Firmware Revision ID	Identifies the Firmware revision of the system that reported the fault.
System Variant	Numeric number distinguish variants of the same type. Rawhide values are TBD
System Revision	Overall System revision. Rawhide values are TBS.

5.10.3.3 Reporting Processor Information Field

31.....00
Processor Type
Processor Revision
Physical Processor Number
Logical Processor Number

Figure 5-69 Processor Information Structure format

Field Name	Description
Processor Type	Identifies the type of processor chip running on the system. Processor type is expected to be consistent across MP system.
Processor Revision	Identifies the revision of the processor chip running on the system. Processor revision is expected to be consistent across MP system.
Physical Processor Number	Physical Processor Number. For Rawhide this would be equivalent to the MC Bus node number
Logical Processor Number	Logical Processor number assigned by NT.

*Table 5-21 Processor Information Structure field definitions***5.10.3.4 Raw Processor Information Field for Uncorrectable Errors(EV5)**

The Raw Processor Information Field will be fill in with information specific to the CPU reporting the error. The following figure provides the content and format of the Raw Processor Information Field pointed to in the Generic WindowsNT Hard Error Frame.

63.....32	31.....00	Offset
IcPerrStat		00
DcPerrStat		08
ScStat		10
ScAddr		18
EiStat		20
BcTagAddr		28
EiAddr		30
FillSyn		38
BcConfig		40
BcControl		48

Figure 5-70 EV5 Uncorrectable Raw Processor Information

5.10.3.5 Raw Processor Information Field for Correctable Errors (EV5)

The Raw Processor Information Field will be fill in with information specific to the CPU reporting the error. The following figure provides the content and format of the Raw Processor Information Field pointed to in the Generic WindowsNT Soft Error Frame.

63.....32	31.....00	Offset
EiAddr		00
FillSyn		08
EiStat		10
BcConfig		18
BcControl		20

Figure 5-71 EV5 Correctable Raw Processor Information

5.10.3.6 Uncorrectable Raw System Information

For errors previously defined as Machine Check 670 errors, no MC_Bus state will be saved. The generic hard error frame will contain a valid EV5 type CPU information field. The Raw System Information Pointer will be NULL.

For errors previously defined as Machine Check 660 error, both CPU detected and IO hard error interrupts the system Raw System Information portion of the packet will contain the RCUD header, Software Flags, and an MC_Bus snapshot. Optionally a Memory Size Register Subpacket (for MC_Bus errors) and PCI Snapshot (if PCI error detected) may also be included.

See Table 5-11 Rawhide Hard Errors for information on error clasifications.

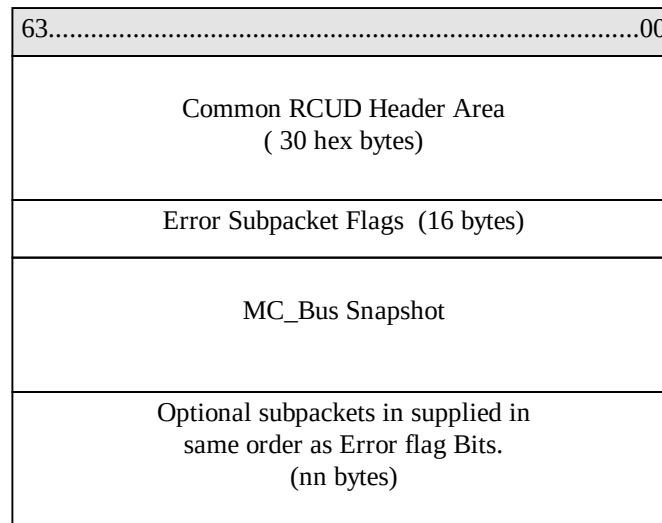


Figure 5-72 Uncorrectable Raw System Information

5.10.3.7 Correctable Raw System Information

5.10.4 Windows NT Error Log Entry Formats

Windows will report errors in either the Generic Hard Error frame format or Generic Soft Error Frame format. The error entry formats along with associated supporting structures are supplied in the NT error section.

5.10.4.1 System Configuration Error Log Entry Format

How WindowsNT will pass configuration is still TBD.

5.10.5 NT Error Log Header Format

This section will contain the contents of the text portion of the event log entry for error events (yet to be defined).

To be supplied by NT engineering.

5.10.6 I/O Error Interrupts

I/O error interrupts are split into two types HARD_ERR and SOFT_ERR. Both Digital UNIX and OpenVMS, PALcode will use the same hard error frame and error handling code to report I/O hard error interrupts. The operating system will append a PCI snapshot to the error log entry based upon the value in the PCI bits 15:00 in the Hard Error Frame. For soft error interrupts PAL will use the same code as the MCHK 620/630. Each Bridge on the MC_Bus will see errors out on the MC_Bus. Software should only generate a single error log entry per error event.

WindowsNT will report I/O hard error interrupts via the Generic Hard Error Frame. The OS should include (in addition to a MC_Bus snapshot) a Memory Register subpacket for errors to cached memory space (addr 39 == 0) and a PCI bus snapshot for any bridge detecting a PCI error.

5.10.7 System Monitoring Information

Implementation on how the system will be notified and collect information such as system temperature, fan operation etc. are currently TBD. Reference note 79.10 in the Rawhide notes conference for further information. When final decisions are made in this area this section will be provided.

5.11 Error logging Information

5.11.1 Software Error Flags

16 Bytes are allocated for Software Error Flags. A packet revision Longword is allocated at offset 00 followed by a reserved longword. The remaining 08 bytes are software flags bits.

63.....32	31.....00	Offset
Reserved	Packet_Revision	00
Error Subpacket Flags (8 Bytes)		08

Figure 5-73 Software Error Flag Format

Packet_Revision refers to the revision of the OS collected error information.

The following table provides a the software flag definitions. If the flag is set to a one then the specified subpacket/snapshot is in addition to what is already specified for an error log entry.

Flag	BIT	Description
SYS_ENV_PRESENT	10	System Environmental Register subpacket present
MEM_SIZE_PRESENT	11	Memory Size Register Subpacket Present
MC_BUS_PRESENT	20	MC_Bus Snapshot Present
	21	reserved
IOD_SUBPACKET_PRES	30	IOD Register Subpacket present
PCI_SNAPSHOT_PRES	31	PCI Bus snapshot present
ESC_SUBPACKET_PRES	32	ECS Register Subpacket present
IOD2_SUBPACKET_PRES	40	Second IOD Register Snapshots present
PCI2_SNAPSHOT_PRES	41	Second PCI Bus Snapshot

Table 5-22 Software Flag Definition⁹⁰

5.11.2 Common RCUD Header Format

This section provides generic information about the Rawhide CUD module that logged the error⁹¹.

63.....48	47.....32	31.....16	15.....00	Offset
HW_REVISION		Active CPU's		00
System Serial number (10 Bytes)				08
RESERVED			Serial Number cont....	10
Module Serial number (10 Bytes)				18
RESERVED		MOD_TYPE	Serial Number cont....	20
SYSEM_REV		DISABLED_RESOURCES		28

Figure 5-74 Common RCUD Header Area Format

- ACTIVE_CPUS. Operating system active CPU bit mask.

⁹⁰ For NT, flag bits are specified in order of appearance when appended to the error log entry. Unix and OpenVMS will only use the PCI and ESC flags. The ESC Subpacket will always follow the PCI snapshots if both are present in the error log entry

⁹¹ Any field not containing valid information will contain zero's

- HW_REVISION. Low Four bytes of HWRPB per CPU offset TBS.
- System Serial Number. contents of HWRPB per CPU offset TBS
- Module Serial Number. contents of HWRPB offset 64 (10 bytes only)
- MOD TYPE is the module type read from the IC bus CPU node Register.
- DISABLE_RESOURCES currently TBD.

5.11.3 Register Subpackets

5.11.3.1 IOD Register Subpacket Format

This register subpacket is used for each bridge in the system area of the Hard Error Frame for OpenVMS and Digital Unix. Also used by NT when forming the MC_Bus snapshot described later.

63.....32	31.....00	Offset
Base Address of Bridge		00
RESERVED	WHOAMI	08
CAP_CTRL	PCI_REV	10
HAE_IO	HAEMEM	18
INT_REG	INT_CTL	20
INT_MASK1	INT_MASK0	28
MC_ERR1	MC_ERR0	30
RESERVED	CAP_ERR	38
MDPA_STAT	PCI_ERR1	40
MDPB_STAT	MDPA_SYN	48
RESERVED	MDPB_SYN	50
RESERVED	RESERVED	58

Figure 5-75 IOD Register Subpacket Format⁹²

5.11.3.2 Memory Size Register Subpacket Format

This subpacket provides information to decode correct memory callout.

63.....56	55.....48	47.....40	39.....32	31.....24	23.....16	15.....08	07.....00	Offset
Valid Bits	RESERVED			Memory size 3	Memory size 2	Memory size 1	Memory size 0	00

Figure 5-76 Memory Size Register Subpacket Format

⁹² These RESERVED bits at offset 08 bits (63:32) may be used by WNT

5.11.3.3 System Environmental Register Subpacket Format

63.....56	55.....48	47.....40	39.....32	31.....24	23.....16	15.....08	07.....00	Offset
Valid Bits	RESERVED		Power system Function Register	Sensor Reg 2	OPC Control	Sensor Reg 1	Fan Fail Reg	00

*Figure 5-16 System Environmental Register Subpacket Format⁹³***5.11.3.4 PCI Node Registers**

NON Bridge This is the common Errorlog frame used for all PCI devices. Contents of these fields are defined in the PCI Specification rev 2.0. PCI Frame Size is size of frame including optional registers. Bit to Text translator must determine Device Type to understand how to display the optional registers (if any). The Status register in the subpacket will contain dynamic error and status information.

⁹³ OpenVMS and Digital UNIX do not plan on enabling interrupts to support this packet.

31.....24	23.....16	15.....08	07.....00	Offset
PCI Device Identification (LO)				00
PCI Device Identification (HI)				04
PCI Frame Size				08
Device ID		Vendor ID		0C
Status		Command		10
Class Code			Revision ID	14
BIST	Header Type	Latency T.	Cache Line #	18
Base Address Registers (hi)				1C
Base Address Registers (lo)				20
Base Address Registers (hi)				24
Base Address Registers (lo)				28
Base Address Registers (hi)				2C
Base Address Registers (lo)				30
Base Address Registers (hi)				34
Base Address Registers (lo)				38
Base Address Registers (hi)				3C
Base Address Registers (lo)				40
Base Address Registers (hi)				44
Base Address Registers (lo)				48
RESERVED				4C
RESERVED				50
Expansion ROM Base Address				54
RESERVED				58
RESERVED				5C
Mac_Lat	Min_Gnt	Interrupt P	Interrupt P	60
Optional Registers				64
				nn

Figure 5-77 PCI Node Subpacket Format(NON-Bridge)

5.11.3.5 PCI Bridge Node Registers

Fields for the PCI Bridge Subpacket are defined in the PCI to PCI Bridge Architecture Spec rev 0.4. PCI Frame Size is the size of the frame (no optional registers).

31.....24	23.....16	15.....08	07.....00	Offset
PCI Device Identification (LO)				00
PCI Device Identification (HI)				04
PCI Frame Size				08
Device ID(482h)		Vendor ID(8086h)		0C
Status		Command		10
Class Code(060499h)			Rev ID	14
BIST	Header Type	Latency T.	Cache Line #	18
Base Address Registers				1C
Base Address Registers				20
Sec Lat Tmr	Sub Bus #	Sec Bus #	Pri Bus #	24
Secondary Bus Status		IO Limit	IO Base	28
Memory Limit		Memory Base		2C
Prefetch Mem Limit		Prefetch Mem Base		30
Prefetchable Base Upper 32 Bits				34
Prefetchable Limit Upper 32 Bits				38
IO Limit Upper 16 bits		IO Base Upper 16 bits		3C
RESERVED				40
Expansion ROM Base Address				44
Bridge Control		Int Pin	Int Line	48

Figure 5-7 PCI Bridge Subpacket Format

5.11.3.6 ESC Subpacket Format

This frame contains copies of registers in the EISA System Component (ESC) chip that may be of use when trouble shooting a problem. The ESC Subpacket will always follow the PCI snapshots if both are present in the error log entry. The inclusion of this frame will be depicted by the setting of Software flag bit 32 in the Software Flags word, see Table 5-22 Software Flag Definition, on page 5-153.

31.....16		15.....08	07.....00	
SBZ	45h	53h	43h	= ASCII "ESC"
Byte_Count				
PCI Device Identification(LO)				= Physical Device ID
PCI Device Identification(HI)				
SBZ			RID	= RevID
SBZ			MS	= Mode Sel
EISAID3	EISAID2	EISAID1	EISAID0	= EISAID Registers
SBZ			SGRBA	= Scatter-gather
PIRQ3	PIRQ2	PIRQ1	PIRQ0	= PIRQ Route CSRs
SBZ			<7:00>	= NMISC
SBZ			<7:00>	= NMISC
SBZ			<7:00>	= LEISAMG

Figure 5-78 ESC Subpacket Format

5.11.4 Rawhide Bus Snapshot Subpackets

5.11.4.1 PCI Bus Snapshot

This snapshot is used whenever a error is discovered in one of the IOD's PCI error registers during a PCI parse. PCI Node Snapshot Registers consist of either PCI Bridge Subpacket or PCI Node Subpacket for each node present on the PCI bus.

63.....32		31.....00
Number of nodes	Physical Bus ID	Snapshot Size
PCI Node Snapshot Registers (variable Length)		

Figure 5-79 PCI Bus Snapshot Format

5.11.4.2 MC_Bus Snapshot

The MC_Bus Snapshot contains a IOD Register Subpacket for each Bridge on the MC_Bus. Neither the CPU nor memory contain any MC_Bus readable registers, therefore no state is collected for these nodes.

63.....31	31.....00	Offset
Nodes Present	Snapshot Size	00
Base Physical I/O Address of reporting CPU		08
MC_Bus Register Subpacket for Each Bridge on the MC_Bus		10
		nn

Figure 5-80 Rawhide MC_Bus Snapshot Subpacket Format

- *Nodes Present* refers to bridge nodes present on the MC_Bus. CPU and memory are not specified here.
- a) *Snapshot Size* is the total size of the MC_Bus Snapshot

6. Power-Up And Initialization

This chapter describes the initialization performed by system firmware at power-up. For now, only initialization performed by Alpha-based Rawhide products is described.

[Note: a detailed power-up initialization and test strategy memo which attempts to look forward and address global bus issues has been drafted and posted in Rawhide note 149. Refer to the strategy memo for more information.]

[Editor's note: ARC firmware initialization will either be described in a separate document or included in this chapter in a subsequent revision of this document.]

6.1 Firmware Overview

Firmware on Rawhide consists of several distributed firmware components:

1. EV5 serial ROM (SROM) : 8KB of code in a 16 KByte serial ROM located on each processor card.
 2. System FEPRM : 1MB of storage in a flash EPROM located on the Saddle (hangs off the Xbus). This ROM contains several distinct firmware images: SRM console, OpenVMS AXP PALcode, DEC OSF/1 AXP PALcode, ARC firmware, Intel BIOS emulator, and a small piece of code commonly referred to as "extended SROM". The 1MB FEPRM has sixteen separately flashable 16 KByte sectors.
 3. BIOS extension option ROMs on select options such as VGA cards.
 4. Device controller firmware, resident on various intelligent device controllers such as most DEC SCSI devices.
- This chapter discusses only those firmware components associated with #1 and #2 above.

6.2 Power-up Sequence Overview

There are essentially three phases of the firmware initialization/test sequence:

- EV5 serial ROM load and initialization/test performed by SROM, including extended SROM load.
- Initialization/test performed by extended SROM, including SRM console firmware load.
- Initialization/test performed by console firmware.

Following a power-up or reset, serial ROM code is loaded into EV5 i-cache. Serial ROM code performs some limited initialization and testing, and then loads additional code from system FEPRM. This additional code is referred to as extended SROM and is responsible for initialization and testing of enough of the hardware to permit console to be loaded. Each processor executes its own local copy of SROM and extended SROM code. A primary CPU is elected based upon power-up test results; this CPU is responsible for copying the console from FEPRM into system memory and then jumping to it. The primary, now running console, instructs secondary CPUs to begin running the console as well. In an ARCboot situation, the extended SROM loads the ARC firmware from FEPRM and then transfers control to it.

6.3 Initialization and Testing Performed By Serial ROM

Serial ROM code is responsible for the following initialization steps:

- Initialize and test the EV5.
- Size the MCBus.
- Perform primary processor determination.
- Enable I/O interrupts. [For now, IOD's write interrupt vector functionality is left disabled. Power-up code relies upon the INT_VECT CSR for interrupt vector information. Once PALcode is loaded along with system console, and the appropriate data structures are built, the IOD's write interrupt vector feature will be enabled.]

- Look for a PCI/EISA bridge. [Power-up code expects to find an EISA bridge chip set on PCI0. If not present or broken, this is a fatal error.] The path is tested by first insuring the device is found and then performing longword writes/reads the a bridge chip CSR by using PCI configuration address space.
- The EISA bridge is initialized to allow access to system FEPROM, the I²C bus, and the operator control panel (OCP). [From this point on during the power-up sequence, initialization/test code will report trace messages to the OCP display to help with problem isolation.]
- CPUs perform system FEPROM pattern tests and load extended SROM into EV5 s-cache (with s-cache FHIT enabled). Extended SROM is checksum-tested during the load and is started if testing succeeds.

6.4 Initialization and Testing Performed By Extended ROM

Once all processors in the system have begun executing extended SROM, they then perform an additional series of initialization and test steps before the console is loaded from system FEPROM. The following describes the steps performed by extended SROM:

- Extended SROM flushes i-cache and then walks through itself via a series of branch instructions to insure every cache block is cached in i-cache. Up to this point, no CPU should have issued any external (off-processor) memory reference. Backup cache, if present, and main memory are still untested and uninitialized.
- Each CPU now checks for b-cache presence and size by reading the IOD's WHOAMI CPU_INFO field. Extended SROM code will support any size b-cache, including no cache. CPUs test b-cache with FHIT set, making certain all cache locations are marked unshared during testing. Tests performed include: b-cache data, b-tag, and good/bad ECC tests. [Duplicate TAGs are initialized by hardware after reset.]
- Each CPU now sizes SIMM memory by accessing the motherboard's I²C memory configuration register. If a mismatched SIMM pair is detected an error message is displayed on the OCP.

The primary CPU initializes and tests all of SIMM memory. Once done, it instructs secondary CPUs to in turn, execute a subset of these same tests (for address line and data path testing on each CPU).

Main memory's ability to wrap on higher memory space addresses allows extended SROM test code to verify all address lines, irrespective of how much memory is configured in the system.

During memory testing, extended SROM builds memory descriptors and lists of bad page information for each memory region (i.e. holes are accounted for). This information is passed to console firmware later during the power-up sequence. Console, in turn, uses this information to build its memory descriptors and bit map which it passes to higher level software.

An environment variable will be supported to allow enabling/disabling memory testing during power-up. If disabled, a 32MB portion of memory will still be initialized for use by firmware for firmware and bootstrap load/execution.

- Once memory testing has been completed, the primary performs basic I/O testing targeting the Horse module and the Horse-Saddle connector. Loop-back tests are performed on each PCI. In addition, various error tests are used to validate error detection/recovery. If all testing for a given PCI succeeds, the Horse's STP LED is lit.
- Extended SROM now loads system console into main memory. It does so by loading and starting decompression code, which in turn loads the console proper.
- Once the primary has loaded and started console, it instructs all secondaries to jump to console.

6.5 Initialization Performed By Console

Once loaded and started, console firmware begins sizing and configuring the I/O subsystem. The primary CPU sizes all PCI buses in the system, beginning with PCI bus zero. All PCI buses spawned by the MC-PCI bus bridges are treated as secondary buses, i.e. Type 1 configuration cycles are used. The bus sizing algorithm is:

For each MC-PCI bus bridge, and starting with IOD0, issue Type 1 configuration cycles to the first PCI bus spawned by the IOD. Do so for IDs zero through five on that bus.

If a PCI-PCI bridge is found, the subordinate bus is fully sized, again with Type 1 configuration cycles, before the primary bus sizing is resumed.

Repeat steps (1) and (2) for the next PCI bus on that same IOD.

Repeat steps (1) through (3) for the other IOD.

The primary expects to find a PCI-EISA bridge chip set (82374/82375) as the first device on PCI0. The EISA bridge registers are initialized, as are Xbus and associated junk I/O devices, before any further PCI bus sizing is performed.

EISA bridge chip register initialization is described in Table 6-1.

Register	Firmware-initialized State
Mode Select	SERR enabled, 8 EISA masters, configuration RAM access enabled
BIOS Chip Select A/B	Disable access to motherboard BIOS
Peripheral Chip Select A	Enable RTC, keyboard controller, and floppy disk controller decode
Peripheral Chip Select B	Port 92 and CRAM decode enabled, COM1 at 0x3F8-f, COM2 at 0x2F8-f, LPT at 0x3BC-f
EISA motherboard ID	TBD
General Purpose Chip Select 0	Set up I ² C controller access
General Purpose Chip Select Mask0	Select address 0x530-1 as legal addresses for I ² C controller
General Purpose Xbus Control	General purpose chip select 0 is enabled, GPCS 1-2 are disabled
NMI Status and Control	Clear any system board NMI errors enable all NMI types but PCI system board errors
PCEB Master Latency Timer	Maximum count value
PCEB EISA to Memory Attributes	Buffer memory region 1
EISA-PCI Memory Region 1	Map region 1 to full 4GB PCI memory space
Internal 8259s	Enable appropriate IRQs and initialize interrupt controllers (cascade the controllers together, set timer and EISA bus refresh rates, etc.)

Table 6-1 (EISA Bridge Chip set Initialization)

Console sizes and configures the EISA bus using the following steps:

- 1) Copy NVRAM contents to memory and verify checksum.
- 2) Decompress the ECU data.
- 3) Initialize EISA slot table (of all previously existing EISA bus adapters in system since last power-up).
- 4) Read port IDs to verify present configuration matches that which was saved (if a mismatch, the user must run ECU).
- 5) Configure all adapters for which there is a match including edge/level for interrupts, DMA timing and transfer sizes, etc.
- 6) For each adapter, if all is okay, enable the adapter.

Console expects the following devices and initializes them as they are found:

- COMBO devices (87312) - COM1, COM2, Parallel Port, Floppy Controller
- Keyboard/Mouse chip (8242)

- Battery backed-up SRAM (1225)
- TOY clock (DS1287)

Next, the primary allocates I/O space and dense/sparse memory space to PCI devices that need it. For each PCI bus there are three base address queues generated by console during PCI bus configuration; one for IO space, and one each for sparse and dense memory space. Each queue contains a sorted list of address space requirements for each device and subordinate bridge on that PCI. The queue is sorted in descending order of alignment. Console traverses the bus and queries each device for address space requirements.

Once the address space requirements are known, the console configures the devices on that PCI by writing the I/O and memory base address registers for each PCI device (based upon the address space needs of that device). Sorting the queue entries guarantees base addresses are naturally aligned.

As base addresses are configured, other parameters are initialized as well (cache_line_size and latency_timer values). In addition, devices are enabled via the command register to respond to I/O space and memory space accesses; bridges are initialized to respond only to memory space accesses.

The aforementioned steps are repeated for each PCI in the system.

Once all PCI devices downstream from the MC-PCI bus bridge are configured, console configures the MC-PCI bridge registers as follows:

- PCI_LAT is initialized as specified in Chapter 4.
- CAP_CTL is initialized to enable PCI transactions in both directions, parity, and ECC checking/reporting as specified in Chapter 4.
- SG_TBIA is written to invalidate scatter/gather translation buffers.
- HAE_MEM and HAE_IO are left as initialized by the hardware (0).
- CAP_ERR is checked and then cleared, if necessary, to unlock the IOD's error registers.
- The four sets of Wn_BASE, Wn_MASK, and Tn_BASE registers are written as specified in section 3.2.2.1

6.6 CSR and IPR Initialization

Refer to Chapter 4 for firmware-initialized CSR and IPR values.

6.7 I/O Space Address Map

Beyond any ISA devices that reside on the EISA bus, there are no fixed addresses that are assigned on a per ID basis. Configuration performed by the console must be used to determine the types of devices in the system and their dynamically-assigned location in the system.

ISA device address map (sparse I/O space) to be supplied.

6.8 Software Considerations

If system software modifies any of the configuration base addresses set up by console, this will cause the console callbacks, as well as the console program itself, to give unpredictable results. Console software expects to find particular device controllers where they were last left.